1. Sherbook API Protocol . . . 2
1.1 Introduction . . . 3
1.1.1 Basic requests and responses . . . 9
1.1.2 Basic workflow . . . 12
1.1.3 Data transfer objects . . . 16
1.1.4 Response status values . . . 24
1.1.5 Protocol changes . . . 26
1.1.6 Security . . . 28
1.2 1. Retail user registration . . . 30
1.3 2. Login process . . . 34
1.4 3. Credentials checking . . . 42
1.5 4. Profile . . . 44
1.6 5. Loading system settings . . . 48
1.7 6. Searching addresses . . . 56
1.8 7. Address book . . . 63
1.9 8. Airports and terminals . . . 74
1.10 9. Train stations and meeting points . . . 80
1.11 10. Special Places and meeting points . . . 83
1.12 11. Service management . . . 87
1.13 12. Payment types . . . 92
1.14 13. Special instructions . . . 94
1.15 14. References . . . 98
1.16 15. Airlines . . . 101
1.17 16. Discounts . . . 103
1.18 17. Contact book . . . 108
1.19 18. Starting booking creation . . . 114
1.20 19. Price calculation . . . 115
1.21 20. Discount condition validation . . . 120
1.22 21. Journey time calculation . . . 122
1.23 22. Prebook limit checking . . . 124
1.24 23. Delay calculation . . . 128
1.25 24. Job offer workflow . . . 132
1.26 25. Credit card management . . . 137
1.27 26. Booking confirmation . . . 142
1.28 27. Booking history . . . 146
1.29 28. Booking details modification . . . 155
1.30 29. Booking cancellation . . . 156
1.31 30. Complaints . . . 158
1.32 31. Passenger survey . . . 160
1.33 32. Tips management . . . 165
1.34 33. Lost property . . . 167
1.35 34. Push notifications . . . 170
1.36 35. Resources . . . 174
1.37 36. Languages . . . 176
1.38 37. Driver job changes . . . 177
1.39 38. Chat with a driver . . . 181
1.40 39. Quotation Calculation . . . 184
1.41 40. Cruise terminals and berths . . . 188
1.42 41. Supervisors . . . 192
1.43 42. Terms&Conditions and Privacy Policy acceptance . . . 196
1.44 43. PayPal payments . . . 197
1.45 44. Route and Duration calculation . . . 201
1.46 45. Referral program . . . 203
1.47 46. Off-Session Payments . . . 207

# Sherbook API Protocol

This document is the official reference for the Sherbook Application Programming Interface. Sherbook API is considered as a subsystem of Sherlock designed to accept, store and load bookings and related information from and to smartphone booking applications (SBA).

Search Sherbook API documentation:

# Introduction

Sherbook API is considered as a subsystem of Sherlock designed to accept, store and load bookings and related information from and to smartphone booking applications (SBA).

## Basics

Basic workflow

Basic requests and responses

Response status values

Data transfer objects

## Usage of customer types

The 'Customer type' field is used to determine data related to different user types and to decrease the amount of requests. There are three types of Sherbook requests:

- Requests with no customer type specified – returned data does not depend on the customer type;
- Requests with a single customer type – data in such requests and responses depends on the customer type;
- Requests with a list of customer types – specifying a list of customer types allows to load data for several authenticated customer types in a single request/response.

URLs of requests subdivided by type are specified in the table below:

| Requests without the CustomerType field | Requests with a single CustomerType (CustomerRequest) | Requests with a list of CustomerType's (CustomerDataRequest) |
|---|---|---|
| /addresses/airports/terminals | /addresses/book/add | /login/logout |
| | /addresses/book/delete | /settings/booking |
| /addresses/search/position | /addresses/book/edit | /addresses/book/load/last |
| /addresses/search/query | /booking/cancel | /addresses/book/load/favourites |
| /addresses/special/places | /booking/cancel/charge | /addresses/airports/full |
| /addresses/special/points | /booking/load/details | /addresses/airports/terminals/meetingPoints |
| /addresses/special/types | /booking/load/driverPosition | /data/services/available |
| /settings/system | /booking/load/version | /data/services/load |
| /addresses/trainStations/meetingPoints | /calculate/delay | /data/paymentTypes |
| /credentials /retail | /calculate/journeyTime | /creditCard/load |
| /credentials/corporate | /calculate/pickupDelay | /discount/loyaltyCards |
| /creditCard/delete | /calculate/prebookLimit | /discount/vouchers |
| /instructions/load | /calculate/price | /contactBook/load/available |
| /login/anonymous | /complaint/create | /booking/load/active |
| /login/corporate | /complaint/load | /booking/load/active/full |
| /login/retail | /creditCard/delete | /booking/load/history |

| | | |
|---|---|---|
| /profile/details/load | /creditCard/save | /booking/load/history/full |
| /profile/details/save | /creditCard/save/complete | /addresses/airports |
| /profile/password | /discount/promoCode/validate | /addresses/trainStations |
| /profile/phone | /discount/voucher/validate | |
| /reference/schema | /instructions/validate | |
| /references/option | /job/book | |
| /references/validation | /job/payment/complete | |
| /registration/check | /offer/cancel | |
| /registration/new | /offer/check | |
| /registration/recovery | /offer/make | |
| /resources/get | /survey/confirm | |
| /resources/load | /survey/load | |
| | /survey/result | |
| | /tips/range | |
| | /tips/submit | |
| | /notification/token | |
| | /notification/confirm | |

# HTTP codes returned by Sherbook

1. 200 OK – the request has been successfully processed.
2. 400 BAD REQUEST – an error has occurred while parsing JSON.
3. 401 UNAUTHORIZED – an authentication error: the customer cannot be authorised on the server side.
4. 403 FORBIDDEN – an authentication error: an invalid token has been specified for the retail user request.
5. 404 NOT FOUND – an incorrect web method has been called.
6. 405 METHOD NOT ALLOWED – a GET request is received while the method can be called by a POST request only, or vice versa.
7. 415 UNSUPPORTED MEDIA TYPE – the entity of the request is in a format not supported by the requested resource for the requested method.
8. 500 INTERNAL SERVER ERROR – an internal error has occurred on the server side (a bug, a technical defect, etc).

# Protocol definition

1. HTTP/HTTPS transport protocol is used.
2. JSON data transfer representation is used.
3. POST methods are only supported.
4. UTF-8 is used as request encoding.
5. HTTP Header Content-Type – "application/json";
6. Data compression:
    a. The server supports only 'gzip' data compression algorithm.
    b. The client sends to the server:
        i. "Accept-Encoding: 'gzip'" header – in case when it accepts data compressed with the 'gzip' algorithm.
        ii. "Content-Encoding: 'gzip'" header in case request data is compressed with the 'gzip' algorithm.
    c. The server sends "Content-Encoding: 'gzip'" header to the client in case response data is compressed with the 'gzip' algorithm.
7. The client should accept and store cookies received from the server for operation under HTTP-session (attrtibute JSESSIONID).
8. Date format is "dd/MM/yyyy HH:mm". Please note, the server restores a date from the string using its own time zone.
9. Enumeration values should be transferred as short strings or integer values, as it is shown below.

- Enumeration with short strings: enum AddressSearchType { PICKUP("P"), DROPOFF("D");}
- Enumeration with integer values: enum BookingStatus { BOOKED(0), DRIVER_ON_WAY(10), DRIVER_AT_PICKUP(20), PASSENGER_ON_BOARD(30), DONE(40), CANCELLED(50); }

JSON object in first case is required to contain one of the "P" or "D" values and one of the 0, 10, 20… 50 values in the second case.

# Client side data caching

Sherbook supports caching data on the client side. For this purposes in cacheable data loading methods the server accepts and returns an additional field which stores information about the last synchronisation date.
In the initial method call the client sends the empty **lastUpdateDate** field. Next, in a response, the server sends all cacheable data and the **lastUpdateDate** field with the current server time which should be stored on the client side. In future method calls the client sends the **lastUpdateDate** value to the server and receives all corresponding data changed since the previous data loading with the refreshed **lastUpdateDate** field.
The following data can be cached:

- Previously used addresses;
- Airports and terminals;
- Train stations and train station meeting points;
- Special Place types, Special Places,Special Place meeting points;
- Archive bookings.

Information about data deleted since the last synchronisation is loaded in the response in a separate field ('**deleted**') containing identifiers of deleted entries.
Basic requests and response object definitions for cacheable data loading are specified below.

# Terms and conditions

**SBA** – a smartphone booking application integrated with Sherbook.
**Smartphone device, smartphone** – a mobile telephone with the installed SBA. Sherbook supports the following smartphone operation systems: Android, iOS, Windows Phone.
**User** – a person who uses SBA to make a booking and manage information related to bookings.
**CustomerType** – a type of the authenticated user. It is used to determine data related to different user types. The following customer types are considered:

- **RETAIL** – a user registered in the Sherlock system and authenticated using a phone number and a verified password;
- **CORPORATE** – a corporate user authenticated using credentials given by his corporate account manager;
- **ANONYMOUS** – a user not registered in the Sherlock system and authenticated using smartphone device information.

**Token** – a value returned to a user with the RETAIL customer type after each successful authentication. It is used to restore a session on the server after it was outdated without re-authentication.
**BookingOperationType** – a type of booking operation made. Possible operation types are specified below:

- **CREATE** – a user creates a new booking;
- **AMEND** – a user modifies the existing booking;
- **REPEAT** – a user creates a new booking on the basis of the existing one.

**usedJobId** – an identifier of the previously created booking sent to the server in case of the AMEND and REPEAT booking operation types.

# Basic requests and responses

Request and response objects given below are used in different methods as input/output parameters through the whole Sherbook specification.

When processing particular requests Sherbook takes into account the type of client which is used (a mobile device or Web Portal). If a device is specified upon login, it is considered that the mobile device is used. Otherwise, if not specified, it is considered that Web Portal is used. It is possible to define the particular client type which is used (for instance, the registration web-form). To do this, it is required to specify the 'CLIENT-ID' http header in a request (max 16 characters). The received response will be displayed in sherbook.log and sherbook-large.log after the session ID (alongside with MOBILE-WS or WEB_PORTAL). The received value can be also used for functionality customisation.

## CustomerRequest and CustomerDataRequest definitions

Request objects specified below are used to generalise Sherbook method calls. The difference between these two definitions is that the second one is used in methods which allow to perform an operation for several authorised customer types, while the first one is for methods in which only the single authorised customer type is allowed.

### CustomerRequest

| Parameter | Type | Description |
|---|---|---|
| **customerType** | CustomerType | Customer type |

### CustomerDataRequest

| Parameter | Type | Description |
|---|---|---|
| **customerTypes** | List<CustomerType> | List of customer types |

### AmendableCustomerDataRequest

| Parameter | Type | Description |
|---|---|---|
| **customerTypes** | List<CustomerType> | List of customer types |
| amendJobId | UUID | Amend job id |

### Usages

Logout
Load booking settings
Load available payment types
Loyalty cards
Voucher loading
Load available passengers
Load available credit cards
Load active bookings
Load full active booking information

## BookingIdRequest definition

**BookingIdRequest** is similar to **CustomerRequest** but contains a field that stores a booking identifier. It is used in methods that perform operations with a single booking related to a specific customer type.

### BookingIdRequest

| Parameter | Type | Description |
|---|---|---|
| **id** | UUID | Booking identifier |
| **customerType** | CustomerType | Customer type |

### Usages

Load detailed information for a single booking
Refresh booking details
Refresh driver details
Load a complaint
Load a survey
Load a survey result
Load a tips range

## BaseResponse definition

A base definition for responses sent to the client: **ALL** Sherbook responses are inherited from this object definition and contain the 'status' and 'errorMessage' fields.

Note: Do not rely on the 'errorMessage' field of the response in the client logic, use the 'status' field instead. Description can be changed on the server at any time.

### BaseResponse

| Parameter | Type | Description |
|---|---|---|
| status | ResponseStatus | Response status returned by the server |
| errorMessage | String | Description of the occurred error. Filled in if the status is not 'OK' |

### Usages

Register a new user
Get a new password
Check whether a user is already registered
Login an anonymous user
Logout
Check retail credentials
Change a name and an email
Change a password
Change a phone number
Delete an address
Instruction validation
Cancel an offer
Delete an existing card
Hide bookings in the Booking History list
Create a complaint
Confirm a survey
Submit tips
Set a push notification token
Set a language

## Base request to retrieve cacheable data

A request specified below is used to load cacheable data hierarchy, i.e. airports and terminals changed or deleted since the specified date.

### BaseDatedRequest

| Parameter | Type | Description |
|---|---|---|
| lastUpdateDate | Date | Timestamp of the last data loading |

### Usages

Load airport hierarchy
Load train station hierarchy
Load Special Place hierarchy

Load cruise terminal hierarchy

## Address hierarchy response

A response specified below contains the current server time and address data changed and deleted since the date specified in the corresponding request or whole data in case the date of last loading is not specified.
There are three hierarchy address groups:

1. Airport hierarchy:
    - Airports
    - Airport terminals
2. Train station hierarchy:
    - Train stations
    - Train station meeting points
3. Special Place hierarchy:
    - Special Place types
    - Special Places
    - Special Place meeting points
4. Cruise terminals hierarchy:
    - Cruise terminals
    - Berths

Corresponding address sources used for each of the addresses described above are specified in the AddressSource table.

### AddressHierarchyResponse

| Parameter | Type | Description |
|---|---|---|
| lastUpdateDate | Date | Current server time |
| addresses | List<RelatedAddressDto> | Loaded address hierarchy |
| deleted | List<UUID> | Identifiers of deleted addresses |

## Usages

Load airport hierarchy
Load train station hierarchy

Load cruise terminal hierarchy

# Basic workflow

For creating and managing bookings a user should register and log in to Sherlock. There are retail and corporate users in the system. Registration is supported only for retail users.

Before registration and login it is required to check whether the application version is the latest or outdated. If the version is outdated, a user gets a link to download the latest version. More details can be found in Credentials checking.

A phone number, a name and an email are required for retail user registration. If all input data is valid and no user with the same phone number has been registered yet, a new user will be registered. An SMS with a password is sent to the user from the server. This password can be changed by the user. The 'RegistrationInfo' instance is created for the user after registration. Retail user registration
The retail user registration scheme is presented on Figure 1.

Figure 1. The retail user registration scheme

A phone number and a password are used for retail user login. If input data is correct, a user will be authenticated successfully. After login the user can modify profile details, create and manage bookings.

Each corporate account can use its own login scheme. The only difference between the login schemes for accounts is a set of required fields. The following fields can be used: an account number (always required), PIN and caller. Login process

Retail and corporate user login schemes are presented on Figure 2.

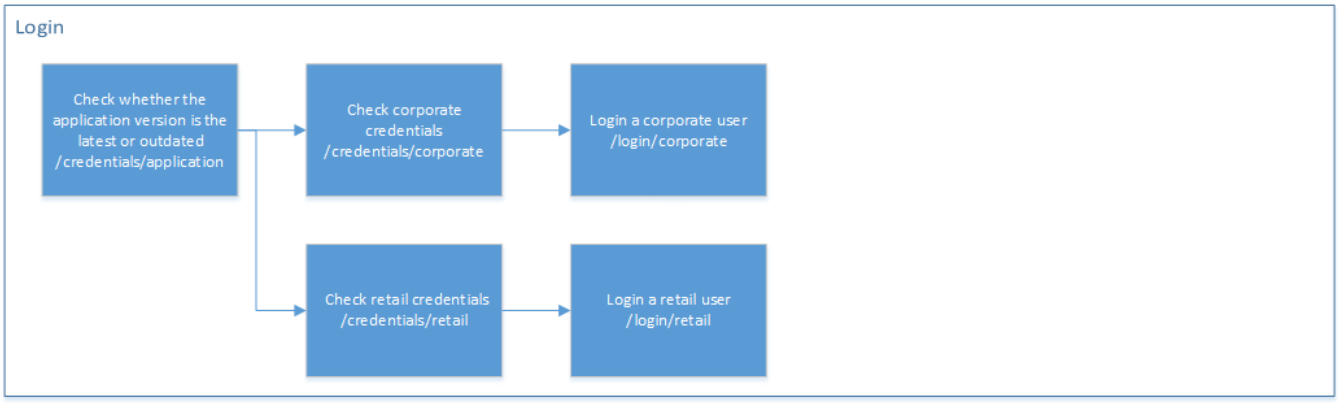


Figure 2. The user login scheme

After successful authentication of the user, the following mandatory initial operations (Figure 3) should be performed:

1. Load a name and an email: Edit profile details
Profile details loading is supported only for retail users.

2. Load booking and system settings Load booking and system settings

4. Load available payment types: Payment types
Each user can have different booking payment types such as cash, credit cards, etc.

5. Set a push notification token: Push notifications
In case of login via a smartphone, a push notification token will be sent to the server. The server will save this token and use it later to send push notifications.

6. Load available services: Service management

7. Get a resource link
To load images for available services mentioned above a customer should request them: Resources

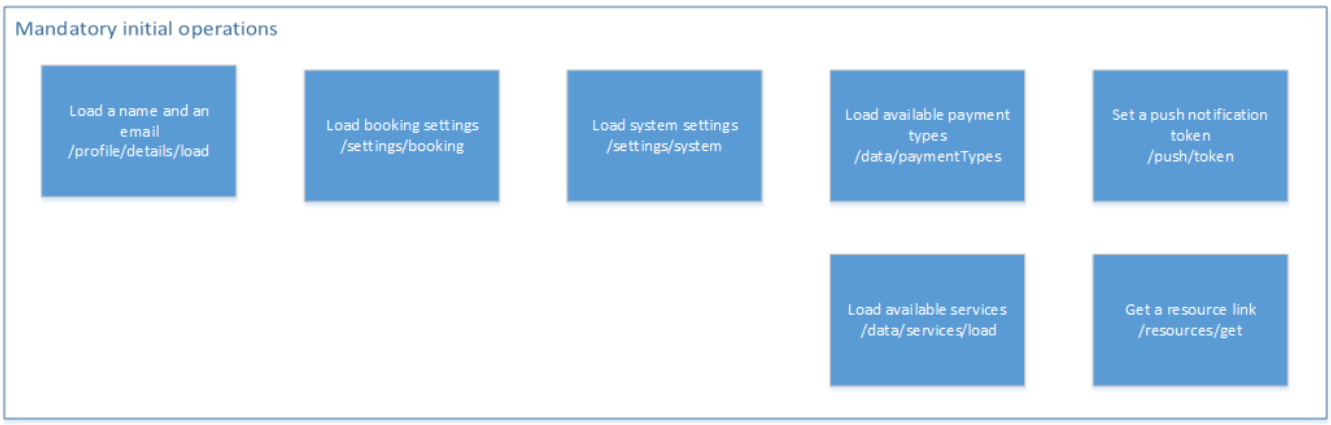


Figure 3. Mandatory initial operations

When these actions are performed, the user is available to create bookings. There are two types of bookings: ASAP and prebook. A booking has some parameters including pickup and dropoff stops. If a current geo position is defined, it will be marked as a pickup stop on the map. Also for this point (Figure 4):

1. Search addresses by position: Searching addresses
2. Check service availablity: Service management
3. Calculate ASAP delay: Delay calculation

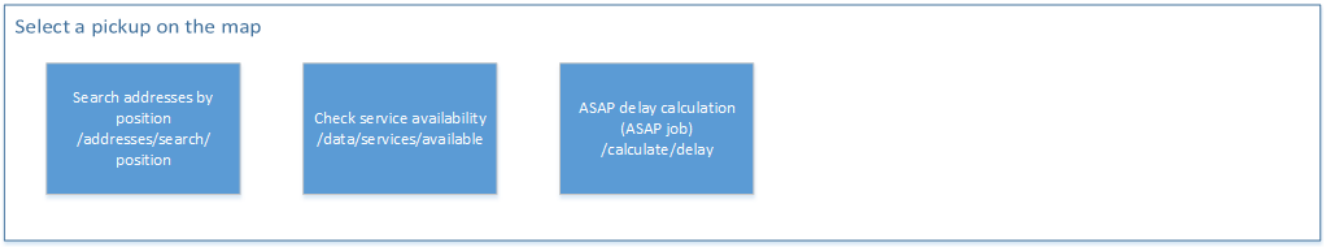


Figure 4. Select a pickup on the map

The user can select addresses for booking stops (pickup, dropoff) from the following lists (Figure 5):

1. Airport hierarchy
Airport hierarchy contains airports and terminals.
A terminal can have meeting points.
Airports and terminals

2. Train station hierarchy
Train station hierarchy consists of train stations and meeting points.
Train stations and meeting points

3. Recently used and favourite addresses: Address book

4. Special Places and Special Place types
To select a Special Place the user should firstly select a Special Place type.
Special Place can have meeting points.
Special Places and meeting points

5. Addresses found by string.
The user has an opportunity of searching addresses by string.
Searching addresses

Figure 5. Booking stops

When pickup/dropoff addresses and a service are selected, booking creation has started. Starting booking creation

If job is a prebook, a booking date should be also specified. Further different booking values are calculated. For an ASAP job one of the values is a delay. If a pickup stop is an airport or a train station, an arrival delay should be calculated. Delay calculation

Prebook limit checking should be preformed for a prebook job. Prebook limit checking

The next step is journey time calculation. Journey time calculation

The final step is calculation of a booking price. Price calculation

All steps specified above are presented on Figure 6.

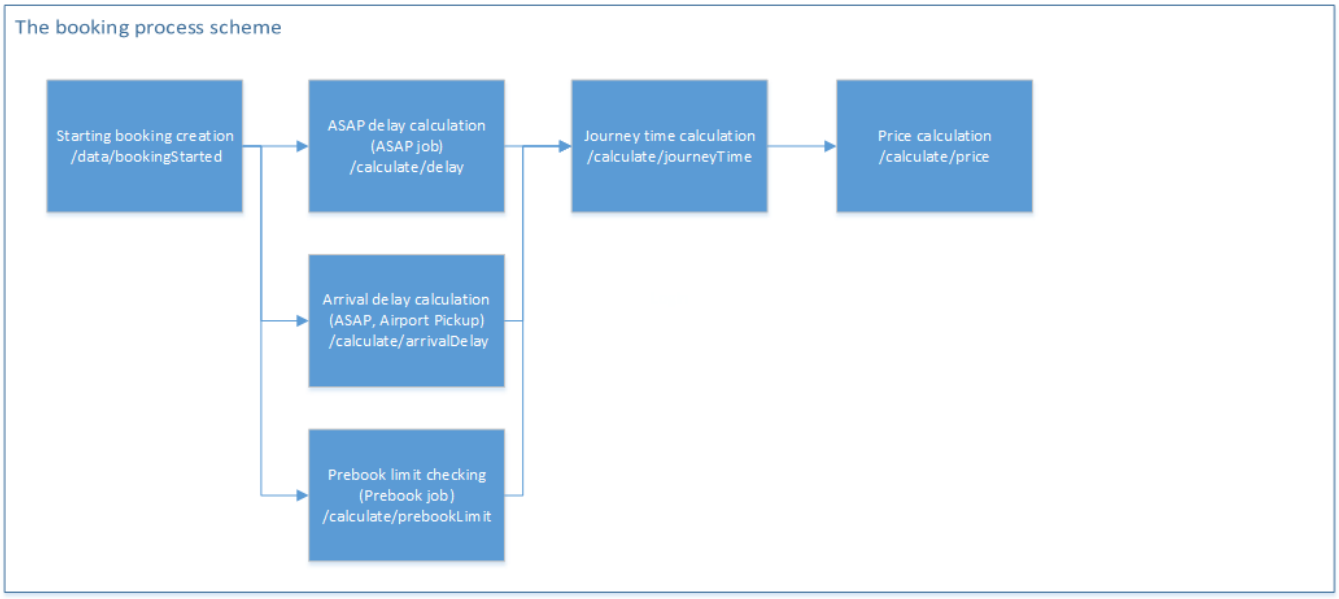


Figure 6. The booking process scheme

After all calculations are finished, a customer could confirm his booking (Figure 7). Booking confirmation

Figure 7. Booking confirmation

In case some of the booking parameters are changed, booking values from the previous steps should be recalculated.

# Data transfer objects

## 1. Smartphone device information

This section provides information about common objects sent from the server to the client and back.

Information about a smartphone used by the client is sent to the server in the DeviceDto table below, required parameters are marked as **bold**.

### DeviceDto

| Parameter | Type | Description |
|---|---|---|
| **imei** | String | Device IMEI |
| **deviceId** | String | Unique device identifier |
| **osId** | String | Device operating system identifier |
| osVersion | String | Device operating system version |
| **appVersion** | String | Version of application |
| **appId** | String | Application identifier |
| screenWidth | Integer | Device screen width |
| screenHeight | Integer | Device screen height |
| fingerprint | String | Device fingerprint |
| **type** | DeviceType | Type of the device. Should be selected from DeviceType enumeration |

### DeviceType

| Value | Description |
|---|---|
| ANDROID("ANDROID") | Android OS based devices |
| IPHONE("IPHONE") | iOS based devices |
| WINPHONE("WINPHONE") | Windows Phone OS based devices |

### Usages

Register new user
Get new password
Login corporate user
Login retail user
Login anonymous user

## 2. Address information

Address information transfers in objects specified below. There are four types of address data objects:

- AddressDto – a base address data object containing information about an address identifier, a caption, coordinates, a source, and additional encoded data. Please note, that the address identifier is not filled in as addresses are received from Google address search, third party address search services and addresses specified by the client manually. Encoded additional data received by the client must be sent back to the server.
- RelatedAddressDto – used to send information about addresses related to some parent addresses or objects, i.e. terminals related to an airport or Special Places related to a Special Place type. Extends the AddressDto object.
- CustomerAddressDto – used to send information about addresses related to a customer type, i.e. airport terminal meeting points. Extends the RelatedAddressDto object.
- FavouriteAddressDto – stores information about favourite addresses specified by the user. Extends the CustomerAddressDto object.
- RecentAddressDto – stores information about addresses recently used by the client. Extends the FavouriteAddressDto.

- MeetingPointDto - stores information about meeting point. Extends the CustomerAddressDto object.

Tables below contain information about each of the described address objects.

### AddressDto

| Parameter | Type | Description |
|---|---|---|
| addressId | UUID | Address identifier |
| caption | String | Formatted address |
| latitude | Double | Latitude of address |
| longitude | Double | Longitude of address |
| source | AddressSource | Source of address |
| data | String | Encoded additional data |
| fields | Map<String, String> | Additional data |
| restrictions | Restrictions | Address restrictions. Equals null if no restrictions for this address |
| note | String | Notes |

### Restrictions

| Parameter | Type | Description |
|---|---|---|
| pickupRestricted | boolean | Indicates if address could not be used as pickup |
| dropRestricted | boolean | Indicates if address could not be used as drop off |
| intermediateRestricted | boolean | Indicates if address could not be used as intermediate stop |

There are the following standard additional data in 'fields':

| Key | Type | Description |
|---|---|---|
| clientNotes | String | Client Notes of Special Place Meeting Point. Only filled in if the search was performed using Special Place + Meeting Point. |

All objects below has the same fieds as AddressDto

### RelatedAddressDto

| Key | Type | Description |
|---|---|---|
| **parentId** | UUID | Related address identifier |

### CustomerAddressDto

| Key | Type | Description |
|---|---|---|
| **customerType** | CustomerType | Customer type |

### FavouriteAddressDto

| Key | Type | Description |
|---|---|---|
| favouriteName | String | Favourite address name |
| lastUsedTs | Date | Address last usage timestamp |

| | | |
|---|---|---|
| global | boolean | Indicates if address is global |

### RecentAddressDto

| Key | Type | Description |
|---|---|---|
| usageCount | Integer | Recent address usage count |
| favourite | boolean | Indicates if the recently used address is a favourite one |

### MeetingPointDto

| Key | Type | Description |
|---|---|---|
| isDefault | boolean | Specifies if meeting point is default |

### AddressSource

| Value | Description |
|---|---|
| GOOGLE_PLACE("GOOGLE_PLACE") | Address source type corresponding to an address found via Google place API |
| GOOGLE_GEO("GOOGLE_GEO") | Address source type corresponding to an address found via Google geocoding API |
| THIRD_PARTY("THIRD_PARTY") | Address source type corresponding to an address found via the third party system |
| MANUAL("MANUAL") | Address source type corresponding to a manual address received from the client |
| UNKNOWN("UNKNOWN") | Address source type is unknown; typical for manual addresses received from the client |
| ADDRESS_BOOK("ADDRESS_BOOK") | Address source type corresponding to an address found in the address book |
| AIRPORT("AIRPORT") | Address source type corresponding to an airport address |
| AIRPORT_TERMINAL("AIRPORT_TERMINAL") | Address source type corresponding to an airport terminal address |
| AIRPORT_MEETING_POINT ("AIRPORT_MEETING_POINT") | Address source type corresponding to an airport meeting point address |
| TRAIN("TRAIN") | Address source type corresponding to a train station address |
| TRAIN_MEETING_POINT("TRAIN_MEETING_POINT") | Address source type corresponding to a train station meeting point address |
| SPECIAL_PLACE("SPECIAL_PLACE") | Address source type corresponding to a Special Place address |
| MEETING_POINT("MEETING_POINT") | Address source type corresponding to a Special Place meeting point address |
| BOOKING_ADDRESS("BOOKING_ADDRESS") | Address source type corresponding to an address taken from the existing booking, amended or repeated |
| `YANDEX_PLACE("YANDEX_PLACE")` | Address source type corresponding to a Yandex address |
| `PLACE_AUTOCOMPLETE("PLACE_AUTOCOMPLETE")` | Address source type corresponding to an address found via Google Place Autocomplete API |

## Usages

Searching addresses
Address book
Airports and terminals
Trains stations and meeting points
Special Places and meeting points

# 3. Booking operation type

Booking operation type enumeration is used to determine what type of booking operation is performed.

### BookingOperationType

| 1. Value | 1. Description |
|---|---|
| CREATE("CREATE") | Create a new booking |
| AMEND("AMEND") | Amend the existing booking |
| REPEAT("REPEAT") | Repeat a booking on the basis of the existing one |

## Usages

Price calculation
Journey time calculation
Prebook limit checking
Delay calculation
Make an offer
Confirm booking creation

# 4. Booking service

Booking service object is a shorten version of ServiceDto without settings fields. Used by the client to send information about a service selected by the user.

### BookingServiceDto

| Parameter | Type | Description |
|---|---|---|
| **id** | UUID | Service identifier |
| caption | String | Service caption |
| description | String | Service description |
| **customerType** | CustomerType | Customer type |
| delivery | boolean | Indicates if service is considered as a Delivery Service |

## Usages:

Load available special instructions
Price calculation
Prebook limit checking
Delay calculation
Make offer
Confirm booking creation
Load full active booking information

# 5. Booking payment type

Booking service object is a shorten version of PaymentTypeDto without settings fields. Used by the client to send information about a payment type selected by the user.

### BookingPaymentTypeDto

| Parameter | Type | Description |
|---|---|---|
| name | String | Payment type name |

| | | |
|---|---|---|
| **code** | String | Payment type code |
| **customerType** | CustomerType | Customer type |
| payInCar | boolean | Indicates if payment to be performed in a car |

### Usages:

Price calculation
Make offer
Confirm booking creation
Load full active booking information

## 6. Stop information

### StopDto

| Parameter | Type | Description |
|---|---|---|
| id | UUID | Stop identifier |
| address | AddressDto | Stop address information |
| arrivalDetails | ArrivalDetails | Arrival details |
| notes | String | Stop notes |
| contact | ContactDto | Contact information. Required for Delivery Service booking |
| operationType | `StopOperationType` | Operation type of the stop - pickup or drop |
| deliveryDetails | DeliveryDetails | Information about delivery status. Used for delivery services |
| paymentAddrees | Boolean | Indicates if delivery payment should be taken at this address |

### ArrivalDetails

| Parameter | Type | Description |
|---|---|---|
| arrived | boolean | Indicates if the client has already arrived |
| pickupTimeDelay | Integer | Pickup time delay requested by the client |
| arrivalTime | Date | Arrival time |
| arrivalNumber | String | Arrival number |
| arrivalFrom | String | Arrival from |
| airline | AirlineDto | Airline |

### DeliveryDetails

| Parameter | Type | Description |
|---|---|---|
| status | DeliveryStatus | Status of delivery for the stop |
| date | Date | Date when parcel was taken/delivered |
| signatureUrl | String | URL for loading POD signature associated with the stop |

### DeliveryStatus

| Value | Description |
|---|---|

| | |
|---|---|
| PENDING("PENDING") | Driver has not started moving towards the stop |
| IN_PROGRESS("IN_PROGRESS") | Driver is on the way to the stop |
| PARCEL_TAKEN("PARCEL_TAKEN") | Driver received the parcel |
| PARCEL_DELIVERED("PARCEL_DELIVERED") | Driver delivered the parcel |
| PARCEL_NOT_TAKEN("PARCEL_NOT_TAKEN") | Driver passed the stop but did not received the parcel |
| PARCEL_NOT_DELIVERED("PARCEL_NOT_DELIVERED") | Driver passed the stop but did not delivered the parcel |

### AirlineDto

| Parameter | Type | Description |
|---|---|---|
| id | UUID | Airline identifier |
| name | String | Airline name |
| iata | String | IATA airline code |
| icao | String | ICAO airline code |

## Usages:

Price calculation
Journey time calculation
Prebook limit checking
Delay calculation
Make an offer
Confirm booking creation
Load full active booking information

# 7. Route information

### RouteInfo

| Parameter | Type | Description |
|---|---|---|
| waitAndReturn | boolean | 'Wait and return' journey |
| destinationUnknown | boolean | 'Destination unknown' journey |
| asDirected | boolean | 'As directed' journey |
| asDirectedMinutes | Integer | Planned 'As directed' minutes |

## Usages:

Price calculation
Journey time calculation
Make an offer
Confirm booking creation
Load full active booking information

# 8. Contacts and passengers

### ContactDto

| Parameter | Type | Description |
|---|---|---|
| customerType | CustomerType | Customer type |

| | | |
|---|---|---|
| individualId | UUID | Individual identifier |
| name | String | Contact name |
| phone | String | Contact phone |
| email | String | Contact email |

### Usages:

Available passengers
Make an offer
Confirm booking creation
Load full active booking information

## 9. Language

### LanguageDto

| Parameter | Type | Description |
|---|---|---|
| locale | String | Locale. Ex.:en_US |
| caption | String | Language/ Ex.: American English |

### Usages:

Set a language
Load available languages

## 10. Contact

### LanguageDto

| Parameter | Type | Description |
|---|---|---|
| name | String | Name |
| phone | String | Phone |
| email | String | Email |
| individualId | UUID | UUID of the existing contact |
| customerType | CustomerType | Customer Type |

### Usages:

Set a language

## 11. Partner integration

### PartnerDto

| Parameter | Type | Description |
|---|---|---|
| caption | String | Supplier caption |
| city | String | Supplier city |

| | | |
|---|---|---|
| timezone | String | Supplier timezone |
| latitude | Double | Latitude of map center point in the partner region |
| longitude | Double | Longitude of map center point in the partner region |

## Usages:

Load active bookings

Load booking history

Load available services

# Response status values

Note: Do not rely on descriptions (the 'errorMessage' field in the response), use codes instead (the 'status' field in the response)! Description can be changed on the server at any time.

| Code | Description |
|---|---|
| OK | Request has been successfully completed |
| INVALID_REQUEST | Request information is not filled in properly |
| INVALID_CUSTOMER_TYPE | Customer type is not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated on the server |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated on the server |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated on the server |
| INVALID_ACCOUNT_INFO | Authenticated user details do not match booking details |
| INVALID_CUSTOMER_DATA | Client tries to use an object relating to a different customer type |
| DATA_NOT_FOUND | Unable to find data by the specified identifier |
| FILL_REQUIRED_DETAILS | Not all required account login details have been specified correctly |
| SMARTPHONE_VALIDATION_FAILED | Missing smartphone details or the smartphone is banned on the server |
| APPLICATION_OUTDATED | Used application is outdated |
| ACCOUNT_ON_HOLD | Specified account is on hold |
| PIN_ON_HOLD | Specified PIN is on hold |
| USER_ON_HOLD | Specified caller (retail or corporate) is on hold |
| FIELDS_VALIDATION_FAILED | Specified email, phone number or user name is invalid |
| REGISTRATION_FAILED | Account for retail users is not specified on the server |
| USER_ALREADY_EXISTS | User with the specified phone number already exists |
| UNABLE_TO_SEND_PASSWORD | Unable to send a password for some internal reasons |
| USER_ NOT_FOUND | Unable to find a user with the specified phone number |
| INVALID_PASSWORD | Specified password is not valid |
| ADDRESS_SEARCH_FAILED | Unable to find an address with the specified details |
| CALLER_UNDEFINED | Caller is undefined during address creation or deletion |
| GLOBAL_ADDRESS | Client tries to modify or delete a global address |
| ADDRESS_ALREADY_EXISTS | Address with the specified favourite name already exists |
| INVALID_ADDRESS_DATA | Address data is not filled in |
| INVALID_COORDINATES | Specified coordinates are not in the valid area |
| INVALID_ADDRESS_SOURCE | Specified address type is not supposed to be used in the address book |
| INVALID_ROUTE_INFO | Attempted to create a 'Wait and return', 'Destination unknown' or 'As directed' booking while it is not supported by the selected service |
| INCORRECT_REQUEST_IDENTIFIER | Specified request identifier has been already used in another booking |

| | |
|---|---|
| FULLY_BOOKED | Unable to make a prebook for the selected pickup circuit and date |
| TOO_EARLY_FOR_PREBOOK | Specified booking date is too early to make a prebook |
| TOO_LATE_FOR_PREBOOK | Specified booking date is too late to make a prebook |
| PREBOOK_NOT_AVAILABLE | Prebooks are not available |
| PICKUP_CIRCUIT_LOCKED | Unable to make a booking in the selected pickup circuit |
| INVALID_OPERATIONAL_AREA | Specified service is not available in the selected pickup circuit |
| REFERENCE_VALIDATION_FAILED | Reference validation failed |
| INSTRUCTION_VALIDATION_FAILED | Special instruction validation failed |
| PAYMENT_TYPE_NOT_SUPPORTED | Specified payment type is not supported for the selected account |
| CREDIT_CARD_PAYMENT_FAILED | Unable to perform a payment with the provided credit card |
| SERVICE_RESTRICTION_FAILED | Unavailable booking time is selected for the service or the price limit is exceeded |
| PRICE_CHANGED | Calculated price and the specified one are different |
| PRICE_NOT_CALCULATED | Unable to calculate a job price |
| INVALID_PROMO_CODE | Specified promo code is invalid |
| INVALID_LOYALTY_CARD | Specified loyalty card is invalid |
| INVALID_VOUCHER | Specified voucher is invalid |
| VOUCHER_NOT_ACTIVE | Specified voucher is not active |
| REDIRECT_REQUIRED | Client should be redirected to the provider site to confirm credit card details |
| BOOKING_COMPLETION_FAILED | Booking confirmation cannot be completed after redirect is finished |
| BOOKING_CONFIRMATION_FAILED | Booking cannot be confirmed for some internal reasons |
| BOOKING_ALREADY_PROCESSED | Booking has been already confirmed |
| BOOKING ALREADY_CANCELLED | Booking has been already cancelled |
| BOOKING_NOT_CANCELLABLE | Booking cannot be cancelled |
| CANCELLATION_CHARGE_CHANGED | Cancellation charge has changed and should be confirmed by the client |
| TIPS_ALREADY_SUBMITTED | Tips have been already submitted |
| INVALID_PUSH_MESSAGE_ID | Specified push notification message identifier is invalid |
| BOOKING_NOT_RATED | Survey for specified booking is not confirmed yet |
| ASAP_ONLY_SERVICE | Only ASAP bookings are available for the service |

# Protocol changes

This chapter contains a summary of changes made to the Sherbook API protocol that require the client code modification (mobile apps only).

The client should send a numerical value of API-VERSION in http headers in each request. Otherwise, the system uses the default value 1.0.

The client should support all features from the previous versions of the protocol.

| Sherbook API version | Description |
|---|---|
| 1.0 | Base version |
| 2.0 | PayPal functionality. In case the PayPal payment type is used and API version is older than 2.0, this payment type will not be available to mobile clients (excluding Web Portal clients) |
| 2.1 | Specical instruction with preference=true should be passed to the server if the client needs it.<br><br>For API verion older than 2.1, in case the client does not send this specical instruction in his booking confirmation request, it will be created automatically on the server side. |
| 2.2 | When the specified caller is on hold and API version >=2.2, the response status will be USER_ON_HOLD, in case of API version <2.2 the response status will be FILL_REQUIRED_DETAILS |
| 2.3 | In case of unsuccessful login or credencial check server will return http status code = 432 instead of 200. This is actual only for mobile devices, not portal. |
| 2.4 | For API version older than 2.4 the baggage capacity functionallity is supported. Number of small/large bags and number of passengers are required to confirm airport/train/cruise booking. |
| 2.5 | Airport and train settings from endpoint /settings/system was reworked. See 5. Loading system settings for details. For API version < 2.5 /settings/system works as before changes. |
| 2.6 | Added /payment/mirror/v2 endpoint, which returns json response body. Unlike the original /payment/mirror, a new endpoint returns the whole parameters map from external provider |
| 2.7 | Added required field *customerType* to the following endpoints*:*<br><br>/addresses/search/details<br>/addresses/search/position<br>/addresses/search/query<br><br>This field is required for statistics needs in autocomplete functionality |

| | |
|---|---|
| 2.8 | 1. Added support of asap only services. Services are validated when they are passed to the following endpoints:<br>• /calculate<br>  • /arrivalDelay<br>  • /prebookLimit<br>  • /price<br>  • /journeyTime<br>  • /quotation<br>  • /proposedAddresses<br>• /job/book<br><br>In case when asap only service is used for prebooks Sherbook returns ASAP_ONLY_SERVICE response.<br>2. Added unique user identification to support overusage protection in the following endpoints:<br><br>• /addresses<br>  • /search/query<br>  • /search/position<br>  • /search/details<br>  • /book/add<br>  • /book/edit<br>• /discount/validate<br>• /calculate<br>  • /prebookLimit<br>  • /quotation<br>  • /price<br>  • /journeyTime<br>  • /delay<br>  • /arrivalDelay<br>  • /proposedAddresses<br>• /offer/make<br>• /job/book<br><br>When the limit of requests is exceeded TOO_MANY_REQUESTS status is returned. |

# Security

There is built-in brute-force attack protection and automatic IP block in Sherbook.
In the security settings, allowed IP addresses can be specified in the ‘IP addresses white list’ setting (using a comma or space; masks are not supported) for which restrictions will not be applied.
In a cluster, the list of blocked IP addresses is shared between all nodes.

### Brute-force attack protection

Some endpoints are protected from brute-force attack. In case the client sends more than 15 requests (setting) per 1 minute (setting), it is blocked for 20 minutes (setting).
Each endpoint (except the 'login and check credentials' pair) has its own counter. Login and credentials URLs increments it's counter only on invalid attempts.
The client receives the HTTP response code 432 (setting) and USER_ON_HOLD sherbook status in case of brute-force detection (but the mobile client with Sherbook API < 2.3 receives the HTTP response code 200).

*Settings*

1. Enable or disable brute-force attack protection
The DB setting taxi.sherbookConfig.bruteForceProtectionEnabled, the default value is true. Available in Menu: Administration > System Settings (Sherbook): Security: Brute-force attack protection enabled.

2. Monitoring interval
The DB setting taxi.bruteForceProtectionConfig.bruteForceCheckInterval, the default value 60 seconds.

3. Block interval
The DB setting taxi.sherbookConfig.bruteForceBlockIntervalSec, the default value 1200 seconds (20 minutes). The value should be less that taxi.bruteForceProtectionConfig.bruteForceBlockInterval (24h by default). Available in Menu: Administration > System Settings (Sherbook): Security: Brute-force block interval (sec)

4. The maximum number of unsuccessful login (authorization, password recovery, etc) attempts
The DB setting taxi.sherbookConfig.maxLoginAttemptsNumber, the default value is 15. Available in Menu: Administration > System Settings (Sherbook): Security: Max number of unsuccessful login

5. HTTP response code for unsuccessful login requests
The DB setting taxi.sherbookConfig.unsuccessfulLoginHttpCode, the default value is 432.

6. The minimum mobile client Sherbook API version that supports the HTTP response code is set up in taxi.sherbookConfig.unsuccessfulLoginHttpCode (432)
The DB setting taxi.sherbookConfig.api.version.specialHttpCodeOnLoginVersion, the default value is 2.3.

7. IP addresses white list
The DB setting taxi.sherbookConfig.ipFilterWhiteList. Available in Menu: Administration > System Settings (Sherbook): Security: IP addresses white list.

### Automatic IP lock

When the client sends more than 200 (setting) anonymous requests per one minute (setting), the server locks for 1200 minutes (setting) the IP address for all requests and answers with the HTTP response code 432 (setting) and an empty body (excluding login, registration, credentials URLs for which the response body contains sherbook status USER_ON_HOLD).

A request is considered anonymous if the client has not authenticated an anonymous, retail or corporate user.
To disable this filtration specify the '0' value in the 'Max number of anonymous calls' setting.

*Settings*

1. Max number of anonymous calls

The DB setting 'taxi.sherbookConfig.ipMaxNumberOfAnonymousCalls'. Available in Menu: Administration > System Settings (Sherbook): Security: Max number of anonymous calls (0-off).

### Monitoring

There is a special tool used to monitor which IP addresses are locked or will be locked soon. Find the 'IpFilter' managed bean in JMX console. To see list of action-ip pairs for sherbook limits please use 'showLockedUsersInfoForSherbook' operation. List contains action, ip address, date when it was locked and time to unlock (or white list attribute).

In addition to that, there is a method for unlocking.

Security logs are now available in a security.log file.

### Mass fake booking protection

When someone does more than 5 anonymous unfinished bookings per 1 day from the same IP address, the server locks for 1 day (setting) the IP address for booking confirmation. Information about locked IPs can be reached using IpFilter managed bean, method 'showBookingIpsInfo()'. Described behavior  doesn't work if IP address is in White list (see above)

*Settings*

1. Monitoring and block interval for anonymous bookings
The DB setting taxi.bruteForceProtectionConfig.anonymousBookingInterval, the default value is 86400 seconds

2. Anonymous bookings max number

The DB setting taxi.sherbookConfig.anonymousBookingMaxNumber, the default value is 5

# 1. Retail user registration

The retail user registration process is described below:

1. The client sends data via URL /registration/new.
2. In case of a successful registration attempt the server sends a password to the specified phone number. The client gets the 'OK' status in the request result.
3. For the next login into Sherbook the user uses the password received in the SMS message.
4. To receive a new password the client sends data via URL /registration/password. The client who is already connected to the server with the old password will not be disconnected until manual logout.
5. The client has an ability to check whether the user already exists by sending the corresponding data via URL /registration/check.

## 1. Register a new user

### URL: /registration/new

### Request:

#### RegistrationRequest

| Parameter | Type | Description |
|---|---|---|
| phone | String | User phone number |
| name | String | User name |
| email | String | User email address |
| device | DeviceDto | User device info |
| language | LanguageDto | User language. If not specified, the user session locale will be used |

### Response:

Cf. BaseResponse

### Possible response codes:

| Value | Description |
|---|---|
| OK | User has been successfully registered |
| INVALID_REQUEST | Phone number is not specified |
| FIELDS_VALIDATION_FAILED | User information has been specified in the wrong format |
| SMARTPHONE_VALIDATION_FAILED | Smartphone is banned or required details are missing |
| APPLICATION_OUTDATED | Application is outdated |
| UNABLE_TO_SEND_PASSWORD | Server is unable to send a password in an SMS message for some technical reasons |
| USER_ALREADY_EXISTS | User with the specified phone number is already registered |

### Request example:

| Request | Response |
|---|---|
| {<br>"device": {<br>"imei": "352000000000000",<br>"deviceId": "00000000-0000-0000-0000-000000000000",<br>"osId": "iPhone OS",<br>"osVersion": "6.0",<br>"appVersion": "1.0",<br>"appId": "com.haulmont.taxi.app",<br>"screenWidth": 375,<br>"screenHeight": 647,<br>"fingerprint": null,<br>"type": "IPHONE"<br>},<br>"name": "Example",<br>"phone": "+70000000000",<br>"email": "example@example.com"<br>} | {<br>"status": "OK",<br>"errorMessage": null<br>} |
| | {<br>"status": "SMARTPHONE_VALIDATION_FAILED",<br>"errorMessage": "Device identifier required"<br>} |
| | {<br>"status": "USER_ALREADY_EXISTS",<br>"errorMessage": " User with phone number +70000000000 is already registered"<br>} |
| | {<br>"status": "UNABLE_TO_SEND_PASSWORD",<br>"errorMessage": null<br>} |

| | |
|---|---|
| USER_ON_HOLD | User with the specified phone number is banned<br><br>The number of unsuccessful login attempts has been exceeded. The account is temporary put on hold |
| REGISTRATION_FAILED | Registration has not been completed due to the internal server error |

## 2. Get a new password

Request a new password (recovery). The server will send the same password each time. Once a user has successfully logged in with the new password, the old password will expire and the new password will be used for the next recovery.

### URL: /registration/password

### Request:

#### PhoneRequest

| Parameter | Type | Description |
|---|---|---|
| **phone** | String | User phone number |
| **device** | DeviceDto | Smartphone information |

### Response:

Cf. BaseResponse

### Possible response codes:

| Value | Description |
|---|---|
| OK | Password has been successfully sent |
| INVALID_REQUEST | Phone number is not specified |
| FIELDS_VALIDATION_FAILED | User information has been specified in the wrong format |
| SMARTPHONE_VALIDATION_FAILED | Smartphone is banned on the server |
| APPLICATION_OUTDATED | Application is outdated |
| USER_NOT_FOUND | Unable to find a user with the specified phone number |
| USER_ON_HOLD | User with the specified phone number is banned |
| UNABLE_TO_SEND_PASSWORD | Server is unable to send a password in an SMS message for some technical reasons |

### Request example:

| Request | Response |
|---|---|
| { "device": { "imei": "352000000000000", "deviceId": "00000000-0000-0000-0000-000000000000", "osId": "iPhone OS", "osVersion": "6.0", "appVersion": "1.0", "appId": "com.haulmont.taxi.app", "screenWidth": 375, "screenHeight": 647, "fingerprint": null, "type": "IPHONE" }, "phone": "+70000000000", } | { "status": "OK", "errorMessage": null } |
| | { "status": " APPLICATION_OUTDATED ", "errorMessage": "Application is outdated. Please, update it." } |
| | { "status": " USER_NOT_FOUND", "errorMessage": "Registration info not found" } |

## 3. Check whether a user is already registered

### URL: /registration/check

### Request:

Cf. PhoneRequest.

### Response:

Cf. BaseResponse

### Possible response codes:

| Value | Description |
|---|---|
| OK | User is already registered |
| INVALID_REQUEST | Phone number is not specified |
| FIELDS_VALIDATION_FAILED | Phone number is specified in the wrong format |
| SMARTPHONE_VALIDATION_FAILED | Smartphone is banned on the server |
| APPLICATION_OUTDATED | Application is outdated |
| USER_NOT_FOUND | Unable to find a user with the specified phone number |
| USER_ON_HOLD | User is banned |

### Request example:

| Request | Response |
|---|---|
| {<br>"device": {<br>"imei": "352000000000000",<br>"deviceId": "00000000-0000-0000-0000-000000000000",<br>"osId": "iPhone OS",<br>"osVersion": "6.0",<br>"appVersion": "1.0",<br>"appId": "com.haulmont.taxi.app",<br>"screenWidth": 375,<br>"screenHeight": 647,<br>"fingerprint": null,<br>"type": "IPHONE"<br>},<br>"phone": "+70000000000",<br>} | {<br>"status": "OK",<br>"errorMessage": null<br>} |
| | {<br>"status": "SMARTPHONE_VALIDATION_FAILED",<br>"errorMessage": "Device type required"<br>} |
| | {<br>"status": " USER_NOT_FOUND",<br>"errorMessage": "Registration info not found"<br>} |

## 4. Phone number validation

### URL: /registration/phone/validate

### Request:

Cf. PhoneRequest.

### Response:

Cf. BaseResponse

### Possible response codes:

| Value | Description |
|---|---|
| OK | User is already registered |
| INVALID_REQUEST | Phone number is not specified |
| FIELDS_VALIDATION_FAILED | Phone number is specified in the wrong format |

### Request example:

| Request | Response |
|---|---|
| {<br>"phone" : "+79374933441"<br>} | {<br>"status": "OK",<br>"errorMessage": null<br>} |
| | {<br>"status": "FIELDS_VALIDATION_FAILED",<br>"errorMessage": "The phone number +791374933441 is invalid<br>} |

# 2. Login process


- 1. Login a corporate user
- 2. Login a retail user
- 3. Login by token
- 4. Login an anonymous user
- 5. Logout
- 6. Header-based authentication flow


To log in to Sherlock the client sends a request which corresponds to its login type. There are three available login types: corporate, retail and anonymous. The client could sequentially log in with different login types. For the all login methods Sherbook-Client-Base header is supported. Sherbook-Client-Base header specifies url where user will be redirected after 3DS or EPP (External Payment Page) payment.

## 1. Login a corporate user

For corporate users login is represented as a multistage process. For different accounts various login schema could be used. The only difference between the used login schemas is a set of required fields, e.g. {account number, PIN, caller name} or {account number, account password} or {account number, PIN}.

Field types:

- Account number (always required)
- PIN
- Caller name

For example, login process for corporate users could be executed by performing the following steps:

1. In CorporateLoginRequest the client transfers the *authentication* field to the server which has a composite value of two fields: *LoginFieldType.ACCOUNT* in the **key** field and the specified account number in the **value** field;
2. The server response includes a directive of the next field to be sent and its status, i.e. PIN and FILL_REQUIRED_DETAILS respectively;
3. The required field mentioned above (see item 2) should be added to the composite **authentication** field. In the example the PIN field must be specified;
4. The server checks if the given field is valid and asks to fill in the caller name;
5. The client sends {Account number + PIN + Caller name} in the **authentication** field to the server;
6. When all data is filled in correctly, the server transfers the 'OK' status code with the response to the client.
7. The client can remember the correct data and during the next login with the specified previously data perform login in a single step.

### URL: /login/corporate

### Request:

#### CorporateLoginRequest

| Parameter | Type | Description |
|---|---|---|
| **authentication** | List<AuthFieldValue> | Authentication fields |

### Request example:

| Request | Response |
|---|---|
| {<br>"device": {<br>……<br>…..<br>},<br>"authentication":<br>[<br>{<br>"key":<br>"ACCOUNT",<br>"value": "TEST"<br>},<br>{<br>"key": "PIN",<br>"value": "000"<br>},<br>{<br>"key": "CALLER",<br>"value": "John Smith"<br>}<br>],<br>"clientType": "MOBILE_WS"<br><br>} | {<br>"status": "OK",<br>"errorMessage": null<br>} |

| | | |
|---|---|---|
| **device** | DeviceDto | Smartphone information |
| **clientType** | ClientType | Client type |

### AuthFieldValue

| Parameter | Type | Description |
|---|---|---|
| **key** | LoginFieldType | Type of login field |
| **value** | String | Login field value |

### LoginFieldType

| Value | Description |
|---|---|
| ACCOUNT("ACCOUNT") | Account number |
| PIN("PIN") | PIN |
| CALLER("CALLER") | Caller name |

## Response:

## CorporateLoginResponse

| Parameter | Type | Description |
|---|---|---|
| loginFields | List<LoginField> | List of login fields required for corporate login type |
| mustBeInList | Boolean | Indicates if a profile value must be selected from the options list |

### LoginField

| Parameter | Type | Description |
|---|---|---|
| key | LoginFieldType | Type of login field |
| label | String | Login field label |
| currentValue | String | Value sent to the server |
| valid | boolean | Indicates if the specified value is valid |
| options | List<String> | Possible value options |
| contacts | List<ContactDto> | Populated only if key=CALLER.<br><br>Contains a list of contacts according to the 'Options' field |

## Possible response codes:

| Value | Description |
|---|---|
| OK | Login of a corporate user is successfully processed |


```
{
"status": "FILL_REQUIRED_DETAILS",
"errorMessage": null,
"loginFields": [
{
"key": "ACCOUNT",
"label": "Account",
"currentValue": "TEST",
"valid": true,
"options":null,
 "contacts": null
},
{
"key": "PIN",
"label": "Pin",
"currentValue": "000",
"valid": true,
"options":null,
 "contacts": null
},
{
"key": "CALLER",
"label": "Caller",
"currentValue": "John Smith",
"valid": false,
"options": ["John Connor", "Sara Connor"],
 "contacts": [
 {
 "name": "John Connor",
 "phone": null,
 "email": "john@somedomen.com",
 "individualId": "e6d4965b-7b62-e6af-3eaf-
6dd24a12476b",
 "customerType": "CORPORATE"
 },
 {
 "name": "Sara Connor",
 "phone": "888",
 "email": "sara@somedomen.com",
 "individualId": "3971eb3a-9bd9-0200-6343-
7edbf3e3cf3d",
 "customerType": "CORPORATE"
 }
 ]

}

] ,

"mustBeInList": false

}
```

| | |
|---|---|
| INVALID_REQUEST | Authentication fields are not specified or not filled in properly |
| SMARTPHONE_VALIDATION_FAILED | Smartphone is banned or required details are missing |
| APPLICATION_OUTDATED | Application is outdated |
| ACCOUNT_ON_HOLD | Specified account is on hold |
| PIN_ON_HOLD | Specified PIN is on hold |
| USER_ON_HOLD | Specified caller is on hold (this status is returned only when Sherbook API version >=2.2, otherwise FILL_REQUIRED_DETAILS).<br><br>The number of unsuccessful login attempts is exceeded. The account is temporary put on hold |
| FILL_REQUIRED_DETAILS | Not all required details for login as a corporate user have been specified correctly |

## 2. Login a retail user

To login via a phone number a user should be registered on the server (see Retail user registration). After registration the user can login as a retail user via his phone number as a login. To login via a phone number the client should send a request containing the phone number and the password fields to the server. A successful response contains the **token** field that the client should store and use in the next requests as HTTP header with the 'token' name. Such mechanism allows the server to restore a retail user session in such cases when it is lost or expired.

### URL: /login/retail

### Request:

#### RetailLoginRequest

| Parameter | Type | Description |
|---|---|---|
| **phone** | String | User phone number |
| **password** | String | User password |
| **device** | DeviceDto | Smartphone information |
| **clientType** | ClientType | Client type |

### Response:

### Request example:

| Request | Response |
|---|---|
| {<br>"device" : {<br>……..<br>},<br>"phone" : "+70000000000",<br>"password" : "587412"<br>,<br>"clientType": "MOBILE_WS"<br><br>} | {<br>"status": "OK",<br>"errorMessage": null,<br>"token" : "03f2bae5-af98-49f3-819b-fcd896db6512"<br>} |
| | {<br>"status" : "INVALID_PASSWORD",<br>"errorMessage" : "Password is invalid"<br>} |

### RetailLoginResponse

| Parameter | Type | Description |
|---|---|---|
| token | UUID | Token generated for a retail user session |

## Possible response codes:

| Value | Description |
|---|---|
| OK | Specified credentials are valid |
| INVALID_REQUEST | Phone number or password is missing |
| SMARTPHONE_VALIDATION_FAILED | Smartphone is banned or required details are missing |
| APPLICATION_OUTDATED | Application is outdated |
| FIELDS_VALIDATION_FAILED | Phone number or password is specified in the wrong format |
| INVALID_PASSWORD | Specified password is invalid |
| USER_NOT_FOUND | User with the specified phone number has not been found |
| USER_ON_HOLD | User is banned<br><br>The number of unsuccessful login attempts is exceeded. The account is temporary put on hold |

# 3. Login by token

Login by token functionality used by Quickbooker and Web Kiosk booking clients to log in the system. To log in by retail individual token, retail individual has to be registered in the web portal or mobile application. Individual without User Registration record in the system can't be logged by token.

## Request:

### Request headres

| Header | Description |
|---|---|
| **Kiosk-version** | A Sherlock release version that Kiosk is expecting |

### TokenLoginRequest

| Parameter | Type | Description |
|---|---|---|
| **token** | String | Token value |
| **clientType** | ClientType | Client type |

### ClientType

| Value | Description |
|---|---|

## Request example:

| Request | Response |
|---|---|
| {<br>"token" : "yAvMhwfTxCQkdHdV",<br>"clientType" : "QUICKBOOKER"<br>} | {<br>"status": "OK",<br>"errorMessage": null,<br>"customerType": "CORPORATE"<br>} |

| | |
|---|---|
| WEB_KIOSK | Web Kiosk clien application |
| QUICKBOOKER | Quickbooker client application |

### Response:

#### TokenLoginResponse

| Parameter | Type | Description |
|---|---|---|
| customerType | CustomerType | Customer type of the profile associated with input token |
| status | ResponseStatus | Response status returned by the server |
| errorMessage | String | Description of the occurred error. Filled in if the status is not 'OK' |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Specified token is valid |
| USER_NOT_FOUND | Profile associated with token is not found |
| OUTDATED_APPLICATION | Used by Kiosk. The code means that kiosk application needs to be updated. |

## 4. Login an anonymous user

Anonymous login functionality has been added to allow the client to log in using only smartphone device information without registration. Unlike a registered user, an anonymous user is not allowed to load previously used addresses, credit cards, finished or cancelled bookings.

### URL: /login/anonymous

### Request:

#### BaseLoginRequest

| Parameter | Type | Description |
|---|---|---|
| **device** | DeviceDto | Smartphone information |
| clientType | String | Client type. Possible values: "WEB_KIOSK", "QUICKBOOKER" and web portal if no type specified |

### Response:

### Request example:

| Request | Response |
|---|---|
| {<br>"device" : {<br>……..<br><br>}<br>} | {<br>"status": "OK",<br>"errorMessage": null<br>} |

Cf. BaseResponse

### Possible response codes:

| 1. Value | 1. Description |
|---|---|
| OK | Specified credentials are valid |
| SMARTPHONE_VALIDATION_FAILED | Smartphone is banned or required details are missing |
| APPLICATION_OUTDATED | Application is outdated |
| USER_ON_HOLD | User is banned |

## 5. Logout

Each of the login types specified above supports the logout operation which is used to delete session information from the server.
The retail user token should be specified in http headers for retail logout (Mobile Client, not Web Portal). It becomes outdated after retail user logout is processed.

### URL: /login/logout

### Request:

Cf. CustomerDataRequest.

### Response:

Cf. BaseResponse.

### Possible response codes:

| Value | Description |
|---|---|
| OK | Specified credentials are valid |
| INVALID_REQUEST | List of profile types is missing |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated or the token is not specified in the http session for Mobile Client |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

### Request example:

| Request | Response |
|---|---|
| {<br>"customerTypes" : [<br>"CORPORATE",<br>"RETAIL"<br>]<br>} | {<br>"status": "OK",<br>"errorMessage": null,<br>} |

# 6. Header-based authentication flow

Sherbook uses authentication based on http-session by default. It is possible to get rid of http-session and use header for user authentication. Sherbook supports both mechanism at the same time.

Header-based mechanism uses access/refresh token pairs for user identification. Access/refresh tokens can be found in response in the following endpoints:

/login/anonymous
/login/token
/login/corporate
/login/retail

Response field **sessionToken** - access token

Response field **sessionRefreshToken** - refresh token

Access token should be used in **SESSION-TOKEN** header. If request has no such header, then http-session will be used as default mechanism. If header is invalid then user will get 401 Unauthorized response status

Access token has limited time to live. Once it's expired, refresh token should be used to acquire a new pair of access/refresh tokens.

If SESSION-TOKEN header used, api will automatically guess customer type according to provided header. Thus, customer type could be omitted in every request as long as SESSION-TOKEN header presents.

## POST /login/refresh

Acquire a new pair of access/refresh token using refresh token.

**Request**

```
{ "refreshToken": "string" }
```

**Response**

```
{
  "errorMessage": "string",
  "newAccessToken": "string",
  "newRefreshToken": "string",
  "status": "OK"
}
```

Possible error statuses:

TOKEN_NOT_VALID
TOKEN_NOT_FOUND

## POST /login/revoke

Revoke access token (user logout)

**Request**

```
{
  "accessToken": "string"
}
```

**Response**

```
{
  "errorMessage": "string",
  "status": "OK"
}
```

Possible error statuses:

TOKEN_NOT_VALID
TOKEN_NOT_FOUND

## POST /login/find

Sherbook sends exposedToken request parameter in callback-urls for application which could not store token information. This token can be used to find previously granted access token without re-login.

**Request**

```
{
  "exposedToken": "string"
}
```

**Response**

```
{
  "accessToken": "string",
  "authInfo": {
    "type": "CORPORATE"
  },
  "errorMessage": "string",
  "refreshToken": "string",
  "status": "OK"
}
```

Possible error statuses:

TOKEN_NOT_VALID
TOKEN_NOT_FOUND

# 3. Credentials checking

### CorporateLoginResponse

| Parameter | Type | Description |
|---|---|---|
| loginFields | List<LoginField> | List of login fields required for corporate login type. |

Credentials checking functionality allows the client to check whether the specified user credentials are correct without authentication. Methods specified below (request and response objects, possible response code values) are similar to corresponding login methods.

## 1. Check corporate credentials

### URL: /credentials/corporate

### Request:

Cf. CorporateLoginRequest.

### Response:

### CorporateCredentialsResponse

| Parameter | Type | Description |
|---|---|---|
| loginFields | List<LoginField> | List of login fields required for the corporate login type |
| accountName | String | Account name |
| caller | ContactDto | Caller details (if applicable) |
| mustBeInList | Boolean | Indicates if a profile value must be selected from the options list |

### Possible response codes:

Cf. corporate login possible response codes.

## 2. Check retail credentials

### URL: /credentials/retail

### Request:

Cf. RetailLoginRequest.

### Response:

Cf. BaseResponse.

### Possible response codes:

Cf. retail login possible response codes.

## 3. Check whether the application version is the latest or outdated

### URL: /credentials/application

### Request example:

## Request:

Cf. BaseLoginRequest.

## Response:

### ApplicationVersionResponse

| Parameter | Type | Description |
|---|---|---|
| applicationLink | String | Link to download the lastest application version |

## Possible response codes:

| Value | Description |
|---|---|
| OK | System settings have been successfully loaded |
| SMARTPHONE_VALIDATION_FAILED | Smartphone required details are missing |
| APPLICATION_OUTDATED | Application is outdated |

| Request | Response |
|---|---|
| {<br>"device": {<br>"imei": "352000000000000",<br>"deviceId": "00000000-0000-0000-0000-000000000000",<br>"osId": "iPhone OS",<br>"osVersion": "6.0",<br>"appVersion": "1.0",<br>"appId": "com.haulmont.taxi.app",<br>"screenWidth": 375,<br>"screenHeight": 647,<br>"fingerprint": null,<br>"type": "IPHONE"<br>}<br>} | {<br>"status": "OK",<br>"errorMessage": null,<br>"applicationLink":null<br>}<br>{<br>"status": "APPLICATION_OUTDATED",<br>"errorMessage": "Application is outdated. Please, update it",<br>"applicationLink": "http://localhost:8080"<br>} |

# 4. Profile

Operations with profiles described below are available to authorised RETAIL users only. Please note that changing of a password and a phone number affects all previously connected sessions – all of them, except the current one, will be disconnected.

## 1. Load a name and an email

### URL: /profile/details/load

### Request:

Empty request.

### Response:

#### LoadProfileResponse

| Parameter | Type | Description |
|---|---|---|
| profile | ProfileDto | User profile details |

#### ProfileDto

| Parameter | Type | Description |
|---|---|---|
| name | String | User name |
| email | String | User email |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Name and email have been successfully loaded |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |

### Request example:

| Request | Response |
|---|---|
| | {<br>"status": "OK",<br>"errorMessage": null,<br>"profile": {<br>"name": "Example",<br>"email": "example@example.com"<br>}<br>} |

## 2. Change a name and an email

### URL: /profile/details/save

### Request:

#### ChangeProfileRequest

| Parameter | Type | Description |
|---|---|---|
| **profile** | ProfileDto | Profile information |

### Response:

Cf. BaseResponse.

### Possible response codes:

| Value | Description |
|---|---|
| OK | Name and email have been successfully saved |
| INVALID_REQUEST | Profile information is missing |
| FIELDS_VALIDATION_FAILED | Name or email is specified in the wrong format |
| INVALID_RETAIL_REQUEST | Retails user is not properly authenticated |

### Request example:

| Request | Response |
|---|---|
| {<br>"profile": {<br>"name": "Example",<br>"email": "example@example.com"<br>}<br>} | {<br>"status": "OK",<br>"errorMessage": null<br>} |

## 3. Change a password

For retail individuals, changes individual's password. All previously connected sessions, except the current one, is disconnected.

For corporate individuals, the endpoint
- validates the provided current password against the actual current password of the individual;
- validates the new password against complexity rules, existing in the system, if any;
- changes individual's password if all the checks are completed successfully.

### URL: /profile/password

### Request:

#### ChangePasswordRequest

| Parameter | Type | Description |
|---|---|---|
| **customerType** | CustomerType | Customer type |
| **currentPassword** | String | Current password |
| **password** | String | New password |

As with other requests, that include customer type, **customerType** may be omitted in case of using token-based request authentication. For backward compatibility, if the token-based authentication is not used, **customerType** is also optional: in this case the request is executed as a retail user request.

The **currentPassword** field is ignored for retail user requests.

### Response:

Cf. BaseResponse.

### Possible response codes:

| Value | Description |
|---|---|
| OK | Password changed successfully |
| INVALID_REQUEST | **password** is missing or (for **customerType** = CORPORATE) **currentPassword** is missing |
| FIELDS_VALIDATION_FAILED | **password** is too simple: too short for retail requests, violates password complexity rules for **customerType** = CORPORATE |
| INVALID_PASSWORD | (**customerType** = CORPORATE only) **currentPassword** does not equal to the actual current password of the individual |
| OPERATION_UNAVAILABLE | (**customerType** = CORPORATE only) individual password is not supported by the account's authentication policy |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Attempt to change password of an anonymous user |

### Request example:

| Request | Response |
|---|---|
| {<br>"password":<br>"123456"<br>} | {<br>"status": "OK",<br>"errorMessage":<br>null<br>} |

## 4. Change a phone number

All previously connected sessions, except current one, will be disconnected.

### URL: /profile/phone

### Request:

#### ChangePhoneRequest

| Parameter | Type | Description |
|---|---|---|
| phone | String | New phone number |

### Response:

Cf. BaseResponse.

### Possible response codes:

| Value | Description |
|---|---|
| OK | Phone number has been successfully changed. A new password has been sent to the specified phone number in an SMS message |
| INVALID_REQUEST | Phone number is not specified |

| | |
|---|---|
| FIELDS_VALIDATION_FAILED | Name or email is specified in the wrong format |
| UNABLE_TO_SEND_PASSWORD | Server is unable to send a password in an SMS message for some technical reasons |
| INVALID_RETAIL_REQUEST | Retails user is not properly authenticated |

### Request example:

| Request | Response |
|---|---|
| {<br>"password":<br>"123456"<br>} | {<br>"status": "OK",<br>"errorMessage":<br>null<br>} |

## 5. Remove profile

Current sherbook sessions will be disconnected. The user registration and user profile will be removed.

### URL: /profile/remove

### Request:

Empty request.

### Response:

Cf. BaseResponse.

### Possible response codes:

| Value | Description |
|---|---|
| OK | The user profile has been successfully removed |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| DATA_NOT_FOUND | Retail user has already been removed |

# 5. Loading system settings

Settings stored on the server can be loaded to the client using the methods specified below. There are two types of settings:

- **Booking settings** – settings relating to a customer type. For each customer type sent in a request the server returns corresponding booking settings.
- **System settings** – actual for all customer types as they do not depend on them.
- **Initial settings** - settings available before login.

## 1. Load booking settings

### URL: /settings/booking

### Request:

Cf. CustomerDataRequest.

### Response:

#### BookingSettingsResponse

| Parameter | Type | Description |
|---|---|---|
| bookingSettings | List<BookingSettingsDto> | List of booking settings corresponding to different profile types |

#### BookingSettingsDto

| Parameter | Type | Description |
|---|---|---|
| customerType | CustomerType | Customer type |
| arrivalPickupDelaySettings | ArrivalPickupDelaySettings | Settings for bookings with an airport pickup |
| creditCardSettings | CreditCardSettings | Credit card payment settings |
| jobOfferSettings | JobOfferSettings | Job offer workflow settings |
| discountSettings | DiscountSettings | Discount settings |
| showPrice | boolean | Indicates if a price should be shown to the client |
| showPassengers | boolean | Indicates if passengers should be shown to the client |
| contactSettings | ProfileSettings | Contact settings. Can be null (anonymous user, etc) |
| passengerSettings | ProfileSettings | Passenger settings. Can be null (anonymous user, etc) |
| showPriceForDestinationUnknown | boolean | Indicates if a price for 'Destination unknown' jobs should be shown on client |
| shipTerminalPrebookAvailable | boolean | Indicates if ship terminal is available as a prebook pickup address for account |

| | | |
|---|---|---|
| etaDurationMinutes | Integer | Maximum time gap between ETA and job date, min |
| accountOrPinSupervisor | boolean | True if user is account or pin supervisor and false otherwise |
| bookingsRequireApproval | boolean | True if user is account/pin supervisor and there are individuals<br><br>with marked 'bookingsRequireApproval' option among account/pin individuals |
| partners | List<PartnerDto> | List of partners which avaible for current user |
| restrictions | Restrictions | Booking stop restrtrictions |
| defaultPickup | AddressDto | Default pickup |
| defaultDropoff | AddressDto | Default dropoff |
| canEnterReferralCode | boolean | Indicates whether 'Referral Code' input field can be shown to client |
| canChangeCorporatePassword | boolean | Indicates whether current user can change corporate password. True for corporate users, if the account's authentication option is "Individual password". False for non-corporate users and for corporate users with the account's authentication option other than "Individual password". |

### ArrivalPickupDelaySettings

| Parameter | Type | Description |
|---|---|---|
| airportPickupTimeDelay | Integer | Preferred value for the client airport pickup time delay |
| trainPickupTimeDelay | Integer | Preferred value for the client train pickup time delay |

### CreditCardSettings

| Parameter | Type | Description |
|---|---|---|
| cvcRequiredForNewProfileCreditCard | boolean | Indicates if a CVC number is required for saving a new card in the user profile |
| creditCardType | CreditCardType | Type of a credit card operation scheme |
| cseAlgorithmCode | String | Code of the CSE algorithm used. Filled in only if the credit card type is CSE |
| csePublicKey | String | Public key used in the CSE algorithm. Filled in only if the credit card type is CSE |
| `internalPaymentPageUsed` | boolean | Indicates if a CBA credit card flow is being used |
| `internalPaymentPageURL` | String | Internal payment page address. Used in CBA flow for credit card details input |

### CreditCardType

| Value | Description |
|---|---|
| PLAIN("PLAIN") | Credit cards are sent to the server with PLAIN details |
| CSE("CSE") | Credit cards are sent to the server with ENCRYPTED details |
| EPP_NEW("EPP_NEW") | Only NEW credit cards are NOT sent to the server at all |
| EPP_ALL("EPP_ALL") | ALL credit cards are NOT sent to the server at all |
| CUSTOM("CUSTOM") | Indicates customised credit card flow, such as CBA |

### JobOfferSettings

| Parameter | Type | Description |
|---|---|---|
| jobOfferRefreshPeriod | Integer | Offer state refresh period |
| jobOfferDecisionTimeout | Integer | Timeout for accepting the offered driver |
| jobOfferTimeout | Integer | Offer lifetime |

### DiscountSettings

| Parameter | Type | Description |
|---|---|---|
| loyaltyCardsUsed | boolean | Indicates if loyalty cards are used |
| promoCodesUsed | boolean | Indicates if promo codes are used |
| vouchersUsed | boolean | Indicates if vouchers are used |
| autoPopulateVoucher | boolean | Indicates if a voucher should be populated automatically |

### ProfileSettings

| Parameter | Type | Description |
|---|---|---|
| tag | String | Profile tag shown to the client |
| used | boolean | Indicates if a profile is used in the client account |
| mustBeInList | boolean | Indicates if a profile value must be selected from the options list |
| showOptions | boolean | Shows available profile options |
| populateByDefault | boolean | Indicates if passenger should be populated by default |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Booking settings have been successfully loaded |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

## Request example:

| Request | Response |
|---|---|
| {<br>"customerTypes" : [<br>"CORPORATE",<br>"CORPORATE"<br>]<br>} | {<br>"status": "OK",<br>"errorMessage": null,<br>"bookingSettings": [<br>{<br>"customerType": "CORPORATE",<br>"arrivalPickupDelaySettings": {<br>"airportPickupTimeDelay": 0,<br>"trainPickupTimeDelay": 0<br>},<br>"creditCardSettings": {<br>"cvcRequiredForNewProfileCreditCard": true,<br>"creditCardType": "CSE", |

"cseAlgorithmCode": "adyen",
"csePublicKey": "10001|B3BB0E95186A62D58D61456E31D67B3F9C84"
},
"jobOfferSettings": {
"jobOfferRefreshPeriod": 3,
"jobOfferDecisionTimeout": 120,
"jobOfferTimeout": 81
},
"discountSettings": {
"loyaltyCardsUsed": true,
"promoCodesUsed": true,
"vouchersUsed": true,
"autoPopulateVoucher": true
},
"showPrice": true,
"showPassengers": true,
"contactSettings": {
"tag": null,
"used": false,
"mustBeInList": false,
"showOptions": true,
"populateByDefault": true
},
"passengerSettings": {
"tag": null,
"used": true,
"mustBeInList": false,
"showOptions": true,
"populateByDefault": true
},
"callerSettings": {
"tag": "Caller",
"used": true,
"mustBeInList": false,
"showOptions": true,
"populateByDefault": false
},
"partners": [
{
"caption": "CarASAP Partner",
"city": "Brussels",
"timezone": "Europe/Brussels",
"latitude": 50.87531114220077,
"longitude": 4.4110107421875
}
],
"restrictions": {
"pickupRestricted": true,
"dropRestricted": true,
"intermediateRestricted": true
},
"accountOrPinSupervisor": false,
"bookingsRequireApproval": false,
"showPriceForDestinationUnknown": true,
"defaultPickup": null,
"defaultDropoff": null,
"canEnterReferralCode" : true,
"etaDurationMinutes": 365,
"shipTerminalPrebookAvailable": false
}
]
}

## 2. Load system settings

## URL: /settings/system

## Request:

Empty request

## Response:

### SystemSettingsResponse

| Parameter | Type | Description |
|---|---|---|
| systemSettings | SystemSettingsDto | Set of different system settings |

### SystemSettingsDto

| Parameter | Type | Description |
|---|---|---|
| addressLoadingLimit | Integer | Limit for address loading |
| addressPositionSearchDistance | Integer | Distance radius in meters for searching addresses by position |
| jobDetailsRefreshPeriod | Integer | Job version refresh period |
| driverPositionRefreshPeriod | Integer | Driver details refresh period |
| maxPrebookingDays | Integer | Maximum days for prebook creation |
| maxAsDirectedMinutes | Integer | Maximum available minutes for 'As directed' bookings |
| timeZoneID | String | Time zone which should be used by the client for non-Partner services. |
| pricingPolicyURL | String | HTTP link to the pricing policy page |
| airportStopSettings | AirportStopSettings | Stop details settings for an airport pickup |
| trainStopSettings | TrainStopSettings | Stop details settings for a train station pickup |
| termsAndConditionUrl | String | Link to the Terms and Condition page on the Customer site |
| privacyPolicyUrl | String | Link to the Privacy Policy page on the Customer site |
| findArrivalInfo | boolean | If enabled, there is the 'Check' button on Web Portal to be used for verification of flight arrival information. |
| showDriverDetails | boolean | If enabled, a driver details (a driver name and a photo) will be shown to users |
| chatEnabled | boolean | If enabled, driver to client chat is available |
| referralProgramEnabled | boolean | If checked, Referral program is enabled in system.<br><br>Customer can specify Referral Code during registration or apply it after login.<br><br>Customer can also load/generate own code and share it with anyone.<br><br>For more information about referral program please refer to 45. Referral program |
| referralProgramLink | String | Additional link for showing 'Share referral code' web page |
| showReferralCodeExpirationDate | boolean | Indicates whether referral code expiration date must be shown to client |
| showReferralCodeUsageCount | boolean | Indicates whether referral code usage count must be shown to client |
| showPassengerLocationToDriver | boolean | Indicate whether 'Show client location to the driver' is enabled |
| passengerPositionRefreshIntervalSec | Integer | Recommended period of sending position from client to Sherbook, seconds |

### AirportStopSettings

| Parameter | Type | Description |
|---|---|---|
| pickupTimeDelayRequired | boolean | Old setting (API-VERSION < 2.5). Indicates if a pickup time delay should be specified |
| flightNumberRequired | boolean | Old setting (API-VERSION < 2.5). Indicates if a flight number should be specified |
| arrivalFromRequired | boolean | Old setting (API-VERSION < 2.5). Indicates if the 'Arrival from' field should be specified |
| terminalRequired | boolean | Old setting (API-VERSION < 2.5). Indicates if a terminal should be specified |
| airlineRequired | boolean | Old setting (API-VERSION < 2.5). Indicates if an airline should be specified |
| airportsUsed | boolean | Indicates if at least one airport exists at system |
| pickupTimeDelayShowForPrebook | boolean | Indicates if PTD field should be shown in the UI for prebooks |
| pickupTimeDelayRequireForPrebook | boolean | Indicates if PTD field should be marked as required in the UI for prebooks |
| flightNumberShowForPrebook | boolean | Indicates if 'Flight Number' field should be shown in the UI for prebooks |
| flightNumberRequireForPrebook | boolean | Indicates if 'Flight Number' field should be marked as required in the UI for prebooks |
| arrivalFromShowForPrebook | boolean | Indicates if 'Arrival from' field should be shown in the UI for prebooks |
| arrivalFromRequireForPrebook | boolean | Indicates if 'Arrival from' field should be marked as required in the UI for prebooks |
| meetingPointShowForASAP | boolean | Indicates if 'Meeting point' field should be shown in the UI for asap bookings |
| meetingPointRequireForASAP | boolean | Indicates if 'Meeting point' field should be marked as required in the UI for asap bookings |
| meetingPointShowForPrebook | boolean | Indicates if 'Meeting point' field should be shown in the UI for prebooks |
| meetingPointRequireForPrebook | boolean | Indicates if 'Meeting point' field should be marked as required in the UI for prebooks |
| airlineShowForPrebook | boolean | Indicates if 'Airline' field should be shown in the UI for prebooks |
| airlineRequireForPrebook | boolean | Indicates if 'Airline' field should be marked as required in the UI for prebooks |

### TrainStopSettings

| Parameter | Type | Description |
|---|---|---|
| trainNumberRequired | boolean | Old setting (API-VERSION < 2.5). Indicates if a train number should be specified |
| arrivalFromRequired | boolean | Old setting (API-VERSION < 2.5). Indicates if the 'Arrival from' field should be specified |
| trainStationsUsed | boolean | Indicates if at least one train station exists at system |
| trainNumberShowForPrebook | boolean | Indicates if 'Train number' field should be shown in the UI for prebooks |
| trainNumberRequireForPrebook | boolean | Indicates if 'Train number' field should be marked as required in the UI for prebooks |
| arrivalFromShowForPrebook | boolean | Indicates if 'Arrival from' field should be shown in the UI for prebooks |
| arrivalFromRequireForPrebook | boolean | Indicates if 'Arrival from' field should be marked as required in the UI for prebooks |
| meetingPointShowForASAP | boolean | Indicates if 'Meeting point' field should be shown in the UI for asap bookings |
| meetingPointRequireForASAP | boolean | Indicates if 'Meeting point' field should be marked as required in the UI for asap bookings |
| meetingPointShowForPrebook | boolean | Indicates if 'Meeting point' field should be shown in the UI for prebooks |
| meetingPointRequireForPrebook | boolean | Indicates if 'Meeting point' field should be marked as required in the UI for prebooks |

### ShipTerminalStopSettings

| Parameter | Type | Description |
|---|---|---|

| | | |
|---|---|---|
| shipTerminalsUsed | boolean | Indicates if at least one ship terminal exists at system |

### Possible response codes:

| Value | Description |
|---|---|
| OK | System settings have been successfully loaded |

### Request example:

| Request | Response |
|---|---|
| | {<br>"status": "OK",<br>"errorMessage": null,<br>"systemSettings": {<br>"airportStopSettings": {<br>"airportsUsed": true,<br>"pickupTimeDelayRequired": true,<br>"flightNumberRequired": true,<br>"arrivalFromRequired": true,<br>"terminalRequired": true,<br>"airlineRequired": true<br>},<br>"trainStopSettings": {<br>"trainStationsUsed": true,<br>"arrivalFromRequired": true,<br>"trainNumberRequired": true<br>},<br>"shipTerminalStopSettings": {<br>"shipTerminalsUsed": true<br>},<br>"addressLoadingLimit": 100,<br>"addressPositionSearchDistance": 100,<br>"jobDetailsRefreshPeriod": 3,<br>"driverPositionRefreshPeriod": 5,<br>"maxPrebookingDays": 365,<br>"maxAsDirectedMinutes": 1320,<br>"timeZoneID": "Europe/London",<br>"chatEnabled": true,<br>"pricingPolicyURL": "",<br>"termsAndConditionUrl": "http://company.com/terms-and-conditions/",<br>"privacyPolicyUrl": "http://company.com/privacy/",<br>"showDriverDetails": true,<br>"findArrivalInfo": false,<br>"showDriverDetails" : true,<br>"chatEnabled" : false,<br>"referralProgramEnabled" : false,<br>"referralProgramLink" : null,<br>"showReferralCodeExpirationDate":false,<br>"showReferralCodeUsageCount" : false<br>} |

## 3. Initial settings

### URL: /settings/initial

### Request:

Empty request

## Response:

### InitialSettingsResponse

| Parameter | Type | Description |
|---|---|---|
| initialSettings | InitialSettingsDto | Set of different system settings |

### InitialSettingsDto

| Parameter | Type | Description |
|---|---|---|
| passwordLength | Integer | Minimal length of the customer password |
| passwordRecoveryCallbackEnabled | Boolean | If checked, password recovery callback will be made when user will request it |
| customerServicePhone | String | Phone number of the Customer Service |
| pricingPolicyURL | String | Pricing policy url |
| termsAndConditionUrl | String | Terms and condition url |
| privacyPolicyUrl | String | Privacy policy url |
| requirePolicyAgreement | Boolean | If checked, the Terms and Conditions and the Privacy Policy agreements should be accepted by customer |
| jobDetailsRefreshPeriod | Integer | Job version refresh period |
| driverPositionRefreshPeriod | Integer | Driver details refresh period |
| timeZoneID | String | Time zone which should be used by the client for non-Partner services. |
| iPhoneAppUrl | String | Link to download iPhone application |
| androidAppUrl | String | Link to download Android application |

## Possible response codes:

| Value | Description |
|---|---|
| OK | Initial settings have been successfully loaded |

# 6. Searching addresses

There are two methods to search addresses:

1. By the specified geo position (latitude and longitude).
2. By the specified string with an address (query);

For each method the server returns the AddressSearchResponse object which contains a list of found addresses. To limit the number of addresses returned from the server the client can use the value received in the **addressLoadingLimit** field of SystemSettingsDto.

Search by query can return addresses with PLACE_AUTOCOMPLETE source. Such addresses found viaGoogle Place Autocomplete, thus has no coordinates information. Coordinates information can be fetched using details endpoint. Details endpoint should be used once, after user selected particular address from the list.

## 1. Search addresses by position

In case of an airport or a terminal found in the requested region (at that, the airport does not allow non-airport addresses), then only this airport or terminal will be returned in the response.

### URL: /addresses/search/position

### Request:

#### AddressSearchPositionRequest

| Parameter | Type | Description |
|---|---|---|
| **latitude** | Double | Latitude |
| **longitude** | Double | Longitude |
| distance | Double | Search distance, in meters |
| limit | Integer | Limit for addresses to load |
| `customerType` | CustomerType | Customer type |

### Response:

#### CoordinatesAddressSearchResponse

| Parameter | Type | Description |
|---|---|---|
| addresses | List<AddressDto\|RelatedAddressDto> | A list of found addresses. RelatedAddressDto to be returned for airport terminals and berths. |

#### Possible response codes:

| Value | Description |
|---|---|
| OK | Addresses have been successfully loaded |
| INVALID_REQUEST | Position coordinates are not specified |

### Request example:

| Request | Response |
|---|---|
| { "latitude": 48.856713, "longitude": 2.327433, "limit": 2 } | { "status": "OK", "errorMessage": null, "addresses": [ { "addressId": null, "caption": "7 Rue Montalembert, 75007 Paris", "latitude": 48.8565252, "longitude": 2.3273621, "source": "GG", "data": "QURSe0Z…bWJlcnQsIDc1MDA3IFBhcmlzfX0=" }, { "addressId": null, "caption": "L'Atelier de Joël Robuchon, 5 Rue Montalembert, Paris", "latitude": 48.856615, "longitude": 2.327503, "source": "GP", "data": "UkVGe0Nu…h3cGQ5ekgxMUZyMUUtSjB0RVl9", } ] } |

## 2. Search addresses by string

### URL: /addresses/search/query

### Request:

### Request example:

| Request | Response |
|---|---|

### AddressSearchRequest

| Parameter | Type | Description |
|---|---|---|
| **searchString** | String | Address to search |
| limit | Integer | Limit for address loading |
| latitude | Double | Latitude |
| longitude | Double | Longitude |
| customerType | CustomerType | Customer Type |

### Response:

#### AddressSearchResponse

| Parameter | Type | Description |
|---|---|---|
| addresses | List<AddressDto\|RelatedAddressDto> | A list of found addresses. RelatedAddressDto to be returned for airport terminals and berths |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Addresses have been successfully loaded |
| INVALID_REQUEST | Search string cannot be blank |

### Request example:

| Request | Response |
|---|---|
| { "searchString": "Rue de Montalembert", "limit": 2, "latitude": null, "longitude": null } | { "status": "OK", "errorMessage": null, "addresses": [ { "addressId": null, "caption": "Rue de Montalembert, 85200 Pissotte, Pays de la Loire", "latitude": 46.493162, "longitude": -0.808697, "source": "GG", "data": "QUR…2lyZX19" }, { "addressId": null, "caption": "Rue de Montalembert, 35410 Châteaugiron, Bretagne", "latitude": 48.0321747, "longitude": -1.4983218, "source": "GG", "data": "QURSe0Z…19" } ] } |
| { "searchString": "Rue de Montalembert", "limit": 2, "latitude": 46.490, "longitude": -0.808777 } | |

## 3. Check position for airport address replacement

### URL: /addresses/checkReplacement

### Request:

#### CoordinatesRequest

| Parameter | Type | Description |
|---|---|---|
| **latitude** | Double | Latitude |
| **longitude** | Double | Longitude |

### Response:

#### AirportReplacementResponse

| Parameter | Type | Description |
|---|---|---|
| address | AddressDto | Found address |
| message | String | Address replacement description |
| allowNonAirportAddress | Boolean | The same as airport's property "Allow non-airport addresses" |

| | |
|---|---|
| { "latitude": 45.931073, "longitude" : 6.106563 } | { "address": { "addressId": "fdf5f366-269e-8368-a0f7-074ed632fef1", "caption": "Annecy Haute Savoie Mont Blanc, Terminal principal", "latitude": 45.931073, "longitude": 6.106563, "source": "AIRPORT_TERMINAL", "data": "TUNOe3RheGkkQWlycG9ydFRlcm1pbmFsfUlEe2ZkZjVmMzYmVmMX0=", "fields": null, "restrictions": null }, "message": "The address you've chosen is inside the airport terminal. Do you want to enter flight details?", "allowNonAirportAddress": true } |
| { "latitude": 45.931073, "longitude" : 6.106563 } | { "address": { "addressId": "fdf5f366-269e-8368-a0f7-074ed632fef1", "caption": "Annecy Haute Savoie Mont Blanc, Terminal principal", "latitude": 45.931073, "longitude": 6.106563, "source": "AIRPORT_TERMINAL", "data": "TUNOe3RheGkkQWlycG9ydFRlcm1pbmFsfUlEe2ZkZjVmMzYmVmMX0=", "fields": null, "restrictions": null }, "message": "The address you've chosen was changed to airport terminal address", "allowNonAirportAddress": false } |
| { "latitude": 44.931073, "longitude" : 6.106563 } | { "address": null, "message": null, "allowNonAirportAddress": null } |

# 4. Proposed addresses search

## URL: /calculate/proposedAddresses

Please note there are fields populated for proposed addresses. They should be passed in job confirmation endpoint if address type is PROPOSED_ADDRESS.

## Request:

### ProposedAddressesRequest

| Parameter | Type | Description |
|---|---|---|
| **customerType** | CustomerType | Customer type |
| **operationType** | BookingOperationType | Booking operation type |
| **requestId** | UUID | Booking request identifier |
| usedJobId | UUID | Used booking identifier |
| **service** | BookingServiceDto | Selected service |
| **paymentType** | BookingPaymentTypeDto | Selected payment type |
| **asap** | boolean | Indicates if the booking is ASAP |
| date | Date | Booking date |
| **stops** | List<StopDto> | List of booking stops |
| **routeInfo** | RouteInfo | Route info |
| instructions | List<SpecialInstructionDto> | Specified special instructions |
| loyaltyCard | LoyaltyCardDto | Selected loyalty card |
| voucher | VoucherDto | Selected electronic voucher |
| promoCode | String | Selected promo code |
| creditCard | CreditCardDto | Credit card information (can affect the price) |
| price | PriceDto | Price calculated for usual addresses (address book, google geo, etc) |

All fields except price field are the same as for /calculate/price endpoint

## Response:

### ProposedAddressesResponse

| Parameter | Type | Description |
|---|---|---|

## Request example:

**Request**


```
{
  "customerType": "CORPORATE",
  "requestId":"1328d164-23ec-4b45-ba54-2078ce6a7d4e",
  "operationType": "CREATE",
  "usedJobId":null,
  "service":{
    "customerType":"CORPORATE",
    "id":"8cdec226-06cf-45a4-bfe9-61b8d13b5dc6",
    "caption":"BlackCab",
    "description":"1",
    "code":null,
    "delivery":false
  },
  "paymentType":{

    "customerType":"CORPORATE",
    "name":"Invoice",
    "code":"INVOICE"
  },
  "asap":true,
  "date":null,
  "stops":[
    {
      "id":null,
      "address":{
        "addressId":null,
        "caption":"Teatrul Naional Bucureti, Bulevardul Nicolae Bălcescu 2, Bucureti",
        "latitude":44.4365587,
        "longitude":26.1035929,
        "source":"GOOGLE_PLACE",
        "data":"UkV.....R019",
        "fields":{
```


**Response**


```
{
 "status": "OK",
 "errorMessage": null,
 "proposedAddressCombinations": [
 {
 "pickupProposedAddress": {
 "addressId": "13f1e38e-fbbe-f3de-c16a-ec32f97dd8d2",
 "caption": "Bibliothèque-Discothèque Hergé, 2 Rue du Département, 75019 Paris (NEAR TO 28 Rue de Tanger, 75019 Paris)",
 "latitude": 48.88500670000001,
 "longitude": 2.3674628,
 "source": "PROPOSED_ADDRESS",
 "data": null,
 "fields": {
 "initialFormattedAddress": "47 Rue d'Aubervilliers 75018 Paris",
 "initialLatitude": "48.8875",
 "initialLongitude": "2.366776"
 },
 "restrictions": null
 },
 "dropOffProposedAddress": null,
 "price": {
 "net": {
 "value": 29.88,
 "formattedValue": "€29,88"
 },
 "taxTotal": {
 "value": 1.49,
 "formattedValue": "€1,49"
 },
 "gross": {
 "value": 31.37,
 "formattedValue": "€31,37"
 },
 "discount": {
 "value": 0,
 "formattedValue": "€0,00"
 },
 "details": null,
 "priceDetails": null,
 "prefix": null
 },
 "asapDelay": 20,
 "journeyTime": 40,
 "journeyDistance": 12.714,
 "walkingDistance": 395
 }
 ]
}
```

| | | |
|---|---|---|
| proposedAddressCombinations | List<ProposedDto> | Proposed address combinations |

### ProposedDto

| Parameter | Type | Description |
|---|---|---|
| pickupProposedAddress | AddressDto | Proposed pickup address if found or null if source pickup should be used for this combination |
| dropOffProposedAddress | AddressDto | Proposed drop off address if found or null if source drop off should be used for this combination |
| price | PriceDto | Price |
| asapDelay | Integer | Delay |
| journeyTime | Integer | Journey time |
| journeyDistance | Double | Distance |
| walkingDistance | Long | Walking distance |


"county": "Municipiul Bucureti",
"street":" Bulevardul Nicolae Bălcescu",
"district": "Sector 1",
"country": "România",
"city":"Bucureti",
"postcode":"010051",
"buildingName":"2",
"neighborhood":null,
"companyName":"Teatrul Naional Bucureti"
},
"restrictions":null
},
"notes":null,
"arrivalDetails":null,
"contact":null
}
],
"routeInfo":{
"waitAndReturn":false,
"destinationUnknown":true,
"asDirected":false,
"asDirectedMinutes":0
},
"instructions":[
{
"typeId":"fa2544e3-4ea2-55ba-f4ae-352245cdd9de",
"code":"gor",
"name":"gor",

"value":"11",
"set":true
}
],
"voucher":null,
"loyaltyCard":null,

"promoCode":null,
"creditCard":null,

```
  "price":{
    "net":{
      "value":17.00,

      "formattedVa
lue":"17.00 RON"
    },
    "taxTotal":{
      "value":0.00,
      "formattedVa
lue":"0.00 RON"
    },
    "gross":{
      "value":17.00,

      "formattedVa
lue":"17.00 RON"
    },
    "discount":{
      "value":0.00,
      "formattedVa
lue":"0.00 RON"
    },
    "details":{
      "Base Amou
nt Without Tax":"1
7.00 RON",
      "Client Total
Price":"17.00 RON"

    },
    "priceDetails":[

      {
        "name":"B
ase Amount Witho
ut Tax",
        "amount":{

          "value":1
7.00,
          "formatte
dValue":"17.00 RO
N"
        },
        "type":"BA
SE"
      },
      {
        "name":"Cli
ent Total Price",
        "amount":{

          "value":1
7.00,
          "formatte
dValue":"17.00 RO
N"
        },
        "type":"TO
TAL"
      }
    ]
  },
}
```

# 4. Address details

## URL: /addresses/search/details

Allows details fetch for google autocomplete addresses.

Request field **caption**, **data**, **source** should be taken from address search by query response without change

**Request | AddressDetailsRequest**

```
{
  "caption": "string",
  "customerType": "CORPORATE",
  "data": "string",
  "source": "PLACE_AUTOCOMPLETE"
}
```

Response contains AddressDto field

**Response | AddressResponse**

```
{
  "address": {
    "addressId": "string",
    "caption": "string",
    "data": "string",
    "fields": {
      "additionalProp1": "string",
      "additionalProp2": "string",
      "additionalProp3": "string"
    },
    "latitude": 0,
    "longitude": 0,
    "note": "string",
    "restrictions": {
      "dropRestricted": true,
      "intermediateRestricted": true,
      "pickupRestricted": true
    },
    "source": "GOOGLE_PLACE"
  },
  "errorMessage": "string",
  "status": "OK"
}
```

# 7. Address book

Address book management is performed using corresponding server methods. The client has an ability to load previously used addresses and manage user's favourite addresses (create, edit, delete and load).

## 1. Load recently used addresses

To load recently used addresses the client specifies a type of loaded addresses and an address loading limit in request information about customer types.

### URL: /addresses/book/load/recent

### Request:

#### RecentAddressRequest

| Parameter | Type | Description |
|---|---|---|
| **customerTypes** | List<CustomerType> | Customer types |
| type | AddressSearchType | Address type to search or nothing to return addresses of all types |
| limit | Integer | Limit for address loading |
| latitude | Double | Latitude for address filtering |
| longitude | Double | Longitude for address filtering |
| distance | Double | Search distance, meters |

#### AddressSearchType

| Value | Description |
|---|---|
| PICKUP("P") | Pickup address |
| DROPOFF("D") | Dropoff address |

### Response:

#### RecentAddressResponse

| Parameter | Type | Description |
|---|---|---|
| addresses | List<RecentAddressDto> | Recently used addresses |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Addresses have been successfully loaded |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

### Request example:

| Request | Response |
| --- | --- |
| {<br>"customerTypes":<br>[<br>"CORPORATE"<br>],<br>"type": "P",<br>"limit": 1,<br>"latitude": 48.88,<br>"longitude":<br>2.312,<br>"distance": 500<br>} | {<br>"status": "OK",<br>"errorMessage": null,<br>"addresses": [<br>{<br>"addressId": "eca78cb0-aca2-d53d-c2cb-cb615e944a8b",<br>"caption": "114 Rue Cardinet, 75017 Paris",<br>"latitude": 48.886,<br>"longitude": 2.312389,<br>"source": "AB",<br>"data": null,<br>"parentId": null,<br>"customerType": "CORPORATE",<br>"favouriteName": null,<br>"usageCount": 10,<br>"lastUsedTs": "05/03/2015 11:36",<br>"global": true<br>}<br>]<br>} |

## 2. Load favourite addresses

### URL: /addresses/book/load/favourites

### Request:

#### AddressesListRequest

| Parameter | Type | Description |
| --- | --- | --- |
| **customerTypes** | List<CustomerType> | Customer types |
| lastUpdateDate | Date | Favourite address last update date |

### Response:

#### AddressesListResponse

| Parameter | Type | Description |
| --- | --- | --- |
| addresses | List<FavouriteAddressDto> | A list of found addresses |
| deleted | Map<UUID, CustomerType> | Deleted favourite address identifier map |
| lastUpdateDate | Date | Last update date |

### Possible response codes:

| Value | Description |
| --- | --- |
| OK | Addresses have been successfully loaded |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |

| | |
|---|---|
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

### Request example:

| Request | Response |
|---|---|
| {<br>"customerTypes":<br>[<br>"CORPORATE"<br>],<br>"type": "P",<br>"limit": 1<br>} | {<br>"status": "OK",<br>"errorMessage": null,<br>"lastUpdateDate": "29/06/2015 12:18",<br>"addresses": [<br>{<br>"addressId": "5f746b02-29e9-0e27-4ac7-0f53e0e16425",<br>"caption": "Massy Auto Services, 454 Avenue du Maréchal Leclerc, 91300 Massy",<br>"latitude": 48.718547,<br>"longitude": 2.298474,<br>"source": "AB",<br>"data": null,<br>"parentId": null,<br>"customerType": "CORPORATE",<br>"favouriteName": "Massy",<br>"usageCount": null,<br>"lastUsedTs": null,<br>"global": false<br>}<br>],<br>"deleted": {}<br>} |

## 3. Add an address

### URL: /addresses/book/add

### Request:

#### AddressCreateRequest

| Parameter | Type | Description |
|---|---|---|
| **customerType** | CustomerType | Customer type |
| **favouriteAddress** | ShortFavouriteAddressDto | Address to add to the address book |

#### ShortFavouriteAddressDto

| Parameter | Type | Description |
|---|---|---|
| customerType | CustomerType | Customer type |
| addressId | UUID | Address identifier |
| **caption** | String | Formatted address |
| **latitude** | Double | Address latitude |
| **longitude** | Double | Address longitude |
| **source** | AddressSource | Address source |
| **favouriteName** | String | Address favourite name |
| note | String | Address notes |

| | | |
|---|---|---|
| data | String | Encoded additional data |
| fields | Map<String, String> | Additional data |

## Response:

### AddressCreateResponse

| Parameter | Type | Description |
|---|---|---|
| address | FavouriteAddressDto | Saved address |

## Possible response codes:

| Value | Description |
|---|---|
| OK | Address has been successfully saved |
| INVALID_REQUEST | Address details are not specified |
| ADDRESS_ALREADY_EXISTS | Address already exists in the address book |
| INVALID_COORDINATES | Provided coordinates are outside the allowed area |
| INVALID_ADDRESS_SOURCE | Address with the specified source cannot be added to the address book |

## Request example:

| Request | Response |
|---|---|
| {<br>"customerType": "CORPORATE",<br>"favouriteAddress": {<br>"data": null,<br>"favouriteName": "example",<br>"addressId": null,<br>"source": "UN",<br>"caption": "114 Rue Cardinet, 75017 Paris",<br>"latitude": 48.886,<br>"longitude": 2.312389<br>}<br>} | {<br>"status": "OK",<br>"errorMessage": null,<br>"address": {<br>"addressId": "7f7cae62-ffe7-0188-ae99-86474c4ef8ac",<br>"caption": "114 Rue Cardinet, 75017 Paris",<br>"latitude": 48.886,<br>"longitude": 2.312389,<br>"source": "AB",<br>"data": null,<br>"parentId": null,<br>"customerType": "CORPORATE",<br>"favouriteName": "example_",<br>"usageCount": null,<br>"lastUsedTs": null,<br>"global": false<br>}<br>} |

# 4. Edit an address

## URL: /addresses/book/edit

## Request:

### AddressEditRequest

| Parameter | Type | Description |
|---|---|---|
| **customerType** | CustomerType | Customer type |

| | | |
|---|---|---|
| **favouriteAddress** | ShortFavouriteAddressDto | Address to edit |

### Response:

Cf. AddressCreateResponse

### Possible response codes:

| Value | Description |
|---|---|
| OK | Address has been successfully modified |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| INVALID_REQUEST | Address details are not specified |
| INVALID_COORDINATES | Provided coordinates are outside the allowed area |
| DATA_NOT_FOUND | Specified address is not found |
| INVALID_ACCOUNT_DATA | User tries to edit an address related to another user |
| INVALID_ADDRESS_SOURCE | Address with the specified source cannot be modified |

### Request example:

| Request | Response |
|---|---|
| {<br>"customerType": "CORPORATE",<br>"favouriteAddress": {<br>"data": null,<br>"favouriteName": "example",<br>"addressId": null,<br>"source": "UN",<br>"caption": "114 Rue Cardinet, 75017 Paris",<br>"latitude": 48.886,<br>"longitude": 2.312389<br>}<br>} | {<br>"status": "OK",<br>"errorMessage": null,<br>"address": {<br>"addressId": "7f7cae62-ffe7-0188-ae99-86474c4ef8ac",<br>"caption": "114 Rue Cardinet, 75017 Paris",<br>"latitude": 48.886,<br>"longitude": 2.312389,<br>"source": "AB",<br>"data": null,<br>"parentId": null,<br>"customerType": "CORPORATE",<br>"favouriteName": "example_",<br>"usageCount": null,<br>"lastUsedTs": null,<br>"global": false<br>}<br>} |

## 5. Delete an address

### URL: /addresses/book/delete

### Request:

#### AddressDeleteRequest

| Parameter | Type | Description |
|---|---|---|
| **addressId** | UUID | Identifier of an address to delete |
| **customerType** | CustomerType | Customer type |

### Response:

Cf. BaseResponse.

### Possible response codes:

| Value | Description |
|---|---|
| OK | Address has been successfully deleted |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| INVALID_REQUEST | Address details are not specified |
| INTERNAL | Operation failed |
| GLOBAL_ADDRESS | Impossible to delete a global address in the address book |
| DATA_NOT_FOUND | Specified address is not found |
| INVALID_ACCOUNT_DATA | User tries to delete an address related to another user |
| INVALID_ADDRESS_SOURCE | Address with the specified source cannot be deleted |

### Request example:

| Request | Response |
|---|---|
| {<br>"customerType": "CORPORATE",<br>"addressId": "eca78cb0-aca2-d53d-c2cb-cb615e944a8b"<br>} | {<br>"status": "OK",<br>"errorMessage": null,<br>} |

## 6. Load most used addresses

Load most used addresses to use in a new booking. It cannot be used for anonymous users.

### URL: /addresses/book/load/mostUsed

### Request:

#### MostUsedAddressRequest

| Parameter | Type | Description |
|---|---|---|
| **customerTypes** | List<CustomerType> | Customer types |
| type | AddressSearchType | Address type to search or nothing to return addresses of all types |

| | | |
|---|---|---|
| limit | Integer | Limit for address loading |

### Response:

Cf. RecentAddressResponse

### Possible response codes:

| Value | Description |
|---|---|
| OK | Addresses have been successfully loaded |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

### Request example:

| Request | Response |
|---|---|
| {<br>"customerTypes":<br>[<br>"CORPORATE"<br>],<br>"type": "P",<br>"limit": 1<br>} | {<br>"status": "OK",<br>"errorMessage": null,<br>"addresses": [<br>{<br>"addressId": "eca78cb0-aca2-d53d-c2cb-cb615e944a8b",<br>"caption": "114 Rue Cardinet, 75017 Paris",<br>"latitude": 48.886,<br>"longitude": 2.312389,<br>"source": "AB",<br>"data": null,<br>"parentId": null,<br>"customerType": "CORPORATE",<br>"favouriteName": null,<br>"usageCount": 10,<br>"lastUsedTs": "05/03/2015 11:36",<br>"global": true<br>}<br>]<br>} |

## 7. Check if a point inside operational area

Check if a point inside operational area. If a service is not specified in the request, check will be performed for all services. Result will be successfull if specified point is located inside at least one service's operational area.

### URL: /addresses/checkOperationArea

### Request:

#### CheckOperationAreaRequest

| Parameter | Type | Description |
|---|---|---|
| serviceDto | CustomerType | service |
| **latitude** | Double | Address latitude |

| | | |
|---|---|---|
| **longitude** | Double | Address longitude |
| **customerType** | CustomerType | Customer type |

### Response:

Cf. BaseResponse.

### Possible response codes:

| Value | Description |
|---|---|
| OK | The point located inside operational area of the specified service. Or there is at least one available service which contains the specified point (in case of unspecified service). |
| INVALID_OPERATIONAL_AREA | The specified point can't be used in operational area. |
| OPERATION_UNAVAILABLE | Latitude or longitude isn't specified. |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_REQUEST | Address details are not specified |

## 8. Load all restricted addresses

### POST /addresses/book/load/available

- Only CORPORATE request type is allowed
- Returns list of account addresses along with usage information, airports and train stations
- Every address contains restrictions information
- Returns empty list if account has no address restrictions in booking settings

Request:

```
{ "customerType": "CORPORATE" }
```

```
{
  "airports": [
    {
      "addressId": "string",
      "caption": "string",
      "customerType": "CORPORATE",
      "data": "string",
      "fields": {
        "additionalProp1": "string",
        "additionalProp2": "string",
        "additionalProp3": "string"
      },
      "latitude": 0,
      "longitude": 0,
      "note": "string",
      "parentId": "string",
      "restrictions": {
        "dropRestricted": true,
```

```
        "intermediateRestricted": true,
        "pickupRestricted": true
      },
      "source": "GOOGLE_PLACE"
    }
  ],
  "book": [
    {
      "addressId": "string",
      "caption": "string",
      "customerType": "CORPORATE",
      "data": "string",
      "favourite": true,
      "favouriteName": "string",
      "fields": {
        "additionalProp1": "string",
        "additionalProp2": "string",
        "additionalProp3": "string"
      },
      "global": true,
      "lastUsedTs": "2019-01-28T13:42:40.800Z",
      "latitude": 0,
      "longitude": 0,
      "note": "string",
      "parentId": "string",
      "restrictions": {
        "dropRestricted": true,
        "intermediateRestricted": true,
        "pickupRestricted": true
      },
      "source": "GOOGLE_PLACE",
      "usageCount": 0
    }
  ],
  "errorMessage": "string",
  "status": "OK",
  "trains": [
    {
      "addressId": "string",
      "caption": "string",
      "customerType": "CORPORATE",
      "data": "string",
      "fields": {
        "additionalProp1": "string",
        "additionalProp2": "string",
        "additionalProp3": "string"
      },
      "latitude": 0,
      "longitude": 0,
      "note": "string",
      "parentId": "string",
      "restrictions": {
        "dropRestricted": true,
        "intermediateRestricted": true,
        "pickupRestricted": true
      },
      "source": "GOOGLE_PLACE"
    }
  ]
}
```


Possible error response statuses:

INVALID_RETAIL_REQUEST

INVALID_ANONYMOUS_REQUEST

## 9. Check availability of job stops according to address restrictions

### POST /addresses/book/stops/check

- Only CORPORATE request type is allowed
- Returns list of stops, which were used in job but currently unavailable for specified booking operation type
- Returns empty array list if account has no address restrictions in booking configuration settings
- Returns message field to display if job has unavailable address

Request:

Accepts job id, customer type and booking operation type

```
{
  "customerType": "CORPORATE",
  "jobId": "string",
  "type": "CREATE"
}
```

```
{
  "errorMessage": "string",
  "message": "string",
  "status": "OK",
  "stops": [
    {
      "address": {
        "addressId": "string",
        "caption": "string",
        "data": "string",
        "fields": {
          "additionalProp1": "string",
          "additionalProp2": "string",
          "additionalProp3": "string"
        },
        "latitude": 0,
        "longitude": 0,
        "note": "string",
        "restrictions": {
          "dropRestricted": true,
          "intermediateRestricted": true,
          "pickupRestricted": true
        },
        "source": "GOOGLE_PLACE"
      },
      "arrivalDetails": {
        "airline": {
          "iata": "string",
          "icao": "string",
          "id": "string",
          "name": "string"
        },
        "arrivalFrom": "string",
        "arrivalNumber": "string",
        "arrivalTime": "2019-01-28T13:47:47.966Z",
        "arrived": true,
        "pickupTimeDelay": 0,
        "shipName": "string"
      },
      "contact": {
        "customerType": "CORPORATE",
        "email": "string",
        "individualId": "string",
        "name": "string",
        "phone": "string"
      },
      "id": "string",
      "notes": "string"
    }
  ]
}
```


Possible error response statuses:

INVALID_RETAIL_REQUEST

INVALID_ANONYMOUS_REQUEST

DATA_NOT_FOUND - job wasn't found by specified id

# 8. Airports and terminals

## 1. Load airports

### URL: /addresses/airports

### Request:

Empty request.

### Response:

#### AddressSearchResponse

| Parameter | Type | Description |
|---|---|---|
| addresses | List<AddressDto> | Loaded airports |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Airports have been successfully loaded |

### Request example:

| Request | Response |
|---|---|
| | {<br>"status": "OK",<br>"errorMessage": null,<br>"addresses": [<br>{<br>"addressId": "cf27536e-d558-4d7c-944f-65fcaaa28b57",<br>"caption": "Charles De Gaulle Airport, 95711 Paris",<br>"latitude": 49.004,<br>"longitude": 2.571,<br>"source": "AP",<br>"data": null<br>}<br>]<br>} |

## 2. Load an airport by ID

### URL: /addresses/airports/loadById

### Request:

#### AddressRequest

| Parameter | Type | Description |
|---|---|---|
| addressId | UUID | Entity ID |

## Response:

### AddressResponse

| Parameter | Type | Description |
|---|---|---|
| address | AddressDto | Loaded entity |

## Possible response codes:

| Value | Description |
|---|---|
| OK | Airport has been successfully loaded |
| INVALID_REQUEST | Airport identifier is required |
| DATA_NOT_FOUND | Airport is not found |

# 3. Load airport terminals

## URL: /addresses/airports/terminals

## Request:

### RelatedAddressesRequest

| Parameter | Type | Description |
|---|---|---|
| **parentId** | UUID | Related address identifier. For this case – an airport identifier |

## Response:

Cf. AddressSearchResponse.

## Possible response codes:

| Value | Description |
|---|---|
| OK | Terminals have been successfully loaded |
| INVALID_REQUEST | Airport identifier is required |

## Request example:

| Request | Response |
|---|---|

| | |
|---|---|
| {<br>"parentId": "cf27536e-d558-4d7c-944f-65fcaaa28b57"<br>} | {<br>"status": "OK",<br>"errorMessage": null,<br>"addresses": [<br>{<br>"addressId": "e83dedaa-e49a-486a-bf49-6de24976b5b0",<br>"caption": "Charles de Gaulle, Terminal 2F",<br>"latitude": 49.01,<br>"longitude": 2.5770545040131,<br>"source": "AT",<br>"data": null<br>},<br>{<br>"addressId": "2a2bcc8f-ad5c-4b3b-bddf-b145a7d52b1c",<br>"caption": "Charles de Gaulle, Terminal 2G",<br>"latitude": 49.01,<br>"longitude": 2.5711,<br>"source": "AT",<br>"data": null<br>}<br>]<br>} |

## 4. Load an airport terminal by ID

### URL: /addresses/airports/terminals/loadById

### Request:

#### AddressRequest

| Parameter | Type | Description |
|---|---|---|
| addressId | UUID | Entity ID |

### Response:

#### AddressResponse

| Parameter | Type | Description |
|---|---|---|
| address | AddressDto | Loaded entity |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Airport terminal has been successfully loaded |
| INVALID_REQUEST | Airport terminal identifier is required |
| DATA_NOT_FOUND | Airport terminal is not found |

## 5. Load terminal meeting points

### URL: /addresses/airports/terminals/meetingPoints

## Request:

### TerminalMeetingPointsRequest

| Parameter | Type | Description |
|---|---|---|
| **customerTypes** | List<CustomerType> | Customer types |
| **parentId** | UUID | Terminal identifier |

## Response:

### TerminalMeetingPointsResponse

| Parameter | Type | Description |
|---|---|---|
| meetingPoints | List<CustomerAddressDto> | List of found addresses |

## Possible response codes:

| Value | Description |
|---|---|
| OK | Terminals have been successfully loaded |
| INVALID_REQUEST | List of customer types is required |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

## Request example:

| Request | Response |
|---|---|

| | |
|---|---|
| {<br>"customerTypes": [<br>"CORPORATE"<br>],<br>"parentId": "c300a50d-0b40-4011-b8c7-d6bc27c76c3d"<br>} | {<br>"status": "OK",<br>"errorMessage": null,<br>"addresses": [<br>{<br>"addressId": "c300a50d-0b40-4011-b8c7-d6bc27c76c3d",<br>"caption": "Terminal 3",<br>"latitude": 49.00319528956596,<br>"longitude": 2.5328636169433594,<br>"source": "AMP",<br>"data": null,<br>"parentId": "c300a50d-0b40-4011-b8c7-d6bc27c76c3d",<br>"customerType": "CORPORATE"<br>},<br>{<br>"addressId": "45fa58e9-7d79-8853-0597-09f326a3533c",<br>"caption": "Inside",<br>"latitude": 49.00319528956596,<br>"longitude": 2.5328636169433594,<br>"source": "AMP",<br>"data": null,<br>"parentId": "c300a50d-0b40-4011-b8c7-d6bc27c76c3d",<br>"customerType": "CORPORATE"<br>}<br>]<br>} |

## 6. Load airport hierarchy

### URL: /addresses/airports/full

### Request:

Cf. BaseDatedRequest.

### Response:

Cf. AddressHierarchyResponse.

### Possible response codes:

| Value | Description |
|---|---|
| OK | Terminals have been successfully loaded |
| INVALID_REQUEST | List of customer types is required |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

### Request example:

| Request | Response |
|---|---|

| | |
|---|---|
| {<br>"lastUpdateDate":<br>null<br>} | {<br>"status": "OK",<br>"errorMessage": null,<br>"lastUpdateDate": "28/06/2015 14:09",<br>"deleted": [<br>"b24868a5-e0f7-4a40-b762-b695bf2befe2",<br>"1b45191f-e094-485d-834f-895711864778",<br>"10fa40e7-09c9-422c-be07-8cef9700949f"<br>],<br>"addresses": [<br>{<br>"addressId": "cf27536e-d558-4d7c-944f-65fcaaa28b57",<br>"caption": "Charles De Gaulle Airport, 95711 Paris",<br>"latitude": 49.004,<br>"longitude": 2.571,<br>"source": "AP",<br>"data": null,<br>"fields": null,<br>"parentId": null<br>},<br>{<br>"addressId": "516481fe-2b36-4536-8595-<br>e65ceb2cea34",<br>"caption": "Charles de Gaulle, Terminal 1",<br>"latitude": 49.01,<br>"longitude": 2.5419477,<br>"source": "AT",<br>"data": null,<br>"fields": null,<br>"parentId": "cf27536e-d558-4d7c-944f-65fcaaa28b57"<br>},<br>{<br>"addressId": "7b05b508-5e88-4d9a-8180-faf8603df1a1",<br>"caption": "Charles de Gaulle, Terminal 2A",<br>"latitude": 49.003,<br>"longitude": 2.562,<br>"source": "AT",<br>"data": null,<br>"fields": null,<br>"parentId": "cf27536e-d558-4d7c-944f-65fcaaa28b57"<br>},<br>]<br>} |

# 9. Train stations and meeting points

## 1. Load train stations

### URL: /addresses/trainStations

### Request:

Empty request.

### Response:

Cf. AddressSearchResponse.

### Possible response codes:

| Value | Description |
|---|---|
| OK | Train stations have been successfully loaded |

### Request example:

| Request | Response |
|---|---|
| | {<br>"status": "OK",<br>"errorMessage": null,<br>"addresses": [<br>{<br>"addressId": "604ee8f3-b288-4210-8753-a14986f88ac1",<br>"caption": "Gare de l'Est",<br>"latitude": 48.876141,<br>"longitude": 2.358283,<br>"source": "TR",<br>"data": null,<br>"parentId": null<br>},<br>{<br>"addressId": "fca79cf5-e24b-4a8a-9a18-e88a6232047f",<br>"caption": "Gare d'Austerlitz",<br>"latitude": 48.84263499999999,<br>"longitude": 2.364969,<br>"source": "TR",<br>"data": null,<br>"fields": null,<br>"parentId": null<br>}<br>]<br>} |

## 2. Load a train station by ID

### URL: /addresses/trainStations/loadById

### Request:

#### AddressRequest

| Parameter | Type | Description |
|---|---|---|
| addressId | UUID | Entity ID |

### Response:

#### AddressResponse

| Parameter | Type | Description |
|---|---|---|
| address | AddressDto | Loaded entity |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Train station has been successfully loaded |
| INVALID_REQUEST | Train station identifier is required |
| DATA_NOT_FOUND | Train station is not found |

## 3. Load train station meeting points

### URL: /addresses/trainStations/meetingPoints

### Request:

Cf. RelatedAddressesRequest.

### Response:

Cf. AddressSearchResponse.

### Possible response codes:

| Value | Description |
|---|---|
| OK | Train station meeting points have been successfully loaded |
| INVALID_REQUEST | Train station identifier is required |

### Request example:

| Request | Response |
|---|---|
| | |

## 4. Load train station hierarchy

### URL: /addresses/trainStations/full

### Request:

Cf. BaseDatedRequest.

### Response:

Cf. AddressHierarchyResponse.

## Possible response codes:

| Value | Description |
| --- | --- |
| OK | Terminals have been successfully loaded |
| INVALID_REQUEST | List of customer types is required |

## Request example:

| Request | Response |
| --- | --- |
| | |

# 10. Special Places and meeting points

## 1. Load Special Place types

### URL: /addresses/special/types

### Request:

Empty request.

### Response:

#### SpecialPlaceTypeResponse

| Parameter | Type | Description |
|---|---|---|
| types | List<SpecialPlaceTypeDto> | Loaded Special Place types |

#### SpecialPlaceTypeDto

| Parameter | Type | Description |
|---|---|---|
| id | UUID | Special Place type identifier |
| name | String | Special Place type name |
| code | String | Special Place type code |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Special Place types have been successfully loaded |

### Request example:

| Request | Response |
|---|---|
| | {<br>"status": "OK",<br>"errorMessage": null,<br>"types": [<br>{<br>"id": "05fd3f42-ef52-4460-b377-87a08c0c4887",<br>"name": "Stades",<br>"code": null<br>}<br>]<br>} |

## 2. Load Special Places

### URL: /addresses/special/places

### Request:

#### SpecialPlaceRequest

| Parameter | Type | Description |
|---|---|---|
| parentId | UUID | Special Place type identifier |
| searchString | String | Special Place filter string |

### Response:

Cf. AddressSearchResponse.

### Possible response codes:

| Value | Description |
|---|---|
| OK | Special Places have been successfully loaded |
| INVALID_REQUEST | Special Place type identifier is required |

### Request example:

| Request | Response |
|---|---|
| {<br>"parentId": "05fd3f42-ef52-4460-b377-87a08c0c4887"<br>} | {<br>"status": "OK",<br>"errorMessage": null,<br>"addresses": [<br>{<br>"addressId": "4db1a81f-757b-4761-9cca-4318e3a59ae0",<br>"caption": "Parc des Princes",<br>"latitude": 48.8417851,<br>"longitude": 2.2522386,<br>"source": "SP",<br>"data": null,<br>"parentId": "05fd3f42-ef52-4460-b377-87a08c0c4887"<br>}<br>]<br>} |

## 3. Load a Special Place by ID

### URL: /addresses/special/places/loadById

### Request:

#### AddressRequest

| Parameter | Type | Description |
|---|---|---|
| addressId | UUID | Entity ID |

### Response:

#### AddressResponse

| Parameter | Type | Description |
|---|---|---|
| address | AddressDto | Loaded entity |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Special Place has been successfully loaded |
| INVALID_REQUEST | Special Place identifier is required |
| DATA_NOT_FOUND | Special Place is not found |

## 4. Load Special Place meeting points

### URL: /addresses/special/points

### Request:

Cf. RelatedAddressesRequest.

### Response:

Cf. AddressSearchResponse.

### Possible response codes:

| Value | Description |
|---|---|
| OK | Special Place meeting points have been successfully loaded |
| INVALID_REQUEST | Special Place identifier is required |

### Request example:

| Request | Response |
|---|---|
| {<br>"parentId": "4db1a81f-757b-4761-9cca-4318e3a59ae0"<br>} | {<br>"status": "OK",<br>"errorMessage": null,<br>"addresses": [<br>{<br>"addressId": "5217457a-41fd-438c-9895-19daffaca302",<br>"caption": "Devant le lycée Jean de la Fontaine - porte Molitor",<br>"latitude": 48.8453581,<br>"longitude": 2.2562627,<br>"source": "MP",<br>"data": null,<br>"parentId": "4db1a81f-757b-4761-9cca-4318e3a59ae0"<br>},<br>{<br>"addressId": "b3b3d2b9-d731-4f4f-aefe-fd09d8e1eb31",<br>"caption": "Blvd Murat - Place du général Stéphanix",<br>"latitude": 48.840008,<br>"longitude": 2.256158,<br>"source": "MP",<br>"data": null,<br>"parentId": "4db1a81f-757b-4761-9cca-4318e3a59ae0"<br>}<br>]<br>} |

## 5. Load Special Place hierarchy

## URL: /addresses/special/full

## Request:

Cf. BaseDatedRequest.

## Response:

### SpecialPlaceHierarchyResponse

| Parameter | Type | Description |
|---|---|---|
| types | List<SpecialPlaceTypeDto> | List of Special Place types |
| deletedTypes | List<UUID> | Deleted Special Place type identifiers |
| addresses | List<RelatedAddressDto> | Loaded Special Places and meeting points |
| deleted | List<UUID> | Deleted identifiers of Special Places and meeting points |
| lastUpdateDate | Date | Date to send in a request for the next loading (used to load data changed after the last loading) |

## Possible response codes:

| Value | Description |
|---|---|
| OK | Terminals have been successfully loaded |
| INVALID_REQUEST | List of customer types is required |

# 11. Service management

## 1. Check service availability

To check whether a service is available for the selected pickup address the client sends a corresponding request. The server returns the 'true' value in a response in case there is an available service for any of the specified customer types.

### URL: /data/services/available

### Request:

#### LoadServicesRequest

| Parameter | Type | Description |
|---|---|---|
| latitude | Double | Latitude |
| longitude | Double | Longitude |
| **customerTypes** | List<CustomerType> | List of profile types |
| amendJobId | UUID | Amend job id |

### Response:

#### CheckServicesResponse

| Parameter | Type | Description |
|---|---|---|
| available | boolean | Indicates if any service is available |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Service availability check has been passed successfully |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

### Request example:

| Request | Response |
|---|---|
| {<br>"customerTypes" : [<br>"CORPORATE",<br>"RETAIL"<br>]<br>"longitude": 51.508529,<br>"latitude": -0.111237<br>} | {<br>"status": "OK",<br>"errorMessage":<br>null,<br>"available": true<br>} |

## 2. Load available services

Services available to an authorised user are loaded using the method specified below. The ServiceDto object returned from the server contains the settings fields that are not required to be sent back to the server. Thus, in price calculation, offer making and booking confirmation methods the client should send the BookingServiceDto object that contains only such data which is necessary for booking creation. If service has not available payment types, it will not returned in this endpoint.

## URL: /data/services/load

## Request:

Cf. LoadServicesRequest

## Response:

### LoadServicesResponse

| Parameter | Type | Description |
|---|---|---|
| services | List<ServiceDto> | Loaded services |

### ServiceDto

| Parameter | Type | Description |
|---|---|---|
| id | UUID | Service identifier |
| customerType | CustomerType | Customer type |
| caption | String | Service caption.<br>For mobile clients (the device is specified during login): Service → Mobile Caption if specified; otherwise, Service → Caption<br>For Web Portal clients (the device is not specified during login): Service → Web Caption if specified; otherwise, Service → Caption |
| description | String | Service description<br>For mobile clients: Service → Mobile Description if specified; otherwise, Service → Caption<br>For Web Portal clients: null |
| delivery | boolean | Indicates if a service is considered as a Delivery Service |
| offerService | boolean | Indicates if a service is considered as an Offer Service |
| routeSettings | RouteSettings | Route settings |
| tariffDescription | String | Tariff Description |
| shortTariffDescription | String | Short Tariff Description |
| pricePrefix | String | Price prefix |
| passengerCount | Integer | Maximum number of passengers. Contains **null** value in case when field 'delivery' is set to true. |
| availablePaymentTypes | List<String> | List of available payment type codes (for this service) |
| unavailablePaymentTypes | Map<String, String> | Map of unavailable payment type codes with reasons of unavailability |
| partner | PartnerDto | Partner information |
| enableAddressProposal | boolean | Indicates if service supports proposal addresses. If true, client can request server for proposal addresses using /calculate/proposedAddresses endpoint. |
| proposedService | ServiceDto | Proposed service |
| passengerCount | Integer | Maximum number of passengers |

| | | |
|---|---|---|
| defaultSmallBags | Integer | Default count of small bags calculated for the bagsCapacity value. Default big bags count is calculated to fit a half of capacity with rouding up if result is not a whole number.<br>Remaining capacity is used to calculate default small bags.<br>Contains **null** value in case when field 'delivery' is set to true. |
| defaultBigBags | Integer | Default count of big bags calculated for the bagsCapacity value. Default big bags count is calculated to fit a half of capacity with rouding up if result is not a whole number.<br>Remaining capacity is used to calculate default small bags.<br>Contains **null** value in case when field 'delivery' is set to true. |
| capacityRequirementsSettings | CapacityRequirementsSettingsDto | Capacity requirements settings. Contains **null** value in case when field 'delivery' is set to true. |
| vehicleModelType | VehicleModelTypeDto | Information about vehicle model type |
| asapOnly | boolean | Indicates if a service can be used only for ASAP bookings |

### RouteSettings

| Parameter | Type | Description |
|---|---|---|
| asDirectedAvailable | boolean | Indicates if 'As directed' bookings can be created with the specified service |
| minAsDirectedMinutes | Integer | The minumum time (in minutes) for 'As directed' bookings if enabled and populated; otherwise, null |
| destinationUnknownAvailable | boolean | Indicates if 'Destination unknown' bookings can be created with the specified service |
| waitAndReturnAvailable | boolean | Indicates if 'Wait & return' bookings can be created with the specified service |

### PartnerDto

| Parameter | Type | Description |
|---|---|---|
| caption | String | Partner localyzed caption |
| city | String | Localized city name where partner located |
| time zone | String | Partner timezone. Please note all dates posted to server or recieved from server should be in partner timezone.<br><br>Example:<br><br>Server timezone is Europe/Paris, server time is 10:00<br><br>Partner timezone is Europe/London, time is 9:00 in London.<br><br>If user need to confirm booking in 9:00, client should sent 9:00. Server will convert it to 10:00 and sent 10:00 and Europe/London timezone to the partner. Partner server will consider this and the taxi will be served by 9:00 London time. |
| latitude | Double | Default latitude |
| longitude | Double | Default longitude |

### CapacityRequirementsSettingsDto

| Parameter | Type | Description |
|---|---|---|
| requiredForTrains | boolean | True if baggage capacity settings is turned on in the service and system settings and false otherwise |
| requiredForAirports | boolean | True if baggage capacity settings is turned on in the service and system settings and false otherwise |

| | | |
|---|---|---|
| requiredForShips | boolean | True if baggage capacity settings is turned on in the service and system settings and false otherwise |
| allowLargeBagsInInterior | boolean | True if a vehicle interior can be used for large bags and false otherwise |
| bagsCapacityUnits | Integer | Maximum baggage capacity in units for a specified service |
| passengerCapacityUnits | Double | Units amount occupied by one passenger |
| bigBagCapacityUnits | Double | Units amount occupied by one big bag |
| smallBagCapacityUnits | Double | Units amount occupied by one small bag |

### VehicleModelTypeDto

| Parameter | Type | Description |
|---|---|---|
| name | String | Vehicle model name |
| code | String | Vehicle model code |

| Value | Description |
|---|---|
| OK | Services have been successfully loaded |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

## Request example:

| Request | Response |
|---|---|

```
{
"customerTypes" : [
"CORPORATE",
"RETAIL"
]
"longitude": null,
"latitude": null
}
```

```
{
"status": "OK",
"errorMessage": null,
"services": [
{
"customerType": "CORPORATE",
"id": "4da69fa0-ae72-4c23-a50c-
85f11bb6f831",
"caption": "Test Service",
"description": "Test Service, 4",
"passengerCount": 4,
"offerService": false,
"routeSettings": {
"asDirectedAvailable": true,
"destinationUnknownAvailable": true,
"waitAndReturnAvailable": true,

"enableAddressProposal": false,
"proposedService": null

"passengerCount": 4,
"defaultSmallBags": 2,
"defaultBigBags": 2,
"capacityRequirementsSettings": {
"requiredForTrains": true,
"requiredForAirports": true,
"requiredForShips": true,
"allowLargeBagsInInterior": true,
"bagsCapacityUnits": 6,
"passengerCapacityUnits": 2.0,
"bigBagCapacityUnits": 2.0,
"smallBagCapacityUnits": 1.0
}

}
}
]
}
```

# 12. Payment types

Payment types available to an authorised user should be loaded using the method specified below. The *PaymentTypeDto* object returned from the server contains the settings fields that are not required to be sent back to the server. Thus, in price calculation, offer making and booking confirmation methods the client should send the *BookingPaymentTypeDto* object that contains only such data which is necessary for booking creation.

## Load available payment types

### URL: /data/paymentTypes

### Request:

#### LoadPaymentTypesRequest

| Parameter | Type | Description |
|---|---|---|
| customerTypes | List<CustomerType> | List of customer types |
| amendJobId | UUID | Amend job id |

### Response:

#### LoadPaymentTypesResponse

| Parameter | Type | Description |
|---|---|---|
| paymentTypes | List<PaymentTypeDto> | List of available payment types |

#### PaymentTypeDto

| Parameter | Type | Description |
|---|---|---|
| customerType | CustomerType | Customer type |
| name | String | Payment type name |
| description | String | |
| code | String | Payment type code |
| defaultType | boolean | Indicates if the payment type should be used as a default one for the specified customer type |
| creditCardsAvailable | boolean | Indicates if credit cards are used within the specified payment type |
| cvcRequiredForNewCreditCards | boolean | Indicates if CVC is required for new credit cards |
| cvcRequiredForExistingCreditCards | boolean | Indicates if CVC is required for existing credit cards |
| payInCar | boolean | Indicates if payment to performed in a car |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Payment types have been successfully loaded |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

## Request example:

| Request | Response |
| --- | --- |
| {<br>"customerTypes" : [<br>"CORPORATE",<br>"RETAIL"<br>]<br>} | {<br>"status": "OK",<br>"errorMessage": null,<br>"paymentTypes": [<br>{<br>"customerType": "CORPORATE",<br>"name": "Invoice",<br>"code": "INVOICE",<br>"description": "Invoice afted booking",<br>"defaultType": true,<br>"creditCardsAvailable": false,<br>"cvcRequiredForNewCreditCards": false,<br>"cvcRequiredForExistingCreditCards":<br>false<br>},<br>{<br>"customerType": "RETAIL",<br>"name": "Credit card",<br>"code": "CREDIT_CARD",<br>"description": "Pay with credit card",<br>"defaultType": true,<br>"creditCardsAvailable": true,<br>"cvcRequiredForNewCreditCards": true,<br>"cvcRequiredForExistingCreditCards":<br>false<br>}<br>]<br>} |

# 13. Special instructions

The list of special instructions available to the client depends on the selected service. For each instruction that contains the **parameterExists** and **parameter Mandatory** fields set to the true value, the **value** field is required.
Values of specified instructions should have data type specified in the **parameterType** field. If the **parameterType** filed contains the *ENUM* value, then a special instruction value should be selected from the list of possible values which is specified in the **values** field.
Special instruction validation should be performed after booking details (a service) are already filled in and also before booking confirmation. Valid instructions should be sent during price calculation, delay calculation, offer making and booking confirmation.

## 1. Load available special instructions

### URL: /data/instructions/load

### Request:

#### LoadSpecialInstructionsRequest

| Parameter | Type | Description |
|---|---|---|
| **service** | BookingServiceDto | Service to load instructions |
| amendJobId | UUID | Amend job id |

### Response:

#### LoadSpecialInstructionsResponse

| Parameter | Type | Description |
|---|---|---|
| instructions | List<SpecialInstructionFieldDto> | List of available special instructions |

#### SpecialInstructionFieldDto

| Parameter | Type | Description |
|---|---|---|
| id | UUID | Instruction identifier |
| code | String | Instruction code |
| name | String | Instruction name |
| parameter Exists | boolean | Indicates if the instruction has a parameter to fill in |
| parameter Mandatory | boolean | Indicates if the parameter is mandatory |
| parameter Type | InstructionParameterType | Parameter type |
| values | List<String> | List of available instruction values |
| preference | boolean | Indicates if this instruction is mandatory. For an API verion older than 2.1, in case the client does not send this specical instruction in a booking confirmation request, it will be created automatically on the server side |
| preference Value | String | Preference value which should be set by default |
| extraTemplateCode | String | Extra template code |
| price | PriceField | Calculated Job extra for this instruction. If special instruction type defines script for job extra calculation no parameters will be passed to it |

### InstructionParameterType

| Value | Description |
|---|---|
| STRING("STRING") | String data type |
| INTEGER("INTEGER") | Integer data type |
| FLOAT("FLOAT") | Float (double) data type |
| DATE("DATE") | Date data type. Should be specified in the 'dd/MM/yyyy HH:mm' format |
| ENUM("ENUM") | Enumeration data type. For this parameter type a value should be selected from the list of possible parameter values |

## Possible response codes:

| Value | Description |
|---|---|
| OK | Special instructions have been successfully loaded |
| INVALID_REQUEST | Service is not specified |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

## Request example:

| Request | Response |
|---|---|
| {<br>"service": {<br>"customerType": "CORPORATE",<br>"id": "4da69fa0-ae72-4c23-a50c-85f11bb6f831",<br>"caption": "Test Service",<br>"description": "Test Service, 4",<br>}<br>} | {<br>"status": "OK",<br>"errorMessage": null,<br>"instructions": [<br>{<br>"id":"af7364a1-4c42-0d6d-e322-e5e1ac8de337",<br>"name":"Meet & Greet.",<br>"parameterExists":false,<br>"parameterMandatory":false,<br>"parameterType":null,<br>"values":null,<br>"preferenceValue":null<br>}<br>]<br>} |

# 2. Instruction validation

## URL: /data/instructions/validate

## Request:

### ValidateSpecialInstructionsRequest

| Parameter | Type | Description |
|---|---|---|
| **customerType** | CustomerType | Customer type |

| | | |
|---|---|---|
| **instructions** | List<SpecialInstructionDto> | Instructions to validate |

### SpecialInstructionDto

| Parameter | Type | Description |
|---|---|---|
| **typeId** | UUID | Special instruction type identifier |
| **code** | String | Special instruction type code |
| **name** | String | Special instruction type name |
| **value** | String | Specified parameter value |

## Response:

### ValidateSpecialInstructionsResponse

| Parameter | Type | Description |
|---|---|---|
| validation | List<InstructionsValidationInfo> | Instruction validation information |

### InstructionsValidationInfo

| Parameter | Type | Description |
|---|---|---|
| instructionId | UUID | Special instruction type identifier |
| name | String | Special instruction type name |
| validationInfo | String | Instruction validation error explanation |
| parameterExistsPassed | boolean | Indicates if the parameter existence check has been passed |
| parameterMandatoryPassed | boolean | Indicates if the parameter mandatory check has been passed |
| parameterTypePassed | boolean | Indicates if the parameter type check has been passed |
| parameterValuePassed | boolean | Indicates if the parameter value check has been passed |

## Possible response codes:

| Value | Description |
|---|---|
| OK | Instruction validation checks have been successfully passed |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| INSTRUCTION_VALIDATION_FAILED | Specified special instruction values are not valid |

## Request example:

| Request | Response |
|---|---|

| | |
|---|---|
| {<br>"customerType": "CORPORATE",<br>"instructions": [<br>{<br>"typeId": "af7364a1-4c42-0d6d-e322-e5e1ac8de337",<br>"code": "MEET_AND_GREET",<br>"name": "Meet & Greet.",<br>"value": null<br>}<br>]<br>} | {<br>"status": "OK",<br>"errorMessage": null,<br>} |

# 14. References

## 1. Load a reference schema

### URL: /data/references/schema

### Request:

Empty request.

### Response:

#### ReferenceSchemaResponse

| Parameter | Type | Description |
|---|---|---|
| references | List<ReferenceFieldDto> | Available references |

#### ReferenceFieldDto

| Parameter | Type | Description |
|---|---|---|
| id | UUID | Reference field identifier |
| dependOn | UUID | Master reference field identifier, if exists |
| name | String | Reference field name |
| tooltip | String | Reference field tooltip |
| pattern | String | Reference field pattern |
| defaultOption | String | Default field value option |
| showOptions | boolean | Indicates if the client should show available options |
| mandatory | boolean | Indicates if the field is mandatory |
| mustBeInList | boolean | Indicates if the field value must be from the options list |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Reference schema has been successfully loaded |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |

## 2. Load reference options

### URL: /data/references/option

### Request:

#### ReferenceOptionsRequest

| Parameter | Type | Description |
|---|---|---|
| entityId | UUID | Reference field identifier |

| | | |
|---|---|---|
| masterValue | String | Master reference field |
| filterValue | String | Reference option filter |

### Response:

#### ReferenceOptionsResponse

| Parameter | Type | Description |
|---|---|---|
| values | List<String> | Reference options |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Reference field options have been successfully loaded |
| INVALID_REQUEST | Reference field identifier is not filled in correctly |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_ACCOUNT_INFO | Authenticated user details do not match reference details |
| REFERENCE_VALIDATION_FAILED | |
| REFERENCE_EMPTY_MASTER_VALUE | |
| REFERENCE_INVALID_MASTER_VALUE | |

## 3. Validate references

### URL: /data/references/validation

### Request:

#### ReferenceValidationRequest

| Parameter | Type | Description |
|---|---|---|
| references | List<ReferenceDto> | Specified reference field values |

#### ReferenceDto

| Parameter | Type | Description |
|---|---|---|
| id | UUID | Reference field identifier |
| name | String | Reference field name |
| value | String | Reference value |

### Response:

#### ReferenceValidationResponse

| Parameter | Type | Description |
|---|---|---|
| validation | List<ReferenceValidationInfo> | Reference validation info |

#### ReferenceVallidationInfo

| Parameter | Type | Description |
| --- | --- | --- |
| referenceId | UUID | Reference field identifier |
| name | String | Reference field name |
| mustBeInListPassed | boolean | Indicates if the 'must be in list' check has been passed |
| patternPassed | boolean | Indicates if the pattern check has been passed |
| requiredPassed | boolean | Indicates if the required check has been passed |

## Possible response codes:

| Value | Description |
| --- | --- |
| OK | Specified references are valid |
| INVALID_REQUEST | Reference list is empty |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_ACCOUNT_INFO | Authenticated user details do not match reference details |
| REFERENCE_VALIDATION_FAILED | |
| REFERENCE_EMPTY_MASTER_VALUE | |
| REFERENCE_INVALID_MASTER_VALUE | |

# 15. Airlines

## 1. Load airlines

### URL: /data/airlines

### Request:

#### AirlinesRequest

| Parameter | Type | Description |
|---|---|---|
| searchString | String | Airline search string |

### Response:

#### AirlinesResponse

| Parameter | Type | Description |
|---|---|---|
| airlines | List<AirlineDto> | Found airlines.Maximum number of airlines is defined in the system setting ("Airline search result limit"). |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Airlines have been successfully loaded |

### Request example:

| Request | Response |
|---|---|
| {<br>"searchString":<br>"Aerofl"<br>} | {<br>"status": "OK",<br>"errorMessage": null,<br>"airlines": [<br>{<br>"id": "6157a216-7afb-b476-ffac-e4ca6c62e392",<br>"name": "Aeroflot",<br>"iata": "SU",<br>"icao": "AFL"<br>}<br>]<br>} |

## 2. Load flight details

### URL: /data/flight

### Request:

| Parameter | Type | Description |
|---|---|---|
| **airportId** | UUID | Airport id |
| **flightNumber** | String | Flight number |

| | | |
|---|---|---|
| **flightDate** | Date | Fligth date |

## Response:

| Parameter | Type | Description |
|---|---|---|
| flight | List<FlightInfoDto> | Found airlines |

## FlightInfoDto:

| Parameter | Type | Description |
|---|---|---|
| number | String | Flight number |
| arrivalFrom | String | Arrival from |
| eta | Date | ETA |
| status | FlightStatus | Flight status |
| airline | AirlineDto | Airline |
| airport | CustomerAddressDto | Airport address |
| terminal | CustomerAddressDto | Terminal address |

## FlightStatus:

| Value | Description |
|---|---|
| SCHEDULED("SCHEDULED") | Scheduled |
| ON_TIME("ON_TIME") | On time |
| EN_ROUTE("EN_ROUTE") | En route |
| ARRIVED("ARRIVED") | Arrived |
| DELAYED("DELAYED") | Delayed |
| CANCELLED("CANCELLED") | Cancelled |

## Possible response codes:

| Value | Description |
|---|---|
| OK | Airlines have been successfully loaded |
| INVALID_REQUEST | Request parameter has missed |

# 16. Discounts

There are three types of discounts in Sherlock: loyalty cards, promo codes and vouchers:

## 1. Loyalty cards

### URL: /discount/loyaltyCards

### Request:

Cf. AmendableCustomerDataRequest.

### Response:

#### LoyaltyCardsResponse

| Parameter | Type | Description |
|---|---|---|
| loyaltyCards | List<LoyaltyCardDto> | List of available loyalty cards |

#### LoyaltyCardDto

| Parameter | Type | Description |
|---|---|---|
| customerType | CustomerType | Customer type |
| id | UUID | Loyalty card identifier |
| name | String | Loyalty card name |
| number | String | Loyalty card number |
| discount | String | Formatted discount value |
| defaultCard | boolean | Indicates if the loyalty card should be used as a default one |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Loyalty cards have been successfully loaded |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

### Request example:

| Request | Response |
|---|---|

| | |
|---|---|
| {<br>"customerTypes" :<br>[<br>"RETAIL"<br>]<br>} | {<br>"status": "OK",<br>"errorMessage": null,<br>"loyaltyCards": [<br>{<br>"customerType": "RETAIL",<br>"id": "ce7b4471-6ff3-3c29-3935-747dc76d28a9",<br>"name": "Gold",<br>"number": "11111111",<br>"discount": "25.50%",<br>"defaultCard": true<br>}<br>]<br>} |

## 2. Voucher loading

### URL: /discount/vouchers

### Request:

Cf. AmendableCustomerDataRequest.

### Response:

#### VouchersResponse

| Parameter | Type | Description |
|---|---|---|
| vouchers | List<VoucherDto> | List of available electronic vouchers |
| amendJobId | UUID | Amend job id |

#### VoucherDto

| Parameter | Type | Description |
|---|---|---|
| customer Type | Customer Type | Customer type |
| id | UUID | Electronic voucher identifier |
| number | String | Electronic voucher number |
| discount | String | Formatted discount value |
| discountE xt | String | Formatted discount value in the alternative currency. Populated for booking history requests only, the 'System settings - Use currency conversion' option should be enabled. |
| expiration Date | Date | Electronic voucher expiration date |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Vouchers have been successfully loaded |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |

| | |
|---|---|
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

### Request example:

| Request | Response |
|---|---|
| {<br>"customerTypes" :<br>[<br>"RETAIL"<br>]<br>} | {<br>"status": "OK",<br>"errorMessage": null,<br>"vouchers": [<br>{<br>"customerType": "RETAIL",<br>"id": "a3997c6a-d63b-a1c4-edb5-425f422ff9b5",<br>"discount": "$25",<br>"discountExt":null,<br>"number": "123 456 99"<br><br>}<br>]<br>} |

## 3. Voucher validation

### URL: /discount/voucher/validate

### Request:

#### ValidateVoucherRequest

| Parameter | Type | Description |
|---|---|---|
| **customerType** | CustomerType | Customer type |
| **number** | String | Electronic voucher number |

### Response:

#### ValidateVoucherResponse

| Parameter | Type | Description |
|---|---|---|
| voucher | VoucherDto | Validated electronic voucher information |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Voucher has been successfully validated |
| INVALID_REQUEST | Voucher number is not specified |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

| | |
|---|---|
| VOUCHER_NOT_ACTIVE | Specified voucher is not active |
| INVALID_VOUCHER | Specified voucher is not found |

### Request example:

| Request | Response |
|---|---|
| {<br>"customerType":<br>"RETAIL",<br>"number": "123 456 99"<br>} | {<br>"status": "OK",<br>"errorMessage": null,<br>"voucher": {<br>"customerType": "RETAIL",<br>"id": "a3997c6a-d63b-a1c4-edb5-425f422ff9b5",<br>"discount": "$25",<br>"number": "123 456 99"<br>}<br>} |

## 4. Promo code validation

### URL: /discount/promoCode/validate

### Request:

#### ValidatePromoCodeRequest

| Parameter | Type | Description |
|---|---|---|
| **customerType** | CustomerType | Customer type |
| **code** | String | Promo code |
| **date** | Date | Booking date. Filled in for prebooks. |

### Response:

#### ValidatePromoCodeResponse

| Parameter | Type | Description |
|---|---|---|
| promoCode | PromoCodeDto | Validated promo code |

#### PromoCodeDto

| Parameter | Type | Description |
|---|---|---|
| customer Type | Customer Type | Customer type |
| number | String | Promo code number |
| description | String | Promo code description |
| discount | String | Promo code discount |
| discountE xt | String | Formatted discount value in the alternative currency. Populated for booking history requests only, the 'System settings - Use currency conversion' option should be enabled. |

### Possible response codes:

| Value | Description |
| --- | --- |
| OK | Promo code has been successfully validated |
| INVALID_PROMO_CODE | Promo code is invalid |

## Request example:

| Request | Response |
| --- | --- |
| {<br>"customerType":<br>"RETAIL",<br>"code": "Promo42"<br>} | {<br>"status": "OK",<br>"errorMessage": null,<br>"promoCode": {<br>"number": "Promo42",<br>"description": "Test promo code",<br>"discount": "$35",<br>"discountExt":null<br>}<br>} |
| | {<br>"status": "INVALID_PROMO_CODE",<br>"errorMessage": "Promo code 'Promo42' is not available",<br>"promoCode": null<br>} |

# 17. Contact book

## 1. Load available individuals

### URL: /contactBook/load/available

### Request:

| Parameter | Type | Description |
|---|---|---|
| partOfName | String | Individual name matching pattern |
| customerTypes | List<CustomerType> | Customer types |

### Response:

#### LoadPassengersResponse

| Parameter | Type | Description |
|---|---|---|
| passengers | List<ContactDto> | List of individuals |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Individuals have been successfully loaded |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

## 2. Load available passengers

### URL: /contactBook/load/available/passengers

### Request:

| Parameter | Type | Description |
|---|---|---|
| partOfName | String | Passenger name matching pattern |
| customerTypes | List<CustomerType> | Customer types |

### Response:

#### LoadPassengersResponse

| Parameter | Type | Description |
|---|---|---|
| contacts | List<ContactDto> | List of available passengers |

### Possible response codes:

| Value | Description |
| --- | --- |
| OK | Passengers have been successfully loaded |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

## 3. Load available contacts

### URL: /contactBook/load/available/contacts

### Request:

| Parameter | Type | Description |
| --- | --- | --- |
| partOfName | String | Contact name matching pattern |
| customerTypes | List<CustomerType> | Customer types |

### Response:

#### LoadPassengersResponse

| Parameter | Type | Description |
| --- | --- | --- |
| contacts | List<ContactDto> | List of available contacts |

### Possible response codes:

| Value | Description |
| --- | --- |
| OK | Contacts have been successfully loaded |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

## 4. Load default email

### URL: /contactBook/find/default/email

### Request:

| Parameter | Type | Description |
| --- | --- | --- |
| individualId | UUID | Individual id |

### Response:

| Parameter | Type | Description |
| --- | --- | --- |

| | | |
|---|---|---|
| email | String | Individual email |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Email have been successfully loaded |
| INVALID_REQUEST | If individual id is not specified |

## 5. Load default phone

### URL: /contactBook/find/default/phone

### Request:

| Parameter | Type | Description |
|---|---|---|
| individualId | UUID | Individual id |

### Response:

| Parameter | Type | Description |
|---|---|---|
| phone | String | Individual phone number |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Phone have been successfully loaded |
| INVALID_REQUEST | If individual id is not specified |

## 6. Load default passenger

### URL: /contactBook/find/default/passenger

### Request:

| Parameter | Type | Description |
|---|---|---|
| customerTypes | List<CustomerType> | Customer types |

### Response:

| Parameter | Type | Description |
|---|---|---|
| contact | ContactDto | Default passenger |

### Possible response codes:

| Value | Description |
|---|---|

| | |
|---|---|
| OK | Passenger has been successfully loaded |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

## 7. Load default contact

### URL: /contactBook/find/default/contact

### Request:

| Parameter | Type | Description |
|---|---|---|
| customerTypes | List<CustomerType> | Customer types |

### Response:

| Parameter | Type | Description |
|---|---|---|
| contact | ContactDto | Default contact |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Contact has been successfully loaded |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

## 8. Set default passenger

### URL: /contactBook/set/default/passenger

### Request:

| Parameter | Type | Description |
|---|---|---|
| contact | ContactDto | Passenger to be set |
| customerType | CustomerType | Customer type |

### Response:

BaseResponse

### Possible response codes:

| Value | Description |
| --- | --- |
| OK | Passenger has been successfully set |
| INVALID_REQUEST | Something gone wrong, there are several possibilities to fail. Look in the message at response for details. |

## 9. Set default contact

### URL: /contactBook/set/default/contact

### Request:

| Parameter | Type | Description |
| --- | --- | --- |
| contact | ContactDto | Contact to be set |
| customerType | CustomerType | Customer type |

### Response:

BaseResponse

### Possible response codes:

| Value | Description |
| --- | --- |
| OK | Contact has been successfully set |
| INVALID_REQUEST | Something gone wrong, there are several possibilities to fail. Look in the message at response for details. |

## 10. Load contacts by name

### URL: /contactBook/find/default/contact

### Request:

| Parameter | Type | Description |
| --- | --- | --- |
| searchContact | boolean | if true only contacts will be searched, if false passengers will be searched |
| partOfName | String | Name pattern for contact |
| loadPhoneAndEmail | boolean | True if need to load default phone and email, if false no email and phone will be loaded |

### Response:

| Parameter | Type | Description |
| --- | --- | --- |
| contacts | List<ContactDto> | Found contacts |

### Possible response codes:

| Value | Description |
| --- | --- |
| OK | Contacts have been successfully loaded |

| | |
|---|---|
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| OPERATION_UNSUPPORTED | 'Must be in list' option checked in the account |

# 18. Starting booking creation

To protect users from creating multiple bookings with the same information the client should receive a unique identifier from the server which is used during booking calculation and confirmation in the **requestId** field.
In case the client tries to call a booking calculation or confirmation method with the request identifier which has been already used for another booking, the server returns the 'INCORRECT_REQUEST_IDENTIFIER' response status.

## URL: /data/bookingStarted

## Request:

Empty request.

## Response:

### BookingStartedResponse

| Parameter | Type | Description |
|---|---|---|
| requestId | UUID | Booking request identifier |

## Possible response codes:

| Value | Description |
|---|---|
| OK | Request identifier has been loaded |

## Request example:

| Request | Response |
|---|---|
| | {<br>"status":"OK",<br>"errorMessage":null,<br>"requestId:"9d07992c-15ee-4e9c-a535-6b9ed40d1e23"<br>} |

# 19. Price calculation

Before booking confirmation the client should calculate a price based on the parameters specified below. The calculated price is used later during booking confirmation.
There are two possible service restrictions:

1. A booking date should be within the service availability interval;
2. A calculated price should be less than the maximum available price declared in the service for the selected account.

## URL: /calculate/price

## Request:

### PriceRequest

| Parameter | Type | Description |
|---|---|---|
| **customerType** | CustomerType | Customer type |
| **operationType** | BookingOperationType | Booking operation type |
| **requestId** | UUID | Booking request identifier |
| usedJobId | UUID | Used booking identifier |
| **service** | BookingServiceDto | Selected service |
| **paymentType** | BookingPaymentTypeDto | Selected payment type |
| **asap** | boolean | Indicates if the booking is ASAP |
| date | Date | Booking date |
| **stops** | List<StopDto> | List of booking stops |
| **routeInfo** | RouteInfo | Route info |
| instructions | List<SpecialInstructionDto> | Specified special instructions |
| loyaltyCard | LoyaltyCardDto | Selected loyalty card |
| voucher | VoucherDto | Selected electronic voucher |
| promoCode | String | Selected promo code |
| creditCard | CreditCardDto | Credit card information (can affect the price) |
| payPalAccount | PayPalAccountDto | PayPal account |
| references | List<ReferenceDto> | References |
| contact | ContactDto | Contact |
| passengers | List<ContactDto> | Passengers |
| numberOfPassengers | Integer | Number of passengers |
| smallBagsNumber | Integer | Number of small bags |

## Request example:

**Request**


```
{
  "customerType":"CORPORATE",
  "requestId":"1328d164-23ec-4b45-ba54-2078ce6a7d4e",
  "operationType":"CREATE",
  "usedJobId":null,
  "service":{
    "customerType":"CORPORATE",
    "id":"8cdec226-06cf-45a4-bfe9-61b8d13b5dc6",
    "caption":"BlackCab",
    "description":"1",
    "code":null,
    "delivery":false
  },
  "paymentType":{
    "customerType":"CORPORATE",
    "name":"Invoice",
    "code":"INVOICE"
  },
  "asap":true,
  "date":null,
  "stops":[
    {
      "id":null,
      "address":{
        "addressId":null,
        "caption":"Teatrul Naional Bucureti, Bulevardul Nicolae Bălcescu 2, Bucureti",
        "latitude":44.4365587,
        "longitude":26.1035929,
        "source":"GOOGLE_PLACE",
        "data":"UkV.........................R019",
        "fields":{
          "county":"Municipiul Bucureti",
          "street":"Bulevardul Nicolae Bălcescu",
          "district":"Sector 1",
          "country":"România",
          "city":"Bucureti",
          "postcode":"010051",
          "buildingName":"2",
          "neighborhood":null,
          "companyName":"Teatrul Naional Bucureti"
        },
        "restrictions":null
      },
      "notes":null,
      "arrivalDetails":null,
      "contact":null
    },
    {
      "id":null,
      "address":{
        "addressId":"0de18960-d04b-e4ff-
```


**Response**


```
{
"status": "OK",
"errorMessage": null,
"price": {
"net": {
"value": 17.00,
"formattedValue": "€17.00",
"code": "EUR",
"currencyFormat": "€%s"
},
"taxTotal": {
"value": 0.00,
"formattedValue": "€0.00",
"code": "EUR",
"currencyFormat": "€%s"
},
"gross": {
"value": 17.00,
"formattedValue": "€17.00",
"code": "EUR",
"currencyFormat": "€%s"
},
"discount": {
"value": 0.00,
"formattedValue": "€0.00",
"code": "EUR",
"currencyFormat": "€%s"
},
"details": {
"Base Amount Without Tax": "€17.00",
"Client Total Price": "€17.00"
},
"priceDetails": [
{
"name": "Base Amount Without Tax",
"amount": {
"value": 17.00,
"formattedValue": "€17.00",
"code": "EUR",
"currencyFormat": "€%s"
},
"type": "BASE",

},
{
"name": "Client Total Price",
"amount": {
"value": 17.00,
"formattedValue":
```

| | | |
|---|---|---|
| bigBagsNumber | Integer | Number of big bags |

## Response:

### PriceResponse

| Parameter | Type | Description |
|---|---|---|
| price | PriceDto | Calculated price |
| extPrice | PriceDto | Calculated price in the alternative currency |
| discount | DiscountInfoDto | Discount information |
| extDiscount | DiscountInfoDto | Discount information converted to the alternative currency |
| asDirectedMinutesRounding | AsDirectedMinutesRoundingInfoDto | Rounding details |

### PriceDto

| Parameter | Type | Description |
|---|---|---|
| net | PriceField | Total price |
| taxTotal | PriceField | Total tax applied |
| gross | PriceField | Total price with a tax applied |
| discount | PriceField | Discount applied |
| details | Map<String, String> | Detailed price explanation |
| priceDetails | List<PriceDetailsDto> | Detailed price explanation |
| prefix | String | Price prefix |

### PriceField

| Parameter | Type | Description |
|---|---|---|
| value | BigDecimal | Price value |
| formattedValue | String | Formatted price value shown to the client |
| code | String | Currency code |
| currencyFormat | String | Currency format |

### DiscountInfoDto

| Parameter | Type | Description |
|---|---|---|
| appliedDiscounts | List<DiscountDetailsDto> | Applied discount information |
| excludedDiscounts | List<DiscountDetailsDto> | Excluded discount information |


3d06-b87e5026441b",
"caption":"Pasajul Nicolae Bălcescu, Sector 1, Bucureti",
"latitude":44.4358564,
"longitude":26.1027515,
"source":"ADDRESS_BOOK",
"data":null,
"fields":null,
"restrictions":null
},
"notes":null,
"arrivalDetails":null,
"contact":null
}
],
"routeInfo":{
"waitAndReturn":false,
"destinationUnknown":false,
"asDirected":false,
"asDirectedMinutes":0
},
"instructions":[
{
"typeId":"fa2544e3-4ea2-55ba-f4ae-352245cdd9de",
"code":"gor",
"name":"gor",
"value":"11",
"set":true
}
],
"voucher":null,
"loyaltyCard":null,
"promoCode":null,
"creditCard":null
}

"€17.00",
"code": "EUR",
"currencyFormat": "€%s"
},
"type": "TOTAL"
}
]
},
"extPrice": {
"net": {
"value": 3.7400,
"formattedValue": "€3.74",
"code": "EUR",
"currencyFormat": "€%s"
},
"taxTotal": {
"value": 0.0000,
"formattedValue": "€0.00",
"code": "EUR",
"currencyFormat": "€%s"
},
"gross": {
"value": 3.7400,
"formattedValue": "€3.74",
"code": "EUR",
"currencyFormat": "€%s"
},
"discount": {
"value": 0.0000,
"formattedValue": "€0.00",
"code": "EUR",
"currencyFormat": "€%s"
},
"details": {
"Base Amount Without Tax": "€3.74",
"Client Total Price": "€3.74"
},
"prefix": "destination unknown",
"priceDetails": [
{
"name": "Base Amount Without Tax",
"amount": {
"value": 3.74,
"formattedValue": "€3.74",
"code": "EUR",
"currencyFormat": "€%s"
},
"type": "BASE"
},
{
"name": "Client Total Price",
"amount": {
"value": 3.74,
"formattedValue": "€3.74",

### DiscountDetailsDto

| Parameter | Type | Description |
|---|---|---|
| promoCode | PromoCodeDto | Promo code, filled in if applied |
| loyaltyCard | LoyaltyCardDto | Loyalty card, filled in if applied |
| voucher | VoucherDto | Electronic voucher, filled in if applied |
| discount | PriceField | Discount value |
| discountedTax | PriceField | Tax value for the applied discount |
| reason | String | Reason for not applying the discount |

### PriceDetailsDto

| Parameter | Type | Description |
|---|---|---|
| name | String | Price line item name |
| amount | PriceField | Price line item amount |
| type | PriceDetailType | Price detail type |
| code | String | Code of job extra. For PriceDetailsType.EXTRA |

### PriceDetailsType

| Value | Description |
|---|---|
| TOTAL("TOTAL") | Total price value |
| BASE("BASE") | Base price value |
| EXTRA_STOPS("EXTRA_STOPS") | Extra stops price value |
| TBS_SERVICE_FEE("TBS_SERVICE_FEE") | Fee for Call Center usage |
| WAITING_TIME("WAITING_TIME") | Waiting time price value |
| ADMIN_FEE("ADMIN_FEE") | Admin fee price value |
| ADMIN_FEE_TAX("ADMIN_FEE_TAX") | Admin fee tax value |
| MODIFIER("MODIFIER") | Modifier value |
| EXTRA("EXTRA") | Extra price value |
| TAX("TAX") | Tax value |
| DISCOUNT("DISCOUNT") | Discount value |
| ROUNDING_ADJUSTMENT ("ROUNDING_ADJUSTMENT") | Rounding adjustment value |

### AsDirectedMinutesRoundingInfoDto


"code": "EUR",
"currencyFormat": "€%s"
},
"type": "TOTAL"
}
]
},
"discount": {
"appliedDiscounts": [

],
"excludedDiscounts": [

]
},
"extDiscount": {
"appliedDiscounts": [

],
"excludedDiscounts": [

]
},
"asDirectedMinutesRounding": null
}

| Parameter | Type | Description |
| --- | --- | --- |
| asDirectedRoundedMinutes | Integer | New 'As directed' minutes value (rounded requested value) |
| caption | String | Localised message about the journey time which has been rounded |

## Possible response codes:

| Value | Description |
| --- | --- |
| OK | Price has been successfully calculated |
| INVALID_REQUEST | Request data is not filled in properly |
| INVALID_CUSTOMER_TYPE | Customer type is not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| INCORRECT_REQUEST_IDENTIFIER | Specified request identifier has been already used in another booking |
| SERVICE_RESTRICTION_FAILED | At least one of service restrictions is not matched |
| PICKUP_CIRCUIT_LOCKED | Specified address area is locked for ASAP bookings |
| INVALID_OPERATIONAL_AREA | Specified service is not available in the selected pickup circuit |
| PAYMENT_TYPE_NOT_SUPPORTED | Selected payment type is not supported |
| INVALID_VOUCHER | Specified voucher is not found |
| VOUCHER_NOT_ACTIVE | Specified voucher is not active |
| INVALID_LOYALTY_CARD | Loyalty card is invalid |
| INVALID_PROMO_CODE | Promo code is invalid |
| INSTRUCTION_VALIDATION_FAILED | Specified special instruction values are not valid |

| | |
|---|---|
| INVALID_ADDRESS_DATA | Unable to assemble a stop address from the specified data |
| INVALID_COORDINATES | Pickup stop coordinates are in the invalid area |
| INVALID_ROUTE_INFO | Attempted to create a 'Wait and return', 'Destination unknown' or 'As directed' booking while it is not supported by the selected service |
| PRICE_NOT_CALCULATED | Price has not been calculated for some internal reasons |
| BOOKING_NOT_AMENDABLE | Booking cannot be amended |
| BOOKING_NOT_REPEATABLE | Booking cannot be repeated |
| DATA_NOT_FOUND | Unable to find booking related data by the identifier specified in the request |
| INVALID_ACCOUNT_DATA | Specified service or payment type is not available to the user |
| OPERATION_UNAVAILABLE | Automatic lock service occurred |
| ASAP_ONLY_SERVICE | Only ASAP bookings are available for the service |

# 20. Discount condition validation

This method should be used by the client to check whether a discount specified by the user satisfies booking discount conditions declared in Sherlock. Discount condition validation is performed without price calculation in order to decrease the request processing time.

## URL: /discount/validate

## Request:

### ValidateDiscountRequest

| Parameter | Type | Description |
|---|---|---|
| **customerType** | CustomerType | Customer type |
| **operationType** | BookingOperationType | Booking operation type |
| requestId | UUID | Booking request identifier |
| usedJobId | UUID | Used booking identifier |
| service | BookingServiceDto | Selected service |
| paymentType | BookingPaymentTypeDto | Selected payment type |
| asap | boolean | Indicates if the booking is ASAP |
| date | Date | Booking date |
| stops | List<StopDto> | List of booking stops |
| routeInfo | RouteInfo | Route info |
| instructions | List<SpecialInstructionDto> | Specified special instructions |
| loyaltyCard | LoyaltyCardDto | Selected loyalty card |
| voucher | VoucherDto | Selected electronic voucher |
| promoCode | String | Selected promo code |
| creditCard | CreditCardDto | Credit card information |
| payPalAccount | PayPalAccountDto | PayPal account |
| references | List<ReferenceDto> | References |
| contact | ContactDto | Contact |
| passengers | List<ContactDto> | Passengers |
| numberOfPassengers | Integer | Number of passengers |
| smallBagsNumber | Integer | Number of small bags |
| bigBagsNumber | Integer | Number of big bags |

### PayPalAccountDto

| Parameter | Type | Description |
|---|---|---|
| id | UUID | Account id |
| email | String | Email |
| name | String | Name |

## Response:

### ValidateDiscountResponse

| Parameter | Type | Description |
|---|---|---|
| discount | DiscountInfoDto | Discount information |
| extDiscount | DiscountInfoDto | Discount information converted to the alternative currency |

## Possible response codes:

| Value | Description |
|---|---|
| OK | Price has been successfully calculated |
| INVALID_REQUEST | Request data is not filled in properly |
| INVALID_CUSTOMER_TYPE | Customer type is not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| INCORRECT_REQUEST_IDENTIFIER | Specified request identifier has been already used in another booking |
| SERVICE_RESTRICTION_FAILED | At least one of service restrictions is not matched |
| INVALID_OPERATIONAL_AREA | Specified service is not available in the selected pickup circuit |
| PAYMENT_TYPE_NOT_SUPPORTED | Selected payment type is not supported |
| INVALID_VOUCHER | Specified voucher is not found |
| VOUCHER_NOT_ACTIVE | Specified voucher is not active |
| INVALID_LOYALTY_CARD | Loyalty card is invalid |
| INVALID_PROMO_CODE | Promo code is invalid |
| INSTRUCTION_VALIDATION_FAILED | Specified special instruction values are invalid |
| INVALID_ADDRESS_DATA | Unable to assemble a stop address from the specified data |
| INVALID_COORDINATES | Pickup stop coordinates are in the invalid area |
| INVALID_ROUTE_INFO | Attempted to create a 'Wait and return', 'Destination unknown' or 'As directed' booking while it is not supported by the selected service |
| PRICE_NOT_CALCULATED | Price has not been calculated for some internal reasons |
| BOOKING_NOT_AMENDABLE | Booking cannot be amended |
| BOOKING_NOT_REPEATABLE | Booking cannot be repeated |
| DATA_NOT_FOUND | Unable to find booking related data by the identifier specified in the request |
| INVALID_ACCOUNT_DATA | Specified service or payment type is not available to the user |

# 21. Journey time calculation

## URL: /calculate/journeyTime

## Request:

### JourneyTimeRequest

| Parameter | Type | Description |
|---|---|---|
| **customerType** | CustomerType | Customer type |
| **operationType** | BookingOperationType | Booking operation type |
| **requestId** | UUID | Booking request identifier |
| usedJobId | UUID | Used booking identifier |
| **service** | BookingServiceDto | Selected service |
| **asap** | boolean | Indicates if the booking is ASAP |
| date | Date | Booking date |
| **stops** | List<StopDto> | List of booking stops |
| **routeInfo** | RouteInfo | Route info |

## Response:

### JourneyTimeResponse

| Parameter | Type | Description |
|---|---|---|
| duration | Integer | Journey duration, in minutes |
| durationInTraffic | Integer | Journey duration with traffic taken into consideration, in minutes |

## Possible response codes:

| Value | Description |
|---|---|
| OK | Journey time has been successfully calculated |
| INVALID_REQUEST | Request data is not filled in properly |
| INVALID_CUSTOMER_TYPE | Customer type is not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| INCORRECT_REQUEST_IDENTIFIER | Specified request identifier has been already used in another booking |
| INVALID_OPERATIONAL_AREA | Specified service is not available in the selected pickup circuit |
| INVALID_ADDRESS_DATA | Unable to assemble a stop address from the specified data |
| INVALID_COORDINATES | Pickup stop coordinates are in the invalid area |

| | |
|---|---|
| INVALID_ROUTE_INFO | Attempted to create a 'Wait and return', 'Destination unknown' or 'As directed' booking while it is not supported by the selected service |
| BOOKING_NOT_AMENDABLE | Booking cannot be amended |
| BOOKING_NOT_REPEATABLE | Booking cannot be repeated |
| DATA_NOT_FOUND | Unable to find booking related data by the identifier specified in the request |
| ASAP_ONLY_SERVICE | Only ASAP bookings are available for the service |

## Request example:

| Request | Response |
|---|---|
| {<br>"customerType": "CORPORATE",<br>"operationType": "CREATE",<br>"usedJobId": null,<br>"stops": [<br>{<br>"id":"1259c4ff-076d-4959-311a-385796490daf",<br>"address":{<br>"addressId":"1308124c-81af-1db4-b88e-7754336518fe",<br>"caption":"91 Rue de la Verrerie, 75004 Paris",<br>"latitude":48.8586556,<br>"longitude":2.3507153,<br>"source":"AB",<br>"data":null<br>},<br>"notes":null,<br>"arrivalDetails":null<br>},<br>{<br>"id":null,<br>"address":{<br>"addressId":null,<br>"caption":"8 Rue Saint-Bon, 75004 Paris",<br>"latitude":48.8584055,<br>"longitude":2.35076,<br>"source":"GG",<br>"data":"QURSe0ZMRDF…IDc1MDA0IFBhcmlzfX0=",<br>"fields":null<br>},<br>"notes":null,<br>"arrivalDetails":null<br>}<br>],<br>"asap": true,<br>"date": "29/06/2015 12:00",<br>"routeInfo": {<br>"waitAndReturn": false,<br>"destinationUnknown": false,<br>"asDirected": false,<br>"asDirectedMinutes": 0<br>}<br>} | {<br>"status": "OK",<br>"errorMessage":<br>"string",<br>"duration": 50,<br>"durationInTraffic": 70<br>} |

# 22. Prebook limit checking

This check should be performed for non-ASAP bookings only, after a user selects a service, a booking date and a pickup stop.

## URL: /calculate/prebookLimit

For each set {Service, Booking Date, Pickup Stop} the server checks if the limit of available bookings is exceeded, and if it happens the server sends available booking dates to the client – the nearest earlier and the nearest later dates.

## Request:

### PrebookLimitRequest

| Parameter | Type | Description |
|---|---|---|
| **customerType** | CustomerType | Customer type |
| **operationType** | BookingOperationType | Booking operation type |
| **requestId** | UUID | Booking request identifier |
| usedJobId | UUID | Used booking identifier |
| **service** | BookingServiceDto | Selected service |
| **date** | Date | Booking date |
| **pickupStop** | StopDto | Booking pickup stop |

## Response:

### PrebookLimitResponse

| Parameter | Type | Description |
|---|---|---|
| nearestEarlier | Date | Nearest earlier available booking date |
| nearestLater | Date | Nearest later available booking date |

## Possible response codes:

| Value | Description |
|---|---|
| OK | Booking is available for the specified date |
| INVALID_REQUEST | Request data is not filled in properly |
| INVALID_CUSTOMER_TYPE | Customer type is not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| INCORRECT_REQUEST_IDENTIFIER | Specified request identifier has been already used in another booking |
| INVALID_OPERATIONAL_AREA | Specified service is not available in the selected pickup circuit |
| INVALID_ADDRESS_DATA | Unable to assemble a stop address from the specified data |
| INVALID_COORDINATES | Pickup stop coordinates are in the invalid area |

| | |
|---|---|
| INVALID_ROUTE_INFO | Attempted to create a 'Wait and return', 'Destination unknown' or 'As directed' booking while it is not supported by the selected service |
| TOO_EARLY_FOR_PREBOOK | Too early date for prebook creation |
| TOO_LATE_FOR_PREBOOK | Too late date for prebook creation |
| FULLY_BOOKED | Limit of available bookings has been exceeded |
| PREBOOK_NOT_AVAILABLE | Prebooks are not available |
| BOOKING_NOT_AMENDABLE | Booking cannot be amended |
| BOOKING_NOT_REPEATABLE | Booking cannot be repeated |
| INVALID_ACCOUNT_DATA | Specified service is not available to the user |
| DATA_NOT_FOUND | Unable to find booking related data by the identifier specified in the request |
| ASAP_ONLY_SERVICE | Only ASAP bookings are available for the service |

## Request example:

| Request | Response |
|---|---|
| {<br>"customerType":"CORPORATE",<br>"operationType":"REPEAT",<br>"usedJobId":"c816f603-87ea-653f-9309-eaf19748c16c",<br>"service":{<br>"customerType":"CORPORATE",<br>"id":"65bff4bc-6fb8-4a1f-a31e-aa2f0ff36bdc",<br>"caption":"Peugeot 508",<br>"description":"Peugeot 508"<br>},<br>"date":"07/07/2015 13:27",<br>"pickupStop":{<br>"id":"1259c4ff-076d-4959-311a-385796490daf",<br>"address":{<br>"addressId":"1308124c-81af-1db4-b88e-7754336518fe",<br>"caption":"91 Rue de la Verrerie, 75004 Paris",<br>"latitude":48.8586556,<br>"longitude":2.3507153,<br>"source":"RP",<br>"data":null<br>},<br>"notes":null,<br>"arrivalDetails":null<br>}<br>} | {<br>"status":"OK",<br>"errorMessage":null,<br>"nearestEarlier":null,<br>"nearestLater":null<br>} |
| | {<br>"status":"TOO_EARLY_FOR_PREBOOK",<br>"errorMessage":"It's too early for booking. It can be made no earlier than 07/07/2015 13:40",<br>"nearestEarlier":"07/07/2015 13:40",<br>"nearestLater":null,<br>} |

| | |
|---|---|
| | {<br>"status":"FULLY_BOOKED",<br>"errorMessage":"We are unable to accept a booking for this date, as we are fully booked. You can choose between 07/07/2015 13:40 and 07/07/2015 13:55",<br>"nearestEarlier":"07/07/2015 13:40",<br>"nearestLater":"07/07/2015 13:55"<br>} |

## URL: /calculate/prebookLimit/range

For each set {Service, Booking Date, Pickup Stop} the server checks if the limit of available bookings is exceeded, and if it happens the server sends to the client a range of available booking dates – the nearest available date and the latest available date.

## Request:

### PrebookLimitRequest

| Parameter | Type | Description |
|---|---|---|
| **customerType** | CustomerType | Customer type |
| **operationType** | BookingOperationType | Booking operation type |
| **requestId** | UUID | Booking request identifier |
| usedJobId | UUID | Used booking identifier |
| **service** | BookingServiceDto | Selected service |
| **date** | Date | Booking date |
| **pickupStop** | StopDto | Booking pickup stop |

## Response:

### PrebookLimitRangeResponse

| Parameter | Type | Description |
|---|---|---|
| `jobDateRangeMin` | Date | Nearest available booking date |
| `jobDateRangeMax` | Date | Latest available booking date |

## Possible response codes:

| Value | Description |
|---|---|
| OK | Booking is available for the specified date |
| INVALID_REQUEST | Request data is not filled in properly |
| INVALID_CUSTOMER_TYPE | Customer type is not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| INCORRECT_REQUEST_IDENTIFIER | Specified request identifier has been already used in another booking |

| | |
|---|---|
| INVALID_OPERATIONAL_AREA | Specified service is not available in the selected pickup circuit |
| INVALID_ADDRESS_DATA | Unable to assemble a stop address from the specified data |
| INVALID_COORDINATES | Pickup stop coordinates are in the invalid area |
| INVALID_ROUTE_INFO | Attempted to create a 'Wait and return', 'Destination unknown' or 'As directed' booking while it is not supported by the selected service |
| BOOKING_NOT_AMENDABLE | Booking cannot be amended |
| BOOKING_NOT_REPEATABLE | Booking cannot be repeated |
| INVALID_ACCOUNT_DATA | Specified service is not available to the user |
| DATA_NOT_FOUND | Unable to find booking related data by the identifier specified in the request |
| ASAP_ONLY_SERVICE | Only ASAP bookings are available for the service |

# 23. Delay calculation

A delay should be calculated for ASAP bookings only, before a booking is confirmed. The delay is calculated in minutes. The delay is shown to the client and it defines time for a taxi to arrive to a pickup point. The delay depends on the selected service and pickup address.
In case when the selected service is locked for the pickup circuit, the server returns the 'PICKUP_CIRCUIT_LOCKED' status and an Offer Service (if specified).

## 1. ASAP delay calculation

### URL: /calculate/delay

### Request:

#### DelayRequest

| Parameter | Type | Description |
|---|---|---|
| **customerType** | CustomerType | Customer type |
| **operationType** | BookingOperationType | Booking operation type |
| **requestId** | UUID | Booking request identifier |
| usedJobId | UUID | Used booking identifier |
| **pickupStop** | StopDto | Pickup stop |
| **service** | BookingServiceDto | Selected service |
| instructions | List<SpecialInstructionDto> | Specified special instructions |
| paymentType | String | Payment type code |

### Response:

#### DelayResponse

| Parameter | Type | Description |
|---|---|---|
| delay | Integer | Calculated delay value |
| offerService | ServiceDto | Offer Service suggested in case the pickup circuit is locked |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Delay has been successfully calculated |
| INVALID_REQUEST | Request data is not filled in properly |
| INVALID_CUSTOMER_TYPE | Customer type is not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| INCORRECT_REQUEST_IDENTIFIER | Specified request identifier has been already used in another booking |

| | |
|---|---|
| INVALID_ADDRESS_DATA | Unable to assemble a stop address from the specified data |
| INVALID_COORDINATES | Pickup stop coordinates are in the invalid area |
| INVALID_ROUTE_INFO | Attempted to create a 'Wait and return', 'Destination unknown' or 'As directed' booking while it is not supported by the selected service |
| INSTRUCTION_VALIDATION_FAILED | Specified special instruction values are invalid |
| PICKUP_CIRCUIT_LOCKED | Specified address area is locked for ASAP bookings |
| INVALID_OPERATIONAL_AREA | Specified service is not available in the selected pickup circuit |
| BOOKING_NOT_AMENDABLE | Booking cannot be amended |
| BOOKING_NOT_REPEATABLE | Booking cannot be repeated |
| INVALID_ACCOUNT_DATA | Specified service is not available to the user |
| DATA_NOT_FOUND | Unable to find booking related data by the identifier specified in the request |
| OPERATION_UNAVAILABLE | Automatic lock service occurred |

### Request example:

| Request | Response |
|---|---|
| {<br>"customerType":"CORPORATE",<br>"operationType":"CREATE",<br>"usedJobId":null,<br>"pickupStop":{<br>"id":null,<br>"address":{<br>"addressId":null,<br>"caption":"91 Rue de la Verrerie, 75004 Paris",<br>"latitude":48.8586556,<br>"longitude":2.3507153,<br>"source":"GG",<br>"data":"QURSe…=",<br>},<br>"notes":null,<br>"arrivalDetails":null<br>},<br>"service":{<br>"customerType":"CORPORATE",<br>"id":"6153a7d3-d400-4e81-97a2-3f4caa13aa6a",<br>"caption":"Mercedes Viano",<br>"description":"Mercedes Viano"<br>},<br>"instructions":null ,<br><br>"paymentType" : "CASH"<br>} | {<br>"status":"OK",<br>"errorMessage":null,<br>"nearestEarlier":null,<br>"nearestLater":null<br>} |

## 2. Arrival delay calculation

An arrival delay calculated for bookings with a pickup from airports and train stations differs from usual delay calculation in additional fields returned in the response: the calculated job date and the ASAP flag, which must be used by the client in booking creation.

### URL: /calculate/arrivalDelay

### Request:

Cf. DelayRequest.

## Response:

### ArrivalDelayResponse

| Parameter | Type | Description |
|---|---|---|
| delay | Integer | Calculated delay value |
| date | Date | Calculated job date |
| asap | boolean | Indicates if the booking should be considered as ASAP |
| offerService | ServiceDto | Offer Service suggested in case the pickup circuit is locked |

## Possible response codes:

| Value | Description |
|---|---|
| OK | Delay has been successfully calculated |
| INVALID_REQUEST | Request data is not filled in properly |
| INVALID_CUSTOMER_TYPE | Customer type is not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| INVALID_ADDRESS_DATA | Unable to assemble a stop address from the specified data |
| INVALID_COORDINATES | Pickup stop coordinates are in the invalid area |
| INVALID_ROUTE_INFO | Attempted to create a 'Wait and return', 'Destination unknown' or 'As directed' booking while it is not supported by the selected service |
| INSTRUCTION_VALIDATION_FAILED | Specified special instruction values are invalid |
| PICKUP_CIRCUIT_LOCKED | Specified address area is locked for ASAP bookings |
| BOOKING_NOT_AMENDABLE | Booking cannot be amended |
| BOOKING_NOT_REPEATABLE | Booking cannot be repeated |
| INVALID_ACCOUNT_DATA | Specified service is not available to the user |
| OPERATION_UNAVAILABLE | Automatic lock service occurred |
| ASAP_ONLY_SERVICE | Only ASAP bookings are available for the service |

## Request example:

| Request | Response |
|---|---|

| | |
|---|---|
| {<br>"customerType":"RETAIL",<br>"operationType":"CREATE",<br>"usedJobId":null,<br>"pickupStop":{<br>"id":null,<br>"address":{<br>"addressId":"f2a35342-4ede-4af8-9019-27f9659f90f0",<br>"caption":"Beauvais-Tillé, Terminal 2",<br>"latitude":49.4560867,<br>"longitude":2.1100769,<br>"source":"AT",<br>"data":null<br>},<br>"notes":null,<br>"arrivalDetails":{<br>"arrived":true,<br>"pickupTimeDelay":5,<br>"arrivalTime":"29/06/2015 11:47",<br>"arrivalNumber":"fghy",<br>"arrivalFrom":"thbct",<br>"meetingPoint":null<br>}<br>},<br>"service":{<br>"customerType":"RETAIL",<br>"id":"65bff4bc-6fb8-4a1f-a31e-aa2f0ff36bdc",<br>"caption":"Peugeot 508",<br>"description":"Peugeot 508"<br>},<br>"instructions":null ,<br><br>"paymentType":"CASH"<br>} | {<br>"status":"OK",<br>"errorMessage":null,<br>"delay":80,<br>"offerService":null,<br>"date":"29/06/2015 13:07",<br>"asap":true<br>} |
| | {<br>"status":"PICKUP_CIRCUIT_LOCKED",<br>"errorMessage":"We are currently fully booked in selected area. Please accept our apologies",<br>"delay":null,<br>"offerService":null,<br>"date":null,<br>"asap":null<br>} |
| | {<br>"status":"PICKUP_CIRCUIT_LOCKED",<br>"errorMessage":"We are currently fully booked in selected area. Please accept our apologies",<br>"delay":null,<br>"offerService":{<br>"customerType":"RETAIL",<br>"id":"9a571f30-9ee6-4f07-ace2-d1f0c1fd77ca",<br>"caption":"Offer Service",<br>"description":"Private Offer"<br>},<br>"date":null,<br>"asap":null<br>} |

# 24. Job offer workflow

Job offer workflow is the process when the client sends booking details and the server immediately starts looking for a driver to allocate to the specified booking. After a driver is found the client can accept or decline the offered driver using the corresponding methods.

The typical workflow comprises the following steps:

1. The client sends an /offer/make request, gets an offerId in the response;
2. The client periodically requests the /offer/check while the system is looking for a driver. When a driver is found, the server will return 'DRIVER_FOUND';
3. The client confirms a job using /job/book, the requestId should be equal to the offerId;
4. To get updates about the job status and driver position, the client uses /booking/load/details

## 1. Make an offer

In case of successful offer processing the server returns an identifier of the created offer used by the client in the following methods.

### URL: /offer/make

### Request:

#### JobOfferRequest

| Parameter | Type | Description |
|---|---|---|
| **customerType** | CustomerType | Customer type |
| **operationType** | BookingOperationType | Booking operation type |
| **requestId** | UUID | Booking request identifier |
| usedJobId | UUID | Used booking identifier, filled in case of the AMEND or REPEAT operation type |
| **requestId** | UUID | Request identifier |
| **service** | BookingServiceDto | Selected service |
| **paymentType** | BookingPaymentTypeDto | Selected payment type |
| **asap** | boolean | Indicates if the booking is ASAP. Should be true for job offer workflow |
| date | Date | Booking date |
| references | List<ReferenceDto> | Reference list |
| **stops** | List<StopDto> | List of booking stops |
| **routeInfo** | RouteInfo | Route info |
| **price** | PriceDto | Calculated price |
| **contact** | ContactDto | Booking contact |
| passengers | List<ContactDto> | Booking passengers |
| instructions | List<SpecialInstructionDto> | Specified special instructions |
| loyaltyCard | LoyaltyCardDto | Selected loyalty card |
| voucher | VoucherDto | Selected electronic voucher |
| promoCode | String | Selected promo code |
| notes | String | Booking notes |

### Response:

### JobOfferResponse

| Parameter | Type | Description |
|---|---|---|
| offerId | UUID | Offer identifier |

## Possible response codes:

| Value | Description |
|---|---|
| OK | Delay has been successfully calculated |
| INVALID_REQUEST | Request data is not filled properly |
| INVALID_CUSTOMER_TYPE | Customer type is not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| INCORRECT_REQUEST_IDENTIFIER | Specified request identifier has been already used in another booking |
| INVALID_ADDRESS_DATA | Unable to assemble a stop address from the specified data |
| INVALID_COORDINATES | Pickup stop coordinates are in the invalid area |
| INVALID_ROUTE_INFO | Attempted to create a 'Wait and return', 'Destination unknown' or 'As directed' booking while it is not supported by the selected service |
| INSTRUCTION_VALIDATION_FAILED | Specified special instruction values are invalid |
| PICKUP_CIRCUIT_LOCKED | Specified address area is locked for ASAP bookings |
| INVALID_OPERATIONAL_AREA | Specified service is not available in the selected pickup circuit |
| PAYMENT_TYPE_NOT_SUPPORTED | Selected payment type is not supported |
| INVALID_VOUCHER | Specified voucher is not found |
| VOUCHER_NOT_ACTIVE | Specified voucher is not active |
| INVALID_LOYALTY_CARD | Loyalty card is invalid |
| INVALID_PROMO_CODE | Promo code is invalid |
| TOO_EARLY_FOR_PREBOOK | Too early date for prebook creation |
| TOO_LATE_FOR_PREBOOK | Too late date for prebook creation |
| FULLY_BOOKED | Limit of available bookings has been exceeded |
| PREBOOK_NOT_AVAILABLE | Prebooks are not available |
| PRICE_NOT_CALCULATED | Price has not been calculated for some internal reasons |
| PRICE_CHANGED | Calculated price and the specified one are different |
| INVALID_ACCOUNT_DATA | Specified service, payment type or passenger are not available to the user |
| DATA_NOT_FOUND | Unable to find booking related data by the identifier specified in the request |
| ASAP_ONLY_SERVICE | Only ASAP bookings are available for the service |

### Request example:

| Request | Response |
|---|---|
| | |

## 2. Check an offer state

The client checks an offer state with an interval specified in JobOfferSettings until the offer status becomes 'IN_PROGRESS'. With the 'DRIVER_FOUND' status the client receives the offered driver details that should be accepted or declined by the user. To accept the offered driver the client sends a booking confirmation request with an offer identifier as the request identifier.

### URL: /offer/check

### Request:

#### JobOfferCheckRequest

| Parameter | Type | Description |
|---|---|---|
| **customerType** | CustomerType | Customer type |
| **offerId** | UUID | Offer identifier recieved in the /offer/make request |

### Response:

#### JobOfferCheckResponse

| Parameter | Type | Description |
|---|---|---|
| offerStatus | JobOfferStatusDto | Offer status |

#### JobOfferStatusDto

| Parameter | Type | Description |
|---|---|---|
| state | JobOfferState | Offer state |
| confirmationDate | Date | Offer confirmation date |
| driver | DriverDto | Offered driver |
| delay | Integer | Calculated offer delay |

#### JobOfferState

| Value | Description |
|---|---|
| UNKNOWN("UNKNOWN") | Offer is not found |
| IN_PROGRESS("IN_PROGRESS") | Server is still looking for a driver |
| DRIVER_FOUND("DRIVER_FOUND") | Driver has been found |
| DRIVER_NOT_FOUND("DRIVER_NOT_FOUND") | Driver is not found |

#### DriverDto

| Parameter | Type | Description |
|---|---|---|
| name | String | Driver name |

| | | |
|---|---|---|
| phone | String | Driver phone number or driver switch number |
| callsign | String | Driver callsign |
| carRegNumber | String | Vehicle registration number |
| carModel | String | Vehicle model |
| vehicleManufacturer | String | Vehicle manufacturer |
| vehicleColor | String | Vehicle colour |
| latitude | Double | Driver latitude |
| longitude | Double | Driver longitude |
| photoId | UUID | Driver photo identifier |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Offer has been successfully checked |
| INVALID_REQUEST | Offer identifier is not specified |
| INVALID_CUSTOMER_TYPE | Customer type is not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

### Request example:

| Request | Response |
|---|---|
| {<br>"customerType": "CORPORATE",<br>"offerId": "00191107-7192-3bb6-f111-c68cf7fc7231"<br>} | {<br>"status": "OK",<br>"errorMessage": "string",<br>"offerStatus": {<br>"state": "DRIVER_FOUND",<br>"confirmationDate": "01/01/2015 11:11",<br>"driver": {<br>"name": "Example Driver",<br>"phone": "+700000000",<br>"carRegNumber": "a1111b",<br>"carModel": "Test model",<br>"vehicleManufacturer": "exampleCar",<br>"latitude": 48.886,<br>"longitude": 2.312389,<br>"callsign": "ED1",<br>"vehicleColor": "black"<br>},<br>"delay": 15<br>}<br>} |

## 3. Cancel an offer

To decline an offered driver or to interrupt the driver searching process the client sends an offer cancellation request with an offer identifier specified.

## URL: /offer/cancel

## Request:

### JobOfferCancelRequest

| Parameter | Type | Description |
|---|---|---|
| **customerType** | CustomerType | Customer type |
| **offerId** | UUID | Offer identifier |

## Response:

Cf. BaseResponse.

## Possible response codes:

| Value | Description |
|---|---|
| OK | Offer has been successfully cancelled |
| INVALID_REQUEST | Offer identifier is not specified |
| INVALID_CUSTOMER_TYPE | Customer type is not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

## Request example:

| Request | Response |
|---|---|
| {<br>"customerType": "CORPORATE",<br>"offerId": "0019ad07-7c92-3bb6-f183-c68cf7fc7231"<br>} | {<br>"status": "OK",<br>"errorMessage": null<br>} |

# 25. Credit card management

A user has an ability to pay by a credit card (CC). This ability is specified in the payment type selected by the user (cf. creditCardsAvailable flag of Payment TypeDto). Settings for credit card payments are sent to the client in the CreditCardSettings object. There are several types of CC payment workflow declared in the CreditCardType enum (cf. above):

1. PLAIN – the application sends plain card details to the server;
2. CSE – the application sends encrypted card details to the server;
3. EPP_NEW – details of new credit cards are specified on the 3D-party web site;
4. EPP_ALL – credit cards are not stored on the server at all and are not loaded to the client.
5. CUSTOM - customized credit card flow
    a. CBA - credit cards are not stored on the server. Details of credit cards are specified using internal payment page, secured by third-party library. Sensitive credit card fields are hosted on third-party web-site. System has no information about credit card, only third-party token.

## 1. Credit card payment workflows

### Plain

The client sends plain card details to the server. Tokens for previously used credit cards are stored on the server and can be loaded to the client.

### Client side encryption

Client side encryption means that the server receives only encrypted (on the client side) credit card details. In case the client tries to send plain credit card details while CSE is used, the server returns the CREDIT_CARD_PAYMENT_FAILED response status. In case of successful login for CSE workflow, the server also sends a code of the encryption algorithm and a public key used during the encryption process.

### External payment pages for new credit cards

External payment page (EPP) workflow means that a user fills in and validates new credit card details on a payment provider web site. As soon as it has been done once, for the next bookings the user can pay with previously used credit cards, without external validation.

### External payment pages for all credit cards

All credit card payments require 3-d party validation. Since the server does not store any card data, saving, loading and deletion of credit card information is not acceptable.
Detailed workflow explanation is specified below in the booking confirmation section.

### 3-D Secure

Sherlock supports the 3-D Secure mechanism: a user fills in payment details or validates a payment on a credit card provider web site.
For these purposes, a user can be redirected to a web site during booking confirmation to fill in details or validate a payment and return back to finish booking creation.
The 3-D Secure process can be divided into the following steps:

1. The client sends a booking confirmation request with credit card details specified according to a credit card type (cf. below);
2. Sherlock performs a credit card transaction and on demand of the payment provider returns a response with redirection details – URL, HTTP parameters and HTTP method which should be used during redirect;
3. The client redirects a user from the application to a payment provider web site using the mobile web browser;
4. The user fills in all required data and submits a payment form;
5. The payment provider sends a 'return' request to Sherlock;
6. Since this request is made as non-authorised in the Sherlock session, it uses the 'mirror' mechanism – returning to the client a response that contains the same parameters as in the 'return' request;
7. The client intercepts the 'mirror' response, extracts given parameters and sends a request for complete booking confirmation to Sherlock (/job /payment/complete for credit cards and /paypal/account/complete in case of pay pal payment).

### CBA (aka MasterCard) and Stripe

Sherbook has no information about used credit card at all. All operations performed using third-party token obtained using internal payment page.

## 2. Load available credit cards

Existing credit card loading is allowed for PLAIN, CSE and EPP_NEW payment workflows, and not supported for EPP_ALL or CBA since credit cards in this case are not saved in Sherlock at all.

## URL: /creditCard/load

## Request:

Cf AmendableCustomerDataRequest.

## Response:

### LoadCreditCardResponse

| Parameter | Type | Description |
|---|---|---|
| creditCards | List<ShortCreditCardDto> | Loaded credit cards |

### ShortCreditCardDto

| Parameter | Type | Description |
|---|---|---|
| customerType | CustomerType | Customer type |
| id | UUID | Credit card identifier |
| number | String | Credit card number |
| type | String | Credit card type |

## Possible response codes:

| Value | Description |
|---|---|
| OK | Offer has been successfully checked |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

## Request example:

| Request | Response |
|---|---|
| {<br>"customerTypes" :<br>[<br>"RETAIL"<br>]<br>} | {<br>"status": "OK",<br>"errorMessage": null,<br>"creditCards": [<br>{<br>"customerType": "CORPORATE",<br>"id": "969f0e6f-5b25-36b2-181a-316ca20375db",<br><br>"number": "********-1234",<br><br>"type": "VISA"<br>}<br>]<br>} |

## 3. Save a new profile card

Credit card saving is allowed for PLAIN, CSE and EPP_NEW payment workflows, and not supported for EPP_ALL since all credit cards details are validated by a credit card provider and stored on its side.
In case a credit card provider supports CSE or EPP_NEW payment workflows and the server sends the REDIRECT_REQUIRED response status, the 3-D Secure authorisation process must be passed.

### URL: /creditCard/save

### Request:

#### SaveCreditCardRequest

| Parameter | Type | Description |
|---|---|---|
| **customerType** | CustomerType | Customer type |
| **creditCard** | CreditCardDto | Credit card information |

#### CreditCardDto

| Parameter | Type | Description |
|---|---|---|
| id | UUID | Credit card identifier |
| number | String | Masked credit card number |
| type | String | Credit card type (i.e. 'Visa' or 'Master Card') |
| validFrom | String | 'Valid from' date of the credit card |
| expiryYear | String | Year of credit card expiration |
| expiryMonth | String | Month of credit card expiration |
| holderName | String | Name of credit card holder |
| issueNumber | String | Credit card issue number |
| securityNumber | String | Credit card security number |
| billingAddress | String | Credit card security number |
| billingPostcode | String | Credit card billing postcode |
| useForInvoicing | boolean | Indicates if the credit card is used for invoicing |
| encryptedDetails | String | Credit card details encrypted on the client side |
| sessionId | String | Token to work with credit cards in CBA flow. Returned by internal payment page in mirror response |

### Response:

#### SaveCreditCardResponse

| Parameter | Type | Description |
|---|---|---|
| creditCard | ShortCreditCardDto | Saved credit card |
| redirectInfo | RedirectDto | Redirection information |

#### RedirectDto

| Parameter | Type | Description |
|---|---|---|

| | | |
|---|---|---|
| url | String | URL for redirect |
| params | Map<String, String> | HTTP parameters for redirection |
| method | RedirectMethod | HTTP method used during redirection |
| html | String | HTML page shown to the client |

### RedirectMethod

| Value | Description |
|---|---|
| GET("GET") | HTTP GET method |
| POST("POST") | HTTP POST method |

## Possible response codes:

| Value | Description |
|---|---|
| OK | Credit card has been successfully saved |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| CREDIT_CARD_PAYMENT_FAILED | Server is unable to save the specified credit card |

# 4. Complete profile card saving

After new credit card details are filled in and validated on a payment provider web site, it is required to complete the 3-D Secure process by calling the **/creditCard/save/complete** url.

## URL: /creditCard/save/complete

## Request:

### CompleteSaveCreditCardRequest

| Parameter | Type | Description |
|---|---|---|
| **parameters** | Map<String, String> | Parameters sent by the provider |
| **content** | String | Content sent by the provider |
| customerType | CustomerType | Customer type |

## Response:

Cf. SaveCreditCardResponse.

## Possible response codes:

| Value | Description |
|---|---|
| OK | Credit card has been successfully saved |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |

| | |
|---|---|
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| CREDIT_CARD_PAYMENT_FAILED | Server is unable to complete credit card saving |

## 5. Delete an existing card

Deletion operation for existing credit cards is allowed for PLAIN, CSE and EPP_NEW payment workflows, and not supported for EPP_ALL since credit cards in this case are not saved in Sherlock at all.

### URL: /creditCard/delete

### Request:

#### DeleteCreditCardRequest

| Parameter | Type | Description |
|---|---|---|
| **customerType** | CustomerType | Customer type |
| **creditCardId** | UUID | Credit card identifier |

### Response:

Cf. BaseResponse

### Possible response codes:

| Value | Description |
|---|---|
| OK | Credit card has been successfully deleted |
| INVALID_REQUEST | Credit card identifier is not filled in or the credit card with the specified identifier does not exist |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

## 6. CBA (aka MasterCard) & Stripe

### URL: GET /creditCard/session

Internal payment page url for credit card details input. Redirects to /payment/mirror after user finished details input. Mirror response body contains session id information. Session id parameter should be used in CreditCardDto.

# 26. Booking confirmation

The booking confirmation process is regulated by the following rules:

1. Service cannot be empty.
2. Date rules:
   2.1. If a booking is ASAP, then a date can be empty;
   2.2. If a booking is NOT ASAP, then a date can NOT be empty;
   2.3. If a booking is NOT ASAP, then a date should be later than {now + 10 minutes};
3. Credit card: if the payment type supports credit cards, then the credit card BOOKING_ALREADY_PROCESSEDd field should be filled in (CreditCardDto):
   3.1. Payment with plain details:
   3.1.1.In case of using a new card, the client should fill in the following fields:
   - number;
   - securityNumber;
   - expiryYear;
   - expiryMonth;
   - holderName;

   The rule for asking the 'securityNumber' value is specified in the 'cvcRequiredForNewCreditCards' field inside the payment type selected by the user.

   3.1.2. In case a previously used credit card is selected, the client should fill in the following fields:

   - id;
   - number;
   - securityNumber.

   The rule for asking the 'securityNumber' value is specified in the 'cvcRequiredForExistingCreditCards' field inside the payment type selected by the user.

   3.2. Client side encryption workflow

   3.2.1. In case of using a new card, the client should fill in the following fields:

   - number;
   - encryptedDetails.

   The credit card number must be masked – only four last digits should be sent in a plain view, i.e. it should look like "********-1234". Otherwise, the server returns the CREDIT_CARD_PAYMENT_FAILED response code.
   Encrypted details must be encrypted with the corresponding encryption algorithm, i.e. 'Adyen', with a public key received from the server in the CreditCardSettings object.
   The rule for asking the 'securityNumber' value is specified in the 'cvcRequiredForNewCreditCards' field inside the payment type selected by the user. In case a CVC is required, its value must be included into encrypted details.
   On booking confirmation the server can return the REDIRECT_REQUIRED response code, which means that the client should be redirected to a payment provider page to validate a payment. Information about redirect in that case is also sent in the ConfirmJobResponse, in the redirectInfo field (cf. RedirectDto). Redirection implies showing of a payment provider webpage using a smartphone built-in web browser.

   3.2.2. Rules for a case when a previously used CC is selected are the same as in 3.1.2.

   3.3. External payment pages for new cards and external payment pages for all cards:

   External payment pages (EPP) mean that a user fills in all CC details on a payment provider web site. In this case, on booking confirmation the server ALWAYS returns the REDIRECT_REQUIRED response code with redirection details included into the response. Application should send *creditCardChanged* flag if user changed credit card during amendment.

   3.4. In case a payment via a credit card fails, the client should have a possibility to change the payment type.

   3.5. The 'CVC-required' settings for new and existing CC's are stored in the payment type selected by the user in corresponding flags (cf. PaymentTypeDto).

4. Rules for stops:

   4.1. List of stops should contain at least one element;

   4.2. Every stop should have a non-empty address;

4.3. If the first stop address contains at least one of arrival flags, then a user should fill in the arrival details: 'flight number', 'arrival time', 'landed', 'arrival from';

4.4. The first stop is always a pickup, the others are dropoffs;

4.5. In case no dropoff is specified, the client should specify at least one of route flags in RouteInfo;

4.6. In case of a Delivery Service, contact details should be specified for each stop.

4.7. In case of Proposed address following fields should be popolated: initialLatitude, initialLongitude, initialFormattedAddress.

5. The price cannot be empty and should be equal to the price calculated earlier.

# 1. Confirm booking creation

## URL: /job/book

## Request:

### BookJobRequest

| Parameter | Type | Description |
|---|---|---|
| **customerType** | CustomerType | Customer type |
| **operationType** | BookingOperationType | Booking operation type |
| **requestId** | UUID | Booking request identifier or offer identifier |
| usedJobId | UUID | Identifier of an amended or repeated booking |
| **service** | BookingServiceDto | Selected service |
| **paymentType** | BookingPaymentTypeDto | Selected payment type |
| **asap** | boolean | Indicates if the booking is ASAP |
| date | Date | Booking date, for prebooks only |
| **stops** | List<StopDto> | Booking stops |
| **routeInfo** | RouteInfo | Booking route info |
| **price** | PriceDto | Calculated booking price |
| **contact** | ContactDto | Booking contact |
| passengers | List<ContactDto> | List of passengers |
| references | List<ReferenceDto> | List of specified references |
| instructions | List<SpecialInstructionDto> | Specified special instructions |
| loyaltyCard | LoyaltyCardDto | Selected loyalty card |
| voucher | VoucherDto | Selected electronic voucher |
| promoCode | String | Promo code |
| notes | String | Booking notes |
| creditCard | CreditCardDto | Credit card |

| | | |
|---|---|---|
| numberOfPassengers | Integer | Number of passengers |
| smallBagsNumber | Integer | Number of small bags |
| bigBagsNumber | Integer | Number of big bags |
| `creditCardChanged` | Boolean | This flag should be set to true during job amendment if provider is of type EPP_ALL and user changed credit card |

## Response:

### BookJobResponse

| Parameter | Type | Description |
|---|---|---|
| jobId | UUID | Confirmed booking identifier |
| number | String | Confirmed booking number |
| date | Date | Booking date |
| delay | Integer | Booking delay |
| price | PriceDto | Booking price |
| extPrice | PriceDto | Booking price converted to the alternative currency |
| redirectInfo | RedirectDto | Information about the client redirection to the 3D-party site to validate/confirm credit card details |

## Possible response codes:

| Value | Description |
|---|---|
| OK | Delay has been successfully calculated |
| INVALID_REQUEST | Request data is not filled in properly |
| INVALID_CUSTOMER_TYPE | Customer type is not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| INCORRECT_REQUEST_IDENTIFIER | Specified request identifier has been already used in another booking |
| INVALID_ADDRESS_DATA | Unable to assemble a stop address from the specified data |
| INVALID_COORDINATES | Pickup stop coordinates are in the invalid area |
| INSTRUCTION_VALIDATION_FAILED | Specified special instruction values are invalid |
| PICKUP_CIRCUIT_LOCKED | Specified address area is locked for ASAP bookings |
| INVALID_OPERATIONAL_AREA | Specified service is not available in the selected pickup circuit |
| INVALID_ROUTE_INFO | Attempted to create a 'Wait and return', 'Destination unknown' or 'As directed' booking while it is not supported by the selected service |
| PAYMENT_TYPE_NOT_SUPPORTED | Selected payment type is not supported |

| | |
|---|---|
| INVALID_VOUCHER | Specified voucher is not found |
| VOUCHER_NOT_ACTIVE | Specified voucher is not active |
| INVALID_LOYALTY_CARD | Loyalty card is invalid |
| INVALID_PROMO_CODE | Promo code is invalid |
| TOO_EARLY_FOR_PREBOOK | Too early date for prebook creation |
| TOO_LATE_FOR_PREBOOK | Too late date for prebook creation |
| FULLY_BOOKED | Limit of available bookings has been exceeded |
| PREBOOK_NOT_AVAILABLE | Prebooks are not available |
| PRICE_NOT_CALCULATED | Price has not been calculated for some internal reasons |
| PRICE_CHANGED | Calculated price and the specified one are different |
| BOOKING_ALREADY_PROCESSED | Booking with the specified request identifier has been already processed |
| BOOKING_CONFIRMATION_FAILED | Unable to confirm a booking due to internal reasons |
| BOOKING_NOT_AMENDABLE | Booking cannot be amended |
| BOOKING_NOT_REPEATABLE | Booking cannot be repeated |
| CREDIT_CARD_PAYMENT_FAILED | Unable to perform a payment with the specified credit card |
| SERVICE_RESTRICTION_FAILED | Faced one of service restrictions |
| REDIRECT_REQUIRED | Client should be redirected to the payment provider site |
| INVALID_ACCOUNT_DATA | Specified service, payment type or passenger are not available to the user |
| DATA_NOT_FOUND | Unable to find booking related data by the identifier specified in the request |
| INVALID_REQUEST | Contact details are not specified for each stop |
| OPERATION_UNAVAILABLE | Automatic lock service occurred |
| ASAP_ONLY_SERVICE | Only ASAP bookings are available for the service |

## 2. Complete a booking payment

### URL: /job/payment/complete

### Request:

#### CompletePaymentRequest

| Parameter | Type | Description |
|---|---|---|
| **parameters** | Map<String, String> | Booking identifier, should be created by the client |
| **content** | String | Booking payment type |
| **customerType** | CustomerType | Customer type |

### Response:

Cf. BookJobResponse.

# 27. Booking history

The client loads a list of active and archive bookings in separate methods. Both active and archive bookings can be loaded in a short view or with full details specified. List is sorted chronologically.

## 1. Load active bookings

### URL: /booking/load/active

### Request:

Cf. CustomerDataRequest.

### Response:

#### BookingsResponse

| Parameter | Type | Description |
|---|---|---|
| bookings | List<BookingListDto> | List of loaded bookings |

#### BookingListDto

| Parameter | Type | Description |
|---|---|---|
| customerType | CustomerType | Customer type |
| id | UUID | Booking identifier |
| number | String | Booking number |
| date | Date | Booking date |
| pickup | String | Formatted first address stop |
| drop | String | Formatted last address stop |
| status | BookingStatus | Booking status |
| driver | DriverDto | Driver information |
| routeInfo | RouteInfo | Route information |
| journeyType | JourneyType | Booking journey type |
| asap | boolean | Indicates if the booking is ASAP |
| confirmDriverChanges | boolean | Indicates if not all driver changes have been confirmed by the customer yet |
| partner | PartnerDto | Information abou partner in case of partner integration |

#### BookingStatus

| Value | Description |
|---|---|
| BOOKED(0) | Booking has just been created. A driver is not assigned yet. |
| DRIVER_ON_WAY(10) | Driver has been already assigned and he is on way to the pickup stop |
| DRIVER_AT_PICKUP(20) | Driver has reached a pickup stop and he is waiting for a customer |
| PASSENGER_ON_BOARD(30) | Passenger on board |

| | |
|---|---|
| DONE(40) | Booking has been successfully done |
| CANCELLED(50) | Booking has been cancelled |

### JourneyType

| Value | Description |
|---|---|
| STANDARD(10) | Normal journey: no trains, no airports |
| PICKUP_FROM_AIRPORT(20) | First stop is an airport |
| DROP_AT_AIRPORT(30) | Last stop is an airport |
| PICKUP_FROM_TRAIN(40) | First stop is a train station |
| DROP_TO_TRAIN(50) | Last stop is a train station |

## Possible response codes:

| Value | Description |
|---|---|
| OK | Booking list has been successfully loaded |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

# 2. Load full active booking information

## URL: /booking/load/active/full

## Request:

Cf. CustomerDataRequest.

## Response:

### DetailedBookingsResponse

| Parameter | Type | Description |
|---|---|---|
| bookings | List<BookingDetailsDto> | List of loaded bookings |

### BookingDetailsDto

| Parameter | Type | Description |
|---|---|---|
| customerType | CustomerType | Customer type |
| id | UUID | Booking identifier |
| number | String | Booking number |
| version | Integer | Booking version |
| status | BookingStatus | Booking status |

| | | |
|---|---|---|
| date | Date | Booking date |
| service | BookingServiceDto | Booking service |
| paymentType | BookingPaymentTypeDto | Booking payment type |
| account | String | Booking account number |
| stops | List<StopDto> | Booking stops |
| routeInfo | RouteInfo | Booking route info |
| driver | DriverDto | Allocated driver |
| creditCard | ShortCreditCardDto | Credit card used in the booking |
| loyaltyCard | LoyaltyCardDto | Selected loyalty card |
| voucher | VoucherDto | Selected electronic voucher |
| promoCode | PromoCodeDto | Used promo code |
| notes | String | Booking notes |
| contact | ContactDto | Booking contact |
| passengers | List<ContactDto> | List of booking passengers |
| amendable | boolean | Indicates if the booking can be modified |
| cancellable | boolean | Indicates if the booking can be cancelled |
| rate | Integer | Booking rate |
| rateSkipped | boolean | Indicates if booking rating has been skipped by the customer |
| price | PriceDto | Booking price |
| extPrice | PriceDto | Price converted to the alternative currency |
| instructions | List<SpecialInstructionDto> | Special instructions used in the booking |
| references | List<ReferenceDto> | References used in the booking |
| asap | boolean | Indicates if the booking is ASAP |
| confirmDriverChanges | boolean | Indicates if not all driver changes have been confirmed by the customer yet |
| unreadChatMessages | boolean | Indicates if there are unread chat messages from the driver |
| numberOfPassengers | Integer | Number of passengers |
| smallBagsNumber | Integer | Number of small bags |
| bigBagsNumber | Integer | Number of big bags |

## Possible response codes:

| Value | Description |
|---|---|
| OK | Booking list has been successfully loaded |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

## 3. Load archive bookings

### URL: /booking/load/history

### Request:

#### ArchiveBookingsRequest

| Parameter | Type | Description |
|---|---|---|
| **customerTypes** | List<CustomerType> | Customer types |
| lastUpdateDate | Date | Last request date (from the response) |
| limit | Integer | Maximum number of loaded bookings |

### Response:

#### ArchiveBookingsResponse

| Parameter | Type | Description |
|---|---|---|
| bookings | List<BookingListDto> | List of loaded bookings |
| hiddenBookings | List<UUID> | List of hidden booking identifiers |
| lastUpdateDate | Date | Date of the response sent |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Booking list has been successfully loaded |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

## 4. Load full archive booking information

### URL: /booking/load/history/full

### Request:

Cf. ArchiveBookingsRequest.

### Response:

#### DetailedArchiveBookingsResponse

| Parameter | Type | Description |
|---|---|---|
| bookings | List<BookingDetailsDto> | List of loaded bookings |
| hiddenBookings | List<UUID> | List of hidden booking identifiers |

| | | |
|---|---|---|
| lastUpdateDate | Date | Date of the response sent |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Booking list has been successfully loaded |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

## 5. Load detailed information for a single booking

This endpoint can be called anonymously

### URL: /booking/load/details

### Request:

Cf. BookingIdRequest.

### Response:

#### BookingDetailsResponse

| Parameter | Type | Description |
|---|---|---|
| booking | BookingDetailsDto | Detailed booking information |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Detailed booking information has been successfully loaded |
| INVALID_REQUEST | Booking identifier is not specified |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| DATA_NOT_FOUND | Unable to find a booking by the specified identifier |

## 6. Refresh booking details

To keep active booking data in the actual state, the client refreshes it using the method specified below. The details refresh period used by the client is specified in the *SystemSettingsDto.jobDetailsRefreshPeriod* field. This endpoint can be called anonymously

### URL: /booking/load/version

### Request:

Cf. BookingIdRequest.

### Response:

#### BookingVersionResponse

| Parameter | Type | Description |
|---|---|---|
| version | Integer | Actual booking version |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Service availability check has been successfully passed |
| INVALID_REQUEST | Booking identifier is not specified |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| DATA_NOT_FOUND | Unable to find a booking by the specified identifier |

## 7. Refresh driver details

When a driver executes an active booking, the client receives the driver`s coordinates from the server and refreshes them using the method specified below. Driver coordinates refresh period used by the client is specified in the driverPositionRefreshPeriod field of SystemSettingsDto.

### URL: /booking/load/driverPosition

### Request:

Cf. BookingIdRequest.

### Response:

#### DriverPositionResponse

| Parameter | Type | Description |
|---|---|---|
| latitude | Double | Driver latitude |
| longitude | Double | Driver longitude |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Driver position has been successfully loaded |
| INVALID_REQUEST | Booking identifier is not specified |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

| | |
|---|---|
| DATA_NOT_FOUND | Unable to find a booking by the specified identifier |

## 8. Hide bookings in the Booking History list

The client uses the method specified below to hide bookings in Booking history after they have been executed or cancelled. Please note, that there is no possibility for the client to revert hidden bookings back.

### URL: /booking/hide

### Request:

#### HideBookingsRequest

| Parameter | Type | Description |
|---|---|---|
| **bookingIds** | Map<UUID, CustomerType> | Map of booking identifiers and customer types related to them |

### Response:

Cf. BaseResponse.

### Possible response codes:

| Value | Description |
|---|---|
| OK | Bookings have been successfully hidden |
| INVALID_REQUEST | One of booking identifiers is not filled in properly |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| DATA_NOT_FOUND | Unable to find a booking by the specified identifier |

## 9. Send booking information by email

This endpoint allows to send booking information to the specified email. Selection of a template depends on the booking status.

### URL: /booking/sendByEmail

### Request:

#### SendBookingRequest

| Parameter | Type | Description |
|---|---|---|
| **jobId** | UUID | Booking identifier |
| **email** | String | Email |
| **customerType** | CustomerType | Customer Type |

## Response:

### SendBookingResponse

| Parameter | Type | Description |
|---|---|---|
| sendSuccess | boolean | Indicates if sending has been completed succesfully |
| sendErrorMessage | String | Email validation error message |

## Possible response codes:

| Value | Description |
|---|---|
| OK | Booking information has been successfully sent |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| DATA_NOT_FOUND | Unable to find a booking by the specified identifier |

## 10. Load booking link

This endpoint allows to load link followed by message text which is supposed to be shared with somebody else

## URL: /booking/load/link

## Request:

### CustomerRequest

Cf. BookingIdRequest.

## Response:

### BookingLinkResponse

| Parameter | Type | Description |
|---|---|---|
| link | String | Link to the booking |
| text | String | Message to be shown to client |

Possible response codes:

| Value | Description |
|---|---|
| OK | Booking link has been succesfully loaded |
| INVALID_REQUEST | Booking identifier or customer type are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| DATA_NOT_FOUND | Unable to find a booking by the specified identifier |

## 11. Load POD signature

This endpoint allows to load an image of a POD signature.

### URL: /booking/load/signature/{imageId}

**Request:**

| Path variable | Description |
|---|---|
| imageId | Identifier of FileDescriptor |

**Response:**

Byte array representing the image.

**Possible error codes:**

| Value | Description |
|---|---|
| SC_NOT_FOUND (404) | FileDescriptor was not found or there was an error during the loading of the file |

# 28. Booking details modification

An ability to modify booking details is defined in the **amendable** flag of the BookingDetailsDto object. Users can modify the following booking details:

- Booking date
- ASAP flag
- Stops
- Route info
- Special instructions
- References
- Booking contact and passengers
- Notes

During booking details modification the client performs the same set of operations as during booking creation but with the AMEND booking operation type and the **usedJobId** filled in with the modified booking identifier.
In case a booking has become unmodifiable, the server returns the BOOKING_NOT_AMENDABLE response status to the client.

# 29. Booking cancellation

An ability to cancel a single booking is defined in the **cancellable** flag of the BookingDetailsDto object.
The cancellation workflow steps are specified below:

1. The client sends a request to the server to get a cancellation type and calculate an amount of the cancellation charge, if exists, to URL **/booking/cancel/charge**;
2. In case of free cancellation the server returns the ***FREE*** value in the response in the **type** field and empty values in the **charge** and **formattedCharg** fields;
3. In case of cancellation on arrival the server calculates a cancellation charge and sends it to the client in the **charge** and **formattedCharge** fields with the **COA** value in the **type** field;
4. The client should confirm booking cancellation by sending a request to URL **/booking/cancel** both in cases of FREE or COA cancellation types;
5. In case of cancellation on arrival, the calculated cancellation charge has a validity period: if the client tries to cancel a booking after the validity period has been exceeded and the cancellation charge has already changed, then the server returns the **CANCELLATION_CHARGE_CHANGED** response status and recalculated values in the **charge** and **formattedCharge** fields, which are required to be confirmed by the client (cf. point 4 above).

## 1. Load a cancellation charge

### URL: /booking/cancel/charge

### Request:

Cf. BookingIdRequest.

### Response:

#### CancelBookingResponse

| 1. Parameter | Type | Description |
|---|---|---|
| type | CancellationType | Cancellation type |
| charge | PriceField | Calculated booking cancellation charge value |

#### CancellationType

| Value | Description |
|---|---|
| FREE("FREE") | Free cancellation, no fee is applied |
| COA("COA") | Cancellation on arrival, a cancellation charge is applied |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Cancellation charge has been successfully calculated |
| INVALID_REQUEST | Booking identifier is not specified |
| INVALID_CUSTOMER_TYPE | Customer type is not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| BOOKING_NOT_CANCELLABLE | Booking cannot be cancelled |

| | |
|---|---|
| BOOKING_ALREADY_CANCELLED | Booking has been already cancelled |
| DATA_NOT_FOUND | Unable to find a booking by the specified identifier |
| PRICE_CALCULATION_ERROR | Operation failed. The customer can try again or contact Customer Service |

## 2. Cancel a booking

### URL: /booking/cancel

### Request:

Cf. BookingIdRequest.

### Response:

Cf. CancelBookingResponse.

### Possible response codes:

| Value | Description |
|---|---|
| OK | Cancellation charge has been successfully calculated |
| INVALID_REQUEST | Booking identifier is not filled in or the booking is not found |
| INVALID_CUSTOMER_TYPE | Customer type is not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| BOOKING_NOT_CANCELLABLE | Booking cannot be cancelled |
| BOOKING_ALREADY_CANCELLED | Booking has been already cancelled |
| CANCELLATION_CHARGE_CHANGED | Cancellation charge has already changed |
| DATA_NOT_FOUND | Unable to find a booking by the specified identifier |

# 30. Complaints

## 1. Create a complaint

To create a feedback or a complaint about a booking the client sends a corresponding request with a booking identifier, a contact and a complaint text specified.

### URL: /complaint/create

### Request:

#### CreateComplaintRequest

| Parameter | Type | Description |
|---|---|---|
| **customerType** | CustomerType | Customer type |
| **id** | UUID | Booking identifier |
| **contact** | ContactDto | Complaint creator |
| **text** | String | Complaint text |

### Response:

Cf. BaseResponse.

### Possible response codes:

| Value | Description |
|---|---|
| OK | Complaint has been successfully created |
| INVALID_REQUEST | Request data is not filled in properly |
| INVALID_CUSTOMER_TYPE | Customer type is not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| DATA_NOT_FOUND | Unable to find a booking by the specified identifier |

## 2. Load a complaint

After a complaint is created, the client can load it using the booking identifier.

### URL: /complaint/load

### Request:

Cf. BookingIdRequest.

### Response:

#### LoadComplaintResponse

| Parameter | Type | Description |
|---|---|---|

| | | |
|---|---|---|
| text | String | Complaint text |

## Possible response codes:

| Value | Description |
|---|---|
| OK | Complaint has been successfully loaded |
| INVALID_REQUEST | Booking identifier is not filled in properly |
| INVALID_CUSTOMER_TYPE | Customer type is not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| DATA_NOT_FOUND | Unable to find a booking by the specified identifier |

# 31. Passenger survey

## 1. Load a survey

### URL: /survey/load

### Request:

Cf. BookingIdRequest.

### Response:

#### LoadSurveyResponse

| Parameter | Type | Description |
|---|---|---|
| survey | SurveyDto | Loaded survey |

#### SurveyDto

| Parameter | Type | Description |
|---|---|---|
| id | UUID | Survey identifier |
| questions | List<QuestionDto> | Survey questions |
| finishDate | Date | Survey confirmation date or null if the survey has not been confirmed yet |

#### QuestionDto

| Parameter | Type | Description |
|---|---|---|
| id | UUID | Question identifier |
| text | String | Question text |
| code | String | Question code |
| type | QuestionType | Question answer type |
| values | List<AnswerValueDto> | Possible answer values, for questions with the ENUM question type |

#### QuestionType

| Value | Description |
|---|---|
| SIMPLE ("SIMPLE") | Answer text should be selected from the 'Y', 'N' or 'N/A' options |
| STRING ("STRING") | String data type |
| INTEGER ("INTEGER") | Integer data type |
| FLOAT ("FLOAT") | Float (double) data type |
| BOOLEAN ("BOOLEAN") | Boolean data type |
| DATE("DATE") | Date data type. It should be specified in the 'dd/MM/yyyy HH:mm' format |

| | |
|---|---|
| ENUM ("ENUM") | It will be used in future versions. Enumeration data type. For this parameter the type value should be selected from the list of possible parameter values. An answer should be placed to the 'answerCode' field |

### AnswerValueDto

| Parameter | Type | Description |
|---|---|---|
| code | String | Question code. It is used during confirmation and should be placed to the answerCode field of AnswerDto |
| value | String | Answer value |

## Possible response codes:

| Value | Description |
|---|---|
| OK | Survey has been successfully loaded |
| INVALID_REQUEST | Booking identifier is not filled in properly |
| INVALID_CUSTOMER_TYPE | Customer type is not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| DATA_NOT_FOUND | Unable to find a booking by the specified identifier |

# 2. Confirm a survey

## URL: /survey/confirm

## Request:

### ConfirmSurveyRequest

| Parameter | Type | Description |
|---|---|---|
| **surveyId** | UUID | Survey identifier |
| rate | Integer | Survey rate |
| **answers** | List<AnswerDto> | List of survey answers |
| skipped | Boolean | Indicates if the survey has been skipped by the user |

### AnswerDto

| Parameter | Type | Description |
|---|---|---|
| questionId | UUID | Question identifier |
| questionCode | String | Question code |
| text | String | Answer text |
| answerCode | String | Answer code, for questions with the ENUM question type only |

## Response:

Cf. BaseResponse.

### Possible response codes:

| Value | Description |
|---|---|
| OK | Survey has been successfully confirmed |
| INVALID_REQUEST | Booking identifier is not filled in properly |
| INVALID_CUSTOMER_TYPE | Customer type is not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| BOOKING_ALREADY_PROCESSED | Survey has been already submitted for the specified booking |

## 3. Load a survey result

### URL: /survey/result

### Request:

Cf. BookingIdRequest.

### Response:

#### LoadSurveyResultResponse

| Parameter | Type | Description |
|---|---|---|
| surveyId | UUID | Survey identifier |
| rate | Integer | Survey rate |
| surveyAnswers | List<AnswerDto> | Survey answers |
| skipped | Boolean | Indicates if the survey has been skipped by the user |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Survey result has been successfully loaded |
| INVALID_REQUEST | Booking identifier is not filled in properly |
| INVALID_CUSTOMER_TYPE | Customer type is not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| BOOKING_NOT_RATED | Survey for the specified booking has not been confirmed yet |
| DATA_NOT_FOUND | Unable to find a booking by the specified identifier |

## 4. Load a survey anonymously using a token

### URL: /survey/load/token

### Request:

#### TokenRequest

| Parameter | Type | Description |
|---|---|---|
| tokenId | String | Token identifier (can be received via email) |

### Response:

Cf. LoadSurveyResponse

#### Possible response codes:

Cf. Load survey response codes and additional:

| Value | Description |
|---|---|
| DATA_NOT_FOUND | The token '%s' is incorrect |

## 5. Confirm a survey anonymously using a token

### URL: /survey/confirm/token

### Request:

#### ConfirmSurveyByTokenRequest

| Parameter | Type | Description |
|---|---|---|
| tokenId | String | Token identifier (can be received via email) |
| **surveyId** | UUID | Survey identifier |
| rate | Integer | Survey rate |
| **answers** | List<AnswerDto> | List of survey answers |
| skipped | Boolean | Indicates if the survey has been skipped by the user |

### Response:

Cf. BaseResponse.

#### Possible response codes:

Cf. Confirm survey response codes and additional:

| Value | Description |
|---|---|
| DATA_NOT_FOUND | The token '%s' is incorrect |

## 6. Load unrated booking identifiers

## URL: /survey/load/unrated

## Request:

Cf. CustomerDataRequest.

## Response:

### UnratedBookingsResponse

| Parameter | Type | Description |
|---|---|---|
| unrated | List<UnratedBookingDto> | List of result DTO's |

### UnratedBookingDto

| Parameter | Type | Description |
|---|---|---|
| id | UUID | Unrated booking identifier |
| customerType | CustomerType | Customer type |

## Possible response codes:

| Value | Description |
|---|---|
| OK | Unrated bookings has been successfully loaded |
| INVALID_CUSTOMER_TYPE | Customer type is not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

# 32. Tips management

## 1. Load a tips range

### URL: /tips/range

### Request:

Cf. BookingIdRequest.

### Response:

#### TipsRangeResponse

| Parameter | Type | Description |
|---|---|---|
| tipsRange | List<TipsDto> | Tips range |

#### TipsDto

| Parameter | Type | Description |
|---|---|---|
| tips | PriceField | Tips range |
| total | PriceField | Total price |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Tips range has been successfully loaded |
| INVALID_REQUEST | Request data is not filled in properly |
| INVALID_CUSTOMER_TYPE | Customer type is not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

## 2. Submit tips

### URL: /tips/submit

### Request:

#### SubmitTipsRequest

| Parameter | Type | Description |
|---|---|---|
| **customerType** | CustomerType | Customer type |
| **id** | UUID | Booking identifier |
| **tips** | PriceField | Tips value |

### Response:

Cf. BaseResponse.

## Possible response codes:

| Value | Description |
| --- | --- |
| OK | Tips have been successfully submitted |
| INVALID_REQUEST | Booking identifier is not filled in properly |
| INVALID_CUSTOMER_TYPE | Customer type is not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| TIPS_NOT_ALLOWED | Tips couldn't be added to the selected booking |

# 33. Lost property

## 1. Load a lost property

### URL: /load/lost

### Request:

Cf. BookingIdRequest.

### Response:

#### LostPropertiesResponse

| Parameter | Type | Description |
|---|---|---|
| properties | List<LostPropertyDto> | Lost properties |

#### LostPropertyDto

| Parameter | Type | Description |
|---|---|---|
| id | UUID | Property identifier |
| property | String | Property description |
| status | LostPropertyStatus | Property status |
| date | Date | Date of property loss |

#### LostPropertyStatus

| Value | Description |
|---|---|
| LOST("LOST") | Property is lost |
| NOT_FOUND("NOT_FOUND") | Property is not found |
| FOUND("FOUND") | Property is found |
| RETURNED("RETURNED") | Property is returned to the owner |
| DISPOSED("DISPOSED") | Property is disposed |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Property has been successfully loaded |
| INVALID_REQUEST | Request data is not filled in properly |
| INVALID_CUSTOMER_TYPE | Customer type is not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| DATA_NOT_FOUND | Unable to find a booking by the specified identifier |
| INVALID_ACCOUNT_INFO | Authenticated user details do not match booking details |

## 2. Add a lost property

### URL: /lost/add

### Request:

#### AddLostPropertyRequest

| Parameter | Type | Description |
|---|---|---|
| **customerType** | CustomerType | Customer type |
| **id** | UUID | Booking identifier |
| **property** | String | Lost property description |

### Response:

Cf. BaseResponse.

### Possible response codes:

| Value | Description |
|---|---|
| OK | Property has been successfully added |
| INVALID_REQUEST | Booking identifier or property description is not filled in properly |
| INVALID_CUSTOMER_TYPE | Customer type is not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| DATA_NOT_FOUND | Unable to find a booking by the specified identifier |
| INVALID_ACCOUNT_INFO | User details do not match booking details |

## 3. Edit a lost property

### URL: /lost/edit

### Request:

#### EditLostPropertyRequest

| Parameter | Type | Description |
|---|---|---|
| **customerType** | CustomerType | Customer type |
| **id** | UUID | Booking identifier |
| **propertyId** | UUID | Lost property identifier |
| **property** | String | Lost property description |

### Response:

Cf. BaseResponse.

### Possible response codes:

| Value | Description |
|---|---|
| OK | Property has been successfully edited |
| INVALID_REQUEST | Booking identifier, property identifier or property description is not filled in properly |
| INVALID_CUSTOMER_TYPE | Customer type is not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| DATA_NOT_FOUND | Unable to find a booking by the specified identifier |
| INVALID_ACCOUNT_INFO | User details do not match booking details |
| INACTIVE_LOST_PROPERTY | Unable to edit the inactive lost property |

## 4. Delete a lost property

### URL: /lost/delete

### Request:

#### LostPropertyRequest

| Parameter | Type | Description |
|---|---|---|
| **customerType** | CustomerType | Customer type |
| **id** | UUID | Booking identifier |
| **propertyId** | UUID | Lost property identifier |

### Response:

Cf. BaseResponse.

### Possible response codes:

| Value | Description |
|---|---|
| OK | Property has been successfully deleted |
| INVALID_REQUEST | Booking identifier or property identifier is not filled in properly |
| INVALID_CUSTOMER_TYPE | Customer type is not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| DATA_NOT_FOUND | Unable to find a booking by the specified identifier |
| INVALID_ACCOUNT_INFO | User details do not match booking details |
| INACTIVE_LOST_PROPERTY | Unable to delete the inactive lost property |

# 34. Push notifications

Push notifications replace SMS messages when it is possible: when a booking is made via the smartphone booking application, a token is passed to the server and push notifications are enabled in the system.
When a push notification is received on a mobile phone, it needs to be confirmed as soon as possible. Otherwise, after a particular time threshold (two minutes - can be configured) an SMS will be sent.
The **fullText** field contains a common text if a short version of a push notification message is configured, otherwise - empty.
If a driver has been already allocated to a booking and the driver has a mobile phone, the *driverMobilePhone* field will be included in the payload.For iPhone *aps->alert->actions* will be included, the *alert* field will be a complex one.
For iPhone OS version older than 8 (can be configured) the size of payload is limited to 256 bytes. The following fields are excluded: *actions*, *driverPhoneNumber*, *fullText*. The *Alert* field is truncated (~45 symbols).

## 1. Set a push notification token

### URL: /push/token

### Request:

#### NotificationTokenRequest

| Parameter | Type | Description |
|---|---|---|
| **customerType** | CustomerType | Customer type |
| **token** | String | Token value or URI for Windows Phone |

### Response:

Cf. BaseResponse.

### Possible response codes:

| Value | Description |
|---|---|
| OK | Smartphone token has been successfully saved |
| INVALID_CUSTOMER_TYPE | Customer type is not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

## 2. Confirm a push notification

A push notification should be confirmed ASAP, because a duplicate SMS will be sent in two minutes (may be configured) if no confirmation is received.

### URL: /push/confirm

### Request:

#### NotificationConfirmRequest

| Parameter | Type | Description |
|---|---|---|
| **pushMessageId** | String | Push notification message ID received in the push notification |

### Response:

Cf. BaseResponse.

### Possible response codes:

| Value | Description |
|---|---|
| OK | Push notification has been successfully confirmed |
| INVALID_PUSH_MESSAGE_ID | Specified push notification message identifier is invalid |

## 2.1 Confirm push notification received in background

In case when push notification was received on device while application was not running, it is recommended to send a confirmation using the URI specified below.

While processing the request, server does not perform authorization checks.

### URL: /push/received

### Request:

Cf. NotificationConfirmRequest

### Response:

Cf. BaseResponse.

### Possible response codes:

| Value | Description |
|---|---|
| OK | Push notification has been successfully confirmed |

## 3. Push notification format

### Android and Windows Phone sample


{
"type":"STATUS_CHANGED",
"payload":{
"fullText":"Votre Cab est arrivé et vous attendra gratuitement pendant 5 min. Numéro du chauffeur: +33176497593 Référence: 000280 XJ900 Numéro de la plaque LR09HTO.",
"jobId":"009a5108-d502-4bc5-84c8-ab7560bf3ee9",
"alert":"Cab est arrivé. Numéro LR09HTO",
"messageType":"DRIVER_AT_PICKUP",
"pushMessageId":"1defc8b5-4bdb-4e98-9e3c-b16bf3da93e0"
}
}


### iPhone sample, no driverPhoneNumber set

{
"aps":{
"content-available":1,
"alert":"Cab est arrivé. Numéro LR09HTO"
},
"jobId":"3f3db2e3-47f3-4981-9473-fdd5f382ec1e",
"messageType":"DRIVER_AT_PICKUP",
"type":"STATUS_CHANGED",
"pushMessageId":"992b5e90-a715-455b-b24d-ad2e52aeebff",
"driverPhoneNumber":"071063258800",
"fullText":"Votre Cab est arrivé et vous attendra gratuitement pendant 5 min. Numéro du chauffeur: +33176497593 Référence: 000280 XJ900 Numéro de la plaque LR09HTO."
}

## iPhone sample, driverPhoneNumber is set.

Alert is complex field, actions and driverPhoneNumber fields added

{
"aps":{
"content-available":1,
"alert":{
"body":"Cab est arrivé. Numéro LR09HTO",
"actions":[{
"id":"follow",
"loc-key":"pushActionFollow"
},
{
"id":"call",
"loc-key":"pushActionCall"
}
]}
},
"driverPhoneNumber":"071063258800",
"jobId":"009a5108-d502-4bc5-84c8-ab7560bf3ee9",
"messageType":"DRIVER_AT_PICKUP",
"type":"STATUS_CHANGED",
"pushMessageId":"dcecaee2-10c5-4f05-8904-4cdeee8c5012",
"fullText":"Votre Cab est arrivé et vous attendra gratuitement pendant 5 min. Numéro du chauffeur: +33176497593 Référence: 000280 XJ900 Numéro de la plaque LR09HTO."
}

## The list of most possible MessageTypes

DRIVER_ALLOCATED

DRIVER_ON_WAY

DRIVER_CHANGED

DRIVER_AT_PICKUP

DRIVER_CIRCLING

DONE

BONUS_PROGRAM_INVITATION

SURVEY_SUBMITTED

WAITING_FOR_CONFIRMATION

BOOKED

AMENDED

CANCELLED

# 35. Resources

The client loads images for entities to show them in the application. Resource loading is processed in the following steps:

1. The client sends a request containing resource entity information in ResourceRequestDto and resource sizes which depend on a user's device screen.
2. The server loads resources by the specified identifiers and checks whether the resource hash has changed or not.
3. In a response the client receives resources that have been changed since the last loading and loads resources to the device using the provided link.

Please note that in the current implementation resource loading is available only for service entities.

## Getting a resource link

### URL: /resource/get

### Request:

#### ResourcesRequest

| Parameter | Type | Description |
|---|---|---|
| **resources** | List<ResourceRequestDto> | List of requested resource information |
| **imageWidth** | Integer | Resource width |
| **imageHeight** | Integer | Resource height |

#### ResourceRequestDto

| Parameter | Type | Description |
|---|---|---|
| **entityId** | UUID | Resource entity identifier |
| **hash** | String | Resource hash |

### Response:

#### ResourcesResponse

| Parameter | Type | Description |
|---|---|---|
| **resources** | List<ResourceResponseDto> | List of requested resource information |

#### ResourceResponseDto

| Parameter | Type | Description |
|---|---|---|
| entityId | UUID | Resource entity identifier |
| resourceId | UUID | Resource identifier |
| resourceName | String | Resource file name |
| hash | String | Resource hash |
| link | String | URL to download a resource from the server |
| resourceWidth | Integer | Image width |

| | | |
|---|---|---|
| resourceHeight | Integer | Image height |
| showCustomBackground | boolean | If true a custom service image will be displayed with an animated city background on the mobile clients and no background will be displayed otherwise |

## Possible response codes:

| Value | Description |
|---|---|
| OK | Resource information has been successfully loaded |
| INVALID_REQUEST | Resource information is not filled in properly |

# 36. Languages

Sherbook allows to set a session locale. This locale is also saved to the Invididual object to be used in case an unsupported locale is sent.

Before the user sets a session locale, a locale from http headers will be used.

The locale sent by the user will be matched to the locales supported by system languages.
For example, there are three languages in the system: France (fr_FR), English (en_EN) and US English (en_US).
If the source locale (sent by the user or received from the http header) is fr_EN, the result locale will be fr_FR. If en_EN, the result locale will be en_EN.
If the user sent cn_CN, then an old locale from the Individual profile will be used, or the system default one if the caller is disabled or the language has not been defined yet.

## 1. Set a language

### URL: /language/set

### Request:

#### SetLanguageRequest

| Parameter | Type | Description |
|---|---|---|
| customerType | CustomerType | Customer type |
| language | LanguageDto | Language information |

### Response:

Cf. BaseResponse

## 2. Load available languages

### URL: /language/load

### Request:

Empty

### Response:

#### LoadLanguagesResponse

| Parameter | Type | Description |
|---|---|---|
| languages | List<LanguageDto> | Customer type |

# 37. Driver job changes

All queries that return some booking details (/load/active/full, load/active/full, /load/history/full, /load/details, load/active) also return the confirmDriverChanges boolean. It equals to true if the paymentType requires confirmation of driver job changes and there are changes with the NEW or REJECTED status.

## 1. Get driver changes for a booking

### URL: /driverChanges/load

### Request:

Cf. BookingIdRequest

### Response:

#### DriverChangesResponse

| Parameter | Type | Description |
|---|---|---|
| changes | List<DriverChangeDto> | Changes |
| oldPrice | PriceField | Price before changes have been made |
| newPrice | PriceField | Price after changes have been made |

#### DriverChangeDto

| Parameter | Type | Description |
|---|---|---|
| id | UUID | ID |
| createTs | Date | Date of the change |
| description | String | Completed description |
| type | DriverChangeType | Change type |
| bookingExtraName | String | Booking extra name (in case of a booking extra applied) |
| status | DriverChangeStatus | Status |
| oldValue | String | Parameter value before the change |
| newValue | String | Parameter value after the change |
| details | String | Additional details of the change |

#### DriverChangeType

| Value | Description |
|---|---|
| WORKING_HOURS | Working hours |
| PAYMENT_TYPE | Payment type |

### Request example:

| Request | Response |
|---|---|
| {"id":"03b696bb-65aa-4139-d048-be74b099a929",<br><br>"customerType":"RETAIL"} | {<br>"status": "OK",<br>"errorMessage": null,<br>"changes": [<br>{<br>"id": "34f3e7f3-7a0f-30d0-6d11-1b97e3cf4928",<br>"createTs": "19/02/2016 05:08",<br>"description": "Pickup waiting time was changed from 0 min to 17 min",<br>"type": "CHANGE_WAITING_TIME",<br><br>"bookingExtraName": null,<br>"status": "NEW",<br>"oldValue": "0",<br>"newValue": "17",<br>"details": null<br><br>},<br>{<br>"id": "12882a22-ebc3-acd5-3744-5ed6c77c6157",<br>"createTs": "19/02/2016 06:54",<br>"description": "Pickup waiting time was changed from 17 min to 18 min",<br>"type": "CHANGE_WAITING_TIME",<br><br>"bookingExtraName": null,<br>"status": "NEW",<br>"oldValue": "17",<br>"newValue": "18",<br>"details": null<br>},<br>{<br>"id": "b32ddf86-3a2f-7b91-0499-d0f8aa7a60c4",<br>"createTs": "19/02/2016 06:56",<br>"description": "Payment type was changed from Cash to CC in ca",<br>"type": "PAYMENT_TYPE",<br>"bookingExtraName": null,<br>"status": "NEW",<br>"oldValue": "CASH",<br>"newValue": "POS",<br>"details": null<br><br>}<br>],<br>"oldPrice": {<br>"value": 3.07,<br>"formattedValue": "€3,07"<br>},<br>"newPrice": {<br>"value": 10.68,<br>"formattedValue": "€10,68"<br>}<br>} |

| | |
|---|---|
| ADD_EXTRA | Add extra |
| DELETE_EXTRA | Delete extra |
| CHANGE_EXTRA | Change extra |
| MANUAL_PRICE | Manual price |
| ADD_PASSENGER | Add passenger |
| DELETE_PASSENGER | Delete passenger |
| ADD_ENROUTE_STOP | Add enroute stop |
| DELETE_ENROUTE_STOP | Delete enroute stop |
| CHANGE_ENROUTE_STOP | Change enroute stop |
| CHANGE_WAITING_TIME | Change waiting time |
| OTHER | Other |

### DriverChangeStatus

| Value | Description |
|---|---|
| NEW | New (not processed by the client or the operator) |
| CONFIRMED | Confirmed by the client |
| REJECTED | Rejected by the client |

## Possible response codes:

| Code | Description |
|---|---|
| DATA_NOT_FOUND | Unable to find the specified booking |
| INVALID_ACCOUNT_INFO | Authenticated user details do not match booking details |
| INVALID_REQUEST | Booking ID cannot be null |

# 2. Confirm driver changes

A user can reject the changes confirmed before (or confirm the rejected ones).

## URL: /sherbook/driverChanges/confirmation

## Request:

### DriverChangesConfirmationRequest

| Parameter | Type | Description |
|---|---|---|
| customerType | CustomerType | Customer type |
| ids | List<UUID> | IDs of changes to be confirmed or rejected |

## Request example:

| Request | Response |
|---|---|
| {"ids":["34f3e7f3-7a0f-30d0-6d11-1b97e3cf4928"],<br><br>"confirm":"true",<br>"customerType":"RETAIL"} | {<br>"status": "OK",<br>"errorMessage": null<br>} |

| | | |
|---|---|---|
| confirm | boolean | Indicates if changes should be confirmed (true) or rejected (false) |

### Response:

Cf. BaseResponse

### Possible response codes:

| Code | Description |
|---|---|
| DATA_NOT_FOUND | One of the specified driver changes is not found by ID |
| INVALID_ACCOUNT_INFO | Authenticated user details do not match booking details |
| INVALID_REQUEST | All changes should relate to the same booking |
| INVALID_REQUEST | Confirmation of driver changes is not allowed for the booking payment type |

## 3. Confirm driver changes using a token

### URL: /driverChanges/confirmation/token

Confirm or reject driver changes made before the token is created.
This URL can be requested anonymously.

### Request:

### DriverChangesConfirmationByTokenRequest

| Parameter | Type | Description |
|---|---|---|
| tokenId | String | Token (for example recieved by email) |
| confirm | boolean | Indicates if changes should be confirmed (true) or rejected (false) |

### Response:

#### BaseResponse

| Parameter | Type | Description |
|---|---|---|
| status | ResponseStatus | Status of the response returned from the server |
| errorMessage | String | Contains a user-friendly result of confirmation in case of the 'OK' status<br><br>Contains description of the error occurred in case of the 'OK' status |

### Request example:

| Request | Response |
|---|---|
| {"tokenId": "d4f49209-696e-4425-9c8d-5b4984f9220d", "confirm":"false"} | { "status": "OK", "errorMessage": "The changes made by the driver has been successfully rejected for booking 123456" } |

### Possible response codes:

| Code | Description |
| --- | --- |
| DATA_NOT_FOUND | Invalid token (not found, expired, etc) |
| INVALID_REQUEST | Token is not specified |

# 38. Chat with a driver

## 1. Send a message to a driver

### URL: /chat/send

### Request:

#### MessageToDriverRequest

| Parameter | Type | Description |
|---|---|---|
| message | String | Message |
| id | UUID | Booking ID |
| customerType | CustomerType | Customer type |

### Response:

#### MessageToDriverResponse

| Parameter | Type | Description |
|---|---|---|
| timestamp | Date | Date of the message<br><br>**Note.** Date format is: 'dd/MM/yyyy HH:mm:ss'. |

### Possible response codes:

| Code | Description |
|---|---|
| DATA_NOT_FOUND | There is no driver allocated to your booking |
| DATA_NOT_FOUND | There is no valid contact data for your booking |
| INVALID_REQUEST | Chat with a driver is not supported |
| INVALID_REQUEST | Job ID cannot be null |
| INVALID_REQUEST | Please add a message text |

## 2. Load chat messages

Allows to load a chat message history. This information is also available in the /booking/load/active/full endpoint.

After loading driver messages are marked as 'received'. To mark it as 'read' the /chat/markRead endpoint should be used.

### URL: /chat/load

### Request:

### LoadChatRequest

| Parameter | Type | Description |
|---|---|---|
| onlyUnread | boolean | Indicates if only messages should be loaded if there are unread messages from the driver |
| lastMessageTimestamp | Date | Last loaded message date. Only new messages will be loaded. It should be equal to the last driver message timestamp<br><br>**Note.** Expected date format is: 'dd/MM/yyyy HH:mm:ss'. |
| id | UUID | Booking ID |
| customerType | CustomerType | Customer type |

## Response:

### LoadChatResponse

| Parameter | Type | Description |
|---|---|---|
| chat | List<ChatDto> | List of chat messages |

### ChatDto

| Parameter | Type | Description |
|---|---|---|
| actorName | String | Actor name (client) |
| driverName | String | Driver name or callsign |
| messageText | String | Message text |
| fromClientToDriver | boolean | Indicates if the direction is from the client to the driver |
| received | boolean | Indicates if the message has been received |
| wasRead | boolean | Indicates if the message has been read |
| timestamp | Date | Date of the message<br><br>**Note.** Date format is: 'dd/MM/yyyy HH:mm:ss'. |

## Possible response codes:

| Code | Description |
|---|---|
| DATA_NOT_FOUND | The booking %s has not been found |
| INVALID_ACCOUNT_INFO | You cannot change the account after booking has been started |
| INVALID_REQUEST | Chat with a driver is not supported |
| INVALID_REQUEST | Job ID cannot be null |

# 3. Mark driver messages as 'read'

## URL: /chat/markRead

## Request:

### LoadChatRequest

| Parameter | Type | Description |
|---|---|---|
| id | UUID | Booking ID |
| customerType | CustomerType | Customer type |

## Response:

Cf. BaseResponse

### Possible response codes:

| Code | Description |
|---|---|
| DATA_NOT_FOUND | The booking %s has not been found |
| INVALID_ACCOUNT_INFO | You cannot change the account after booking has been started |
| INVALID_REQUEST | Chat with a driver is not supported |
| INVALID_REQUEST | Job ID cannot be null |

# 39. Quotation Calculation

The quotation request is a composition of delay, journey time and price requests - it calculates those altogether. So read the descriptions of those requests as they are applicable for this one as well. The single difference is the quotation request requires a job contact..

The primary purpose of this request - minimize requests to the server, when calculation of all parameters (delay, journey time, price) is needed.

## URL: /calculate/quotation

## Request:

### QuotationRequest

| Parameter | Type | Description |
|---|---|---|
| **customerType** | CustomerType | Customer type |
| **operationType** | BookingOperationType | Booking operation type |
| **requestId** | UUID | Booking request identifier |
| usedJobId | UUID | Used booking identifier |
| **service** | BookingServiceDto | Selected service |
| **paymentType** | BookingPaymentTypeDto | Selected payment type |
| **asap** | boolean | Indicates if booking is ASAP |
| date | Date | Booking date |
| **stops** | List<StopDto> | Booking stops list |
| **routeInfo** | RouteInfo | Route info |
| instructions | List<SpecialInstructionDto> | Filled special instructions |
| loyaltyCard | LoyaltyCardDto | Selected loyalty card |
| voucher | VoucherDto | Selected electronic voucher |
| promoCode | String | Selected promo code |
| creditCard | CreditCardDto | Credit Card information (can affect the price) |
| contact | ContactDto | Job contact |
| passengers | List<ContactDto> | Job passengers |

## Response:

### QuotationResponse

| Parameter | Type | Description |
|---|---|---|
| price | PriceDto | Calculated price |
| extPrice | PriceDto | Calculated price in alternative currency |
| discount | DiscountInfoDto | Discount information |
| extDiscount | DiscountInfoDto | Discount information converted to alternative currency |
| asDirectedMinutesRounding | AsDirectedMinutesRoundingInfoDto | Rounding details |

| | | |
|---|---|---|
| duration | Integer | Journey duration, in minutes |
| durationInTraffic | Integer | Journey duration with traffic consideration, in minutes |
| delayDto | DelayDto | Delay and Arrival delay information |
| offerService | ServiceDto | Offer service suggested in cases when pickup circuit is locked |

### DelayDto

| Parameter | Type | Description |
|---|---|---|
| delay | Integer | Delay |

### ArrivalDelayDto (extends DelayDto)

| Parameter | Type | Description |
|---|---|---|
| date | Date | Date |
| asap | boolean | Indicates if booking is ASAP based on the flight information |

## Possible response codes:

| Value | Description |
|---|---|
| OK | Price successfully calculated |
| INVALID_REQUEST | Request data not filled properly |
| INVALID_CUSTOMER_TYPE | Customer type is not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| INCORRECT_REQUEST_IDENTIFIER | Specified request identifier already used in another booking |
| SERVICE_RESTRICTION_FAILED | At least one of service restrictions is not matched |
| PICKUP_CIRCUIT_LOCKED | Specified address area is locked for ASAP bookings |
| INVALID_OPERATIONAL_AREA | Specified service is not available in selected pickup zone |
| PAYMENT_TYPE_NOT_SUPPORTED | Selected payment type is not supported |
| INVALID_VOUCHER | Specified voucher is not found |
| VOUCHER_NOT_ACTIVE | Specified voucher is not active |
| INVALID_LOYALTY_CARD | Loyalty card is invalid |
| INVALID_PROMO_CODE | Promo code is invalid |
| INSTRUCTION_VALIDATION_FAILED | Specified special instructions values are not valid |
| INVALID_ADDRESS_DATA | Unable to assemble stop address from specified data |

| | |
|---|---|
| INVALID_COORDINATES | Pickup stop coordinates are not in valid area |
| INVALID_ROUTE_INFO | Attempted to create "Wait and return", "Destination unknown" or "As directed" booking while it's not supported by selected service |
| PRICE_NOT_CALCULATED | Price not calculated for some internal reason |
| BOOKING_NOT_AMENDABLE | Booking couldn't be amended |
| BOOKING_NOT_REPEATABLE | Booking couldn't be repeated |
| DATA_NOT_FOUND | Unable to found booking related data by identifier specified in request |
| INVALID_ACCOUNT_DATA | Specified service or payment type are not available to user |
| ASAP_ONLY_SERVICE | Only ASAP bookings are available for the service |

## Request example:

| Request | Response |
|---|---|
| | {<br>"status":"OK",<br>"errorMessage":null,<br>"price":{<br>"net":{<br>"value":14.00,<br>"formattedValue":"<br>€14.00"<br>},<br>"taxTotal":{<br>"value":0.70,<br>"formattedValue":"€0.70"<br>},<br>"gross":{<br>"value":14.70,<br>"formattedValue":"<br>€14.70"<br>},<br>"discount":{<br>"value":0.00,<br>"formattedValue":"€0.00"<br>},<br>"details":{<br>}<br>},<br>"extPrice":null,<br>"discount":null,<br>"extDiscount":null,<br>"duration":100,<br>"durationInTraffic":100,<br>"delayDto": {<br>delay:15<br>}<br>} |

{
"customerType":"CORPORATE",
"operationType":"CREATE",
"requestId": "1cbca887-0398-4e14-ae64-18bac383c0f8",
"usedJobId":null,
"service":{
"customerType":"CORPORATE",
"id":"65bff4bc-6fb8-4a1f-a31e-aa2f0ff36bdc",
"caption":"Peugeot 508",
"description":"Peugeot 508"
},
"paymentType":{
"customerType":"CORPORATE",
"name":"Invoice",
"code":"INVOICE"
},
"asap":true,
"date":null,
"stops":[
{
"id":"1259c4ff-076d-4959-311a-385796490daf",
"address":{
"addressId":"1308124c-81af-1db4-b88e-
7754336518fe",
"caption":"91 Rue de la Verrerie, 75004 Paris",
"latitude":48.8586556,
"longitude":2.3507153,
"source":"AB",
"data":null
},
"notes":null,
"arrivalDetails":null
},
{
"id":null,
"address":{
"addressId":null,
"caption":"8 Rue Saint-Bon, 75004 Paris",
"latitude":48.8584055,
"longitude":2.35076,
"source":"GG",
"data":"QURSe0ZMRDF…IDc1MDA0IFBhcmlzfX0=",
"fields":null
},
"notes":null,
"arrivalDetails":null
}
],
"routeInfo":{
"waitAndReturn":false,
"destinationUnknown":false,
"asDirected":false,
"asDirectedMinutes":0
},
"instructions":null,
"voucher":null,
"loyaltyCard":null,
"promoCode":null,
"contact": {
"name": "John",
"phone": "+33140759783",
"email": "john@doe.com"
"customerType": "CORPORATE"
}
}

# 40. Cruise terminals and berths

## 1. Load cruise terminals

### URL: /addresses/shipTerminals

### Request:

Cf. CustomerDataRequest.

### Response:

#### AddressSearchResponse

| Parameter | Type | Description |
|---|---|---|
| addresses | List<AddressDto> | Loaded cruise terminals |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Cruise terminals have been successfully loaded |

### Request example:

| Request | Response |
|---|---|
| | {<br>"status": "OK",<br>"errorMessage": null,<br>"addresses": [<br>{<br>"addressId": "59f268ab-d9cd-1759-e00a-dc9f20bf38c4",<br>"caption": "Mayflower Cruise Terminal",<br>"latitude": 50.90492099999999,<br>"longitude": -1.428824,<br>"source": "SHIP_TERMINAL",<br>"data": null,<br>"fields": null,<br>"restrictions": null,<br>"parentId": null,<br>"customerType": null<br>}<br>]<br>} |

## 2. Load a cruise terminal by ID

### URL: /addresses/shipTerminals/loadById

### Request:

#### AddressRequest

| Parameter | Type | Description |
|---|---|---|
| addressId | UUID | Entity ID |

### Response:

#### AddressResponse

| Parameter | Type | Description |
|---|---|---|
| address | AddressDto | Loaded entity |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Cruise terminal has been successfully loaded |
| INVALID_REQUEST | Cruise terminal identifier is required |
| DATA_NOT_FOUND | Cruise terminal is not found |

## 3. Load cruise terminal berths

### URL: /addresses/shipTerminals/berths

### Request:

#### RelatedAddressesRequest

| Parameter | Type | Description |
|---|---|---|
| **parentId** | UUID | Related address identifier. For this case – a cruise terminal identifier |

### Response:

Cf. 6. Searching addresses#_AddressSearchResponse.

### Possible response codes:

| Value | Description |
|---|---|
| OK | Berths have been successfully loaded |
| INVALID_REQUEST | Cruise terminal identifier is required |

### Request example:

| Request | Response |
|---|---|

| | |
|---|---|
| {<br><br>"parentId" : "59f268ab-d9cd-1759-e00a-dc9f20bf38c4"<br><br>} | {<br>"status": "OK",<br>"errorMessage": null,<br>"addresses": [<br>{<br>"addressId": "66a22787-3e95-0156-ec20-e1b77ac28a2d",<br>"caption": "Berth 106",<br>"latitude": 50.90492099999999,<br>"longitude": -1.428824,<br>"source": "BERTH",<br>"data": null,<br>"fields": null,<br>"restrictions": null,<br>"parentId": null<br>}<br>]<br>} |

## 4. Load a berth by ID

### URL: /addresses/shipTerminals/berths/loadById

### Request:

#### AddressRequest

| Parameter | Type | Description |
|---|---|---|
| addressId | UUID | Entity ID |

### Response:

#### AddressResponse

| Parameter | Type | Description |
|---|---|---|
| address | Data transfer objects#_AddressDto | Loaded entity |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Berth has been successfully loaded |
| INVALID_REQUEST | Berth identifier is required |
| DATA_NOT_FOUND | Berth is not found |

## 5. Load cruise terminal hierarchy

### URL: /addresses/shipTerminals/full

### Request:

Cf. Basic requests and responses#_Base_request_to.

### Response:

Cf. Basic requests and responses#_Address_hierarchy_response.

## Possible response codes:

| Value | Description |
|---|---|
| OK | Cruise terminals have been successfully loaded |
| INVALID_REQUEST | List of customer types is required |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

## Request example:

| Request | Response |
|---|---|
| {<br>"lastUpdateDate":<br>null<br>} | {<br>"status": "OK",<br>"errorMessage": null,<br>"lastUpdateDate": "11/04/2017 15:11",<br>"deleted": [],<br>"addresses": [<br>{<br>"addressId": "59f268ab-d9cd-1759-e00a-dc9f20bf38c4",<br>"caption": "Mayflower Cruise Terminal",<br>"latitude": 50.90492099999999,<br>"longitude": -1.428824,<br>"source": "SHIP_TERMINAL",<br>"data": null,<br>"fields": null,<br>"restrictions": null,<br>"parentId": null,<br>"customerType": null<br>},<br>{<br>"addressId": "66a22787-3e95-0156-ec20-e1b77ac28a2d",<br>"caption": "Berth 106",<br>"latitude": 50.90492099999999,<br>"longitude": -1.428824,<br>"source": "BERTH",<br>"data": null,<br>"fields": null,<br>"restrictions": null,<br>"parentId": null,<br>"customerType": null<br>}<br>]<br>} |

# 41. Supervisors

## 1. Approve or reject booking

### URL:/supervisor/confirmation

### Request:

| Parameter | Type | Description |
|---|---|---|
| **id** | String | Booking id |
| **confirm** | boolean | True in case of booking approval and false in case of rejection |

### Response:

Cf. BaseResponse

### Possible response codes:

| Value | Description |
|---|---|
| OK | Booking has been successfully approved or rejected |
| INVALID_REQUEST | Booking is not found |
| INCORRECT_DATA | If booking status is not 'Waiting for confirmation' |

## 2. Approve or reject booking by token

This is anonymous version of previous request.

### URL:/supervisor/confirmation/token

### Request:

| Parameter | Type | Description |
|---|---|---|
| **id** | String | Booking token |
| **confirm** | boolean | True in case of booking approval and false in case of rejection |

### Response:

Cf. BaseResponse

### Possible response codes:

| Value | Description |
|---|---|
| OK | Booking has been successfully approved or rejected |
| INVALID_REQUEST | Booking is not found |
| INCORRECT_DATA | If token is invalid or booking status is not 'Waiting for confirmation' |

# 3.Get list of pending approval job

## URL:/supervisor/pendingApproval/list

## Request:

| Parameter | Type | Description |
|---|---|---|
| limit | Integer | Limit for loading |
| offset | Integer | Offset for loading |
| filter | BookingFilterDto | Filter |
| customerType | CustomerType | Customer type |

## Response:

| Parameter | Type | Description |
|---|---|---|
| bookings | List<SupervisorBookingDto> | Booking list |
| version | Long | Booking approval list version |

BookingFilterDto

| Parameter | Type | Description |
|---|---|---|
| number | String | Job number |
| client | String | Contact name |
| serviceId | UUID | Service id |
| references | String | Reference text |
| address | String | Pickup or drop address |
| from | Date | Job date from |
| to | Date | Job date to |
| callerName | String | Caller name |

## SupervisorBookingDto

SupervisorBookingDto the same as BookingListDto with several additional fiels.

| Parameter | Type | Description |
|---|---|---|
| price | PriceDto | Job price |
| contact | ContactDto | Job contact |

## Possible response codes:

| Value | Description |
|---|---|
| OK | Bookings have been successfully loaded |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

## 4.Count pending approval jobs

### URL:/supervisor/pendingApproval/count

### Request:

| Parameter | Type | Description |
|---|---|---|
| limit | Integer | Limit for loading |
| offset | Integer | Offset for loading |
| filter | BookingFilterDto | Filter |
| customerType | CustomerType | Customer type |

### Response:

| Parameter | Type | Description |
|---|---|---|
| count | int | count |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Bookings have been successfully counted |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

## 5.Get version of pending approval job list

### URL:/supervisor/pendingApproval/version

### Request:

| Parameter | Type | Description |
|---|---|---|
| limit | Integer | Limit for loading |
| offset | Integer | Offset for loading |
| filter | BookingFilterDto | Filter |
| customerType | CustomerType | Customer type |

### Response:

| Parameter | Type | Description |
|---|---|---|
| latestJob | Long | version |

### Possible response codes:

| Value | Description |
|---|---|

| | |
|---|---|
| OK | Bookings have been successfully counted |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

# 42. Terms&Conditions and Privacy Policy acceptance

## 1. Check Terms&Conditions and Privacy Policy acceptance

It's the POST request with no parameters. The request will be processed only for retail user.

### URL: /policy/agreement/check

### Response:

#### PolicyAgreementResponse

| Parameter | Type | Description |
|---|---|---|
| accepted | Boolean | True if user accepted policy agreement and false if didn't |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Agreement acceptence data has successfully loaded |
| INVALID_RETAIL_REQUEST | User is logged in as corporate |

## 2. Accept Terms&Conditions and Privacy Policy

It's the POST request with no parameters. The request will be processed only for retail user.

### URL: /policy/agreement/accept

### Response:

Cf. BaseResponse.

### Possible response codes:

| Value | Description |
|---|---|
| OK | Agreement has successfully accepted |
| INVALID_RETAIL_REQUEST | User is logged in as corporate |

# 43. PayPal payments

## 1.Add account

### URL:/paypal/account/save

### Request:

Empty request

### Response:

#### SavePayPalAccountResponse

| Parameter | Type | Description |
|---|---|---|
| redirectInfo | RedirectDto | Redirect information |

#### Possible response codes:

| Value | Description |
|---|---|
| REDIRECT_REQUIRED | Need to redirect user to the specified url |
| PAYPAL_OPERATION_FAILED | Error on adding a paypal account |

## 2.Complete account creation

### URL:/paypal/account/complete

### Request:

#### PayPalFeedbackRequest

| Parameter | Type | Description |
|---|---|---|
| customerType | CustomerType | Customer type |
| feedback | PayPalFeedbackDto | Result of account creation from PayPal |

### Response:

#### PayPalResponse

| Parameter | Type | Description |
|---|---|---|
| account | PayPalAccountDto | Created account |

#### Possible response codes:

| Value | Description |
|---|---|
| OK | Account successfully created |

| | |
|---|---|
| INVALID_REQUEST | Missed PayPal feedback |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| INVALID_PAYPAL_ACCOUNT | Specified PayPal account is not available |
| PAYPAL_OPERATION_FAILED | Error during account creation |

## 3.Load accounts

### URL:/paypal/account/load

### Request:

#### CustomerDataRequest

| Parameter | Type | Description |
|---|---|---|
| customerTypes | List<CustomerType> | Customer types |

#### Response:

#### LoadPayPalAccountResponse

| Parameter | Type | Description |
|---|---|---|
| accounts | List<PayPalAccountDto> | Loaded accounts |

#### Possible response codes:

| Value | Description |
|---|---|
| OK | Account successfully created |
| INVALID_REQUEST | Missed PayPal feedback |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |

## 4.Remove account

### URL:/paypal/account/delete

### Request:

#### DeletePayPalAccountRequest

| Parameter | Type | Description |
|---|---|---|
| account | List<PayPalAccountDto> | Account to be removed |

### Response:

Cf. BaseResponse.

### Possible response codes:

| Value | Description |
|---|---|
| OK | Account successfully deleted |
| INVALID_REQUEST | Missed PayPal feedback |
| INVALID_CUSTOMER_TYPE | Customer types are not specified |
| INVALID_CORPORATE_REQUEST | Corporate user is not properly authenticated |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_ANONYMOUS_REQUEST | Anonymous user is not properly authenticated |
| INVALID_PAYPAL_ACCOUNT | Account can't be deleted |

## 5.Complete payment

## URL:/paypal/payment/complete

## Request:

### PayPalFeedbackRequest

| Parameter | Type | Description |
|---|---|---|
| customerType | CustomerType | Customer type |
| feedback | PayPalFeedbackDto | Result of account creation from PayPal |

### Response:

**BookJobResponse**

### Possible response codes:

| Value | Description |
|---|---|
| OK | Payment successfully completed |
| INVALID_REQUEST | Missed PayPal feedback |
| INVALID_PAYPAL_ACCOUNT | Account isn't available |
| PAYPAL_OPERATION_FAILED | Error on paypal account completion |
| The same as BookJobResponse codes | |

### PayPalAccountDto

| Parameter | Type | Description |
|---|---|---|

| | | |
|---|---|---|
| customerType | CustomerType | Customer type |
| id | UUID | Account id |
| email | String | Email |
| name | String | Account name |
| business | boolean | Business account |

### PayPalFeedbackDto

| Parameter | Type | Description |
|---|---|---|
| transactionId | UUID | Transaction id |
| token | String | Token |
| success | boolean | Completion success |

# 44. Route and Duration calculation

Uses Google Directions API

Calculates route parameters for specified map points.

Group: Calculation

- [x] Distance
- [x] Route
- [x] Duration

Request accepts list of map points. Requires at least 2 points for pickup and dropoff.

Latitude and longitude are required for every point.

**Request**

```
{
  "points": [
    {
      "latitude": 0,
      "longitude": 0
    },
    {
      "latitude": 0,
      "longitude": 0
    }
  ]
}
```

## Duration

### /calculate/duration

Returns duration information in *seconds* with and without traffic.

**Response**

```
{
  "duration": 0,
  "durationInTraffic": 0,
  "errorMessage": "string",
  "status": "OK"
}
```

## Route

### /calculate/route

Returns route map points and distance in *meters* along with infomation about duration.

**Response**


```
{
  "distance": 0,
  "duration": 0,
  "durationInTraffic": 0,
  "errorMessage": "string",
  "route": [
    {
      "latitude": 0,
      "longitude": 0
    }
  ],
  "status": "OK"
}
```

# 45. Referral program

**Referral program is used for retail users only**

## 1. Load Referral Code

### URL: /referralCode/load

### Request:

Empty request.

### Response:

#### LoadReferralCodeResponse

| Parameter | Type | Description |
|---|---|---|
| referralCode | ReferralCodeDto | Generated or already existing referral code.<br><br>Value can be null in case when referral program is disabled in Sherlock |

#### ReferralCodeDto

| Parameter | Type | Description |
|---|---|---|
| id | UUID | Referral code identifier |
| code | String | Code to share with other customers |
| status | ReferralCodeStatus | Referral code status |
| expirationDate | Date | Expiration date |
| usageCount | Integer | Shows how many times referral code **was** used by other customers |
| maxUsageCount | Integer | Shows how many times referral code **can be** used by other customers |
| receivedVouchersCount | Long | Shows how many vouchers were generated for the referral code |
| message | String | Message to be used by customer to share owned referral code with other customers |
| subject | String | Title of message to be used by customer to share owned referral code with other customers |

#### ReferraalCodeStatus

| Value | Description |
|---|---|
| ACTIVE("ACTIVE") | Referral code is active and **can** be shared with other taxi clients |
| USED("USED") | Referral code has been used maximum count of times and **cannot** be shared with other taxi clients |
| BANNED("BANNED") | Referral code has been banned and **cannot** be shared with other taxi clients |
| EXPIRED("EXPIRED") | Referral code has been expired and **cannot** be shared with other taxi clients |

### Possible response codes:

| Value | Description |
|---|---|
| OK | Referral code have been successfully loaded |

| | |
|---|---|
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |

### Request example:

| Request | Response |
|---|---|
| {} | {<br>"status": "OK",<br>"errorMessage": null,<br>"referralCode":<br>{<br>"id": "e562d03b-7aa9-b1fb-8b31-721aaf2dc13f",<br>"code": "fnwd7y",<br>"status": "ACTIVE",<br>"expirationDate": null,<br>"usageCount": 3,<br>"maxUsageCount": 4,<br>"receivedVouchersCount": 2,<br>"message": "Get your first free ride with Sherlock using my promo code REFERRAL_CODE. Download apps iPhone https://... Android https://..."<br>}<br>} |

## 2. Check whether user can apply referral code

### URL: /referralCode/check

### Request:

Empty request.

## Response:

Cf. BaseResponse

#### Possible response codes:

| Value | Description |
|---|---|
| OK | Referral code have been successfully loaded |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| REFERRAL_CODE_ALREADY_USED | Referral code is already assigned to customer profile |
| REFERRAL_PROGRAM_DISABLED | Referral program is disabled in Sherlock |
| REFERRAL_PROGRAM_UNAVAILABLE | Retail user has previously completed or cancelled on arrival bookings |

### Request example:

| Request | Response |
|---|---|
| {} | {<br>"status": "OK",<br>"errorMessage": null<br>} |

| | |
|---|---|
| {} | {<br>"status": "REFERRAL_CODE_ALREADY_USED",<br>"errorMessage": "Referral code is already assigned to your profile"<br>} |

## 3. Apply referral code after registration

### URL: /referralCode/apply

### Request:

#### ReferralCodeRequest

| Parameter | Type | Description |
|---|---|---|
| referralCode | String | Referral code to be applied to the retail profile |

### Response:

#### ReferralCodeResponse

| Parameter | Type | Description |
|---|---|---|
| message | String | Message saying that referral code is successfully applied. Should be shown to customer. |

#### Possible response codes:

| Value | Description |
|---|---|
| OK | Referral code have been successfully loaded |
| INVALID_RETAIL_REQUEST | Retail user is not properly authenticated |
| INVALID_REQUEST | Referral code is not specified |
| REFERRAL_CODE_ALREADY_USED | Referral code is already assigned to customer profile |
| REFERRAL_CODE_NOT_FOUND | Referral code is not found in the system |
| REFERRAL_CODE_BANNED | Referral code is banned (optional: reason) |
| REFERRAL_CODE_USAGE_LIMIT | Referral code usage limit exceeded |
| REFERRAL_CODE_EXPIRED | Referral code is already expired |
| REFERRAL_CODE_OWNER | User is owner of specified referral code |
| REFERRAL_PROGRAM_DISABLED | Referral program is disabled in Sherlock |
| REFERRAL_PROGRAM_UNAVAILABLE | Retail user has previously completed or cancelled on arrival bookings |

### Request example:

| Request | Response |
|---|---|
| {<br>"referralCode":"<br>BJNS34gf1"<br>} | {<br>"status": "OK",<br>"errorMessage": null,<br>"message":"You have been accrued with a bonus in the amount of 1£. Your voucher number is Welcome281119-GMQW1F."<br>} |

| | |
|---|---|
| {<br>"referralCode":"<br>BJNS34gf2"<br>} | {<br>"status": "REFERRAL_CODE_EXPIRED",<br>"errorMessage": "Referral code BJNS34gf2 already expired",<br>"message":null<br>} |
| {<br>"referralCode":"<br>BJNS34gf3"<br>} | {<br>"status": "REFERRAL_CODE_NOT_FOUND",<br>"errorMessage": "Referral code BJNS34gf3 not found",<br>"message":null<br>} |

# 46. Off-Session Payments

**Off-Session Payments (OSP) process is performed:**

1. **From a separate sherbook client (OSP page)**
2. **In anonymous mode without login**

## 1. Load OSP details

### URL: /osp/load

### Request:

#### TokenRequest

| Parameter | Type | Description |
|---|---|---|
| tokenId | String | Token identifier (can be received via email) |

### Response:

#### OffSessionPaymentDataResponse

| Parameter | Type | Description |
|---|---|---|
| data | OffSessionPaymentData | Data to show to client |

#### OffSessionPaymentData

| Parameter | Type | Description |
|---|---|---|
| jobItem | OffSessionJobPaymentItem | Job data to show to client |
| invoiceItem | OffSessionInvoicePaymentItem | Invoice data to show to client |
| creditCard | ShortCreditCardDto | Credit card used during initial transaction |
| amount | PriceField | Amount that client have to pay during OSP |
| convertedAmount | PriceField | Amount converted to the alternative currency |

#### OffSessionJobPaymentItem

| Parameter | Type | Description |
|---|---|---|
| id | UUID | Booking identifier |
| number | String | Booking number |
| jobDate | Date | Booking date |
| pickup | String | Formatted first address stop |
| drop | String | Formatted last address stop |
| status | BookingStatus | Booking status |
| driver | DriverDto | Driver information |
| routeInfo | RouteInfo | Route information |
| asap | boolean | Indicates if the booking is ASAP |

| | | |
|---|---|---|
| mainPassengerName | String | Name of the main booking contact/passenger |
| numberOfPassengers | Integer | Number of booking passengers |
| account | String | Customer account name |
| price | PriceDto | Detailed price information |
| convertedPrice | PriceDto | Price converted to the alternative currency |

### OffSessionInvoicePaymentItem

| Parameter | Type | Description |
|---|---|---|
| invoiceNumber | String | Invoice number |
| invoiceDate | Date | Invoice date |
| accountName | String | Customer account number |
| accountNumber | String | Customer account name |
| individualName | String | Individual name |
| jobsCount | Integer | Number of jobs included in the invoice |
| jobs | List<OffSessionJobPaymentItem> | Information about jobs included in the invoice |

**Possible response codes:**

| Value | Description |
|---|---|
| OK | OSP data have been successfully load |
| INVALID_REQUEST | Token identifier is not filled properly |
| DATA_NOT_FOUND | Payment related data is not found by specified token identifier |

## Request example:

| Request | Response |
|---|---|
| {"tokenId":"56f30d7c-1025-fc77-42d8-dea59e65b622"} | {<br>"status": "OK",<br>"errorMessage": null,<br>"data": {<br>"items": [<br>{<br>"id": "c80e8a4f-a2cb-6000-2c9d-a4fd003b3141",<br>"number": "2120006",<br>"jobDate": "10/09/2019 14:29",<br>"status": 40,<br>"driver": {<br>"name": "HAMID REZA GOLZARI HD0743 Test",<br>"phone": "99901052011478",<br>"carRegNumber": "NB07CAB 160",<br>"carModel": "Tx4",<br>"vehicleManufacturer": "Hackney",<br>"latitude": null,<br>"longitude": null,<br>"callsign": "150",<br>"vehicleColor": "Black",<br>"photoId": null<br>},<br>"routeInfo": {<br>"waitAndReturn": false, |

"destinationUnknown": false,
"asDirected": false,
"asDirectedMinutes": null
},
"asap": true,
"service": {
"customerType": "ANONYMOUS",
"id": "5d379279-c4f1-cd78-a282-612978cba05d",
"caption": "Any Vehicle",
"description": null,
"code": "A",
"delivery": false,
"vehicleModelType": {
"name": "Any vehicle",
"code": "ANY_VEHICLE"
}
},
"pickup": "72 A4114, CV1 4AQ Coventry",
"drop": "123 Saint Nicholas Street, CV1 4FD Coventry",
"mainPassengerName": "DK",
"numberOfPassengers": 1,
"account": "Cash or Credit Card",
"price": {
"net": {
"value": 4.03,
"formattedValue": "£4.03",
"currency": "GBP",
"currencyFormat": "£%s"
},
"taxTotal": {
"value": 0,
"formattedValue": "£0.00",
"currency": "GBP",
"currencyFormat": "£%s"
},
"gross": {
"value": 4.03,
"formattedValue": "£4.03",
"currency": "GBP",
"currencyFormat": "£%s"
},
"discount": {
"value": 0.00,
"formattedValue": "£0.00",
"currency": "GBP",
"currencyFormat": "£%s"
},
"details": null,
"priceDetails": null,
"prefix": "Set Fare"
},
"convertedPrice": null
}
],
"creditCard": {
"customerType": "ANONYMOUS",
"id": null,
"number": "****-****-****-1234",
"type": "Mastercard"
},
"amount":{
"value": 4.03,
"formattedValue": "£4.03",
"currency": "GBP",
"currencyFormat": "£%s"

| | |
|---|---|
| | },<br>"convertedAmount":null<br><br>} |

## 2. Start OSP process

### URL: /osp/start

### Request:

#### TokenRequest

| Parameter | Type | Description |
|---|---|---|
| tokenId | String | Token identifier (can be received via email) |

#### Response:

#### OffSessionPaymentResponse

| Parameter | Type | Description |
|---|---|---|
| redirectInfo | RedirectDto | Information about the client redirection to the 3D-party site to validate/confirm credit card details.<br><br>Empty if redirect is not required, which usually means that payment successfully processed. |

#### Possible response codes:

| Value | Description |
|---|---|
| OK | Payment processed. No redirect required |
| REDIRECT_REQUIRED | Client should be redirected to the payment provider site |
| INVALID_REQUEST | Token identifier is not filled properly |
| DATA_NOT_FOUND | Payment related data is not found by specified token identifier |
| CREDIT_CARD_PAYMENT_FAILED | Unable to perform a payment with the specified credit card |

### Request example:

| Request | Response |
|---|---|
| {"tokenId":"56f30d7c-1025-fc77-42d8-dea59e65b622"} | {<br>"status": "OK",<br>"errorMessage": null<br>} |

| | |
|---|---|
| {"tokenId":"56f30d7c-1025-fc77-42d8-dea59e65b622"} | {<br>"status": "REDIRECT_REQUIRED",<br>"errorMessage": null,<br>"redirectDto": {<br>"url": "https://test.adyen.com/hpp/3d/validate.shtml",<br>"params": {<br>"MD":<br>"QUN1Z0xYOVKktm87z8fA6RAcxRP8Q5bjqIyGnZJibd55Nzq156HckITfs83Egb5aJfOupbJiKmMB8t1vDXEAcG",<br>"PaReq": "eNpVUu1uNI7z8X8eayii/gGTw60cg==",<br>"TermUrl": "http://localhost:8080/sherbook/osp/complete/02fc4753-8ad3-6399-d703-b9c3581e6bad"<br>},<br>"method": "POST",<br>"html": null<br>}<br>} |

## 3. Complete OSP process

This endpoint is usually called by Credit Card Provider after client has passed 3d-secure authorization process.

### URL: /osp/complete/{token}

### Request:

Token is passed by CCProvider as the path variable

### Response:

Redirect URL either to the success page or to the initial page depending on payment status.

Success page has next format: **http(s)://{ServerURL}/portal/static/success.html**

Initial page has next format: **http(s)://{ServerURL}/portal/static?token={token}&locale={locale}&ccPaymentFailed**

#### Possible response codes:

| Value | Description |
|---|---|
| OK | OSP successfully completed |
| INVALID_REQUEST | Token identifier is not filled properly |
| DATA_NOT_FOUND | Payment related data is not found by specified token identifier |